

เอกสารประกอบการเรียนวิชาคณิตศาสตร์ดิสครีต

(Handout for Discrete Mathematics)

Phaphontee Yamchote

Information System for Digital Business, Faculty of Business Administration

Southeast Asia University

Update at 8 เมษายน พ.ศ. 2569

สารบัญ

I	พื้นฐานการพิสูจน์และการให้เหตุผลทางคณิตศาสตร์	1
1	Mathematics as a Language for Computer Science	3
1.1	คณิตศาสตร์ในฐานะภาษา	3
1.2	ประพจน์และประโยคเปิด	4
1.2.1	ประพจน์	4
1.2.2	Predicate หรือประโยคเปิด	4
1.2.3	จากประโยคเปิดสู่ประพจน์	5
1.3	เอกภพสัมพัทธ์และข้อตกลงพื้นฐาน	5
1.4	การพิสูจน์คืออะไร	6
1.4.1	ทำไมการผ่าน test cases ไม่ใช่การพิสูจน์	6
1.5	นิยามพื้นฐานที่ใช้ฝึกพิสูจน์	7
1.6	โครงสร้างมาตรฐานของการพิสูจน์แบบอิมพลีเคชัน	7
1.6.1	ตัวอย่างต้นแบบ: ถ้าจำนวนคี่แล้วยกกำลังสองยังคงเป็นคี่	8
1.6.2	อ่านพิสูจน์อย่างนักเขียนพิสูจน์	8
1.6.3	อีกหนึ่งตัวอย่างสั้น: ถ้าจำนวนคู่แล้วยกกำลังสองยังคงเป็นคู่	9
1.7	ตัวอย่างค้านและการปฏิเสธข้อความแบบ ``สำหรับทุก"	9
1.8	การเขียนพิสูจน์ให้เหมือนหนังสือ มากกว่าเหมือนแบบเดิมคำ	9
1.9	สรุปท้ายบท	10
2	ตรรกศาสตร์ประพจน์และโครงสร้างของการพิสูจน์	13
2.1	ตัวเชื่อมตรรกะและสูตรตรรกะ	13
2.2	อิมพลีเคชัน: ความหมายของข้อความรูป $P \rightarrow Q$	14
2.2.1	อิมพลีเคชันในฐานะคำสัญญา	14
2.2.2	อิมพลีเคชันเป็นเท็จได้เมื่อใด	14
2.2.3	ความจริงแบบว่าง	15
2.2.4	ถ้อยคำที่มักแปลผิด	15
2.3	จากรูปประโยครูปแบบการพิสูจน์	16
2.4	สมมูลตรรกะที่มีประโยชน์ต่อการพิสูจน์	17

2.4.1	Contrapositive	17
2.4.2	De Morgan	18
2.4.3	อิมพลีเคชันในรูปดิสจังก์ชัน	18
2.5	การพิสูจน์แบบ contradiction	18
2.6	การใช้อิมพลีเคชันที่พิสูจน์แล้ว: Modus Ponens	19
2.7	ดิสจังก์ชันและการพิสูจน์แบบแยกกรณี	20
2.7.1	ความหมายของ $P \vee Q$	20
2.7.2	การพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง	20
2.7.3	การใช้ $P \vee Q$: proof by cases	21
2.8	ไบคอนดิชันนัลและการพิสูจน์สองทิศ	22
2.9	ข้อผิดพลาดที่พบบ่อยในการเขียนพิสูจน์	23
2.9.1	พิสูจน์ converse แทนที่จะพิสูจน์ข้อความเดิม	23
2.9.2	พิสูจน์ iff เพียงทิศเดียว	23
2.9.3	ใช้ proof by cases แบบไม่ครบทุกกรณี	23
2.9.4	พิสูจน์แบบ contradiction โดยสมมติผิดตัว	23
2.10	สรุป	23
3	ตัวบ่งปริมาณและแพตเทิร์นพื้นฐานของการพิสูจน์	27
3.1	ทบทวน: รูปของข้อความบอกวิธีตั้งสมมติฐาน	27
3.1.1	อิมพลีเคชัน	27
3.1.2	ไบคอนดิชันนัล	28
3.1.3	คำว่า "หรือ" ปรากฏในบทบาใด	29
3.2	ตัวบ่งปริมาณและเอกภพสัมพัทธ์	29
3.3	เมื่อเป้าหมายเป็น $\forall$: เลือกสมาชิกแบบ arbitrary	30
3.4	เมื่อเป้าหมายเป็น $\exists$: ระบุพยาน	31
3.5	หลายตัวบ่งปริมาณ: ลำดับมีผล	31
3.5.1	แนวคิดเชิงภาษา	32
3.5.2	ตัวอย่างเชิงจำนวน	32
3.5.3	ตัวอย่างเชิงระบบคอมพิวเตอร์	33
3.6	การปฏิเสธตัวบ่งปริมาณ	33
3.7	ตัวอย่างทฤษฎีบทสม $\forall \exists$	34
3.8	ตัวอย่างทฤษฎีบทแบบ $\exists \forall$	35
3.9	การหักล้างข้อความที่มีตัวบ่งปริมาณ	36
3.10	คลินิกข้อผิดพลาดที่พบบ่อย	37
3.10.1	สมมติผิดด้านของอิมพลีเคชัน	37
3.10.2	พิสูจน์ iff เพียงทิศเดียว	37
3.10.3	ใช้ contradiction แต่สมมติผิด	37

3.10.4	เลือกค่าคงที่ตัวเดียวแทนที่จะให้ขึ้นกับตัวแปรก่อนหน้า	38
3.11	ตัวอย่างเสริมเกี่ยวกับการแปลความหมาย	38
3.12	สรุปแพตเทิร์นของบทนี้	39
II	การแก้ปัญหาเชิงกระบวนการและการเวียนเกิด	41
4	แนวคิดพื้นฐานของการเวียนเกิด	43
4.1	Recursion	43
4.1.1	Recursion ในฐานะรูปแบบของการแก้ปัญหา	44
4.1.2	เปรียบเทียบกับการทำงานแบบวนซ้ำ	44
4.2	ตัวอย่างอันเครื่อง: การนิยามลำดับแบบเวียนเกิด	45
4.2.1	ลำดับเลขคณิต	45
4.2.2	ลำดับฟีโบนัคซี	45
4.3	Recursion ในฐานะขั้นตอนวิธี: Tower of Hanoi	46
4.3.1	กติกาของปัญหา	46
4.3.2	เริ่มจากกรณี 4 แผ่น	47
4.3.3	รูปแบบทั่วไปสำหรับ n แผ่น	47
4.3.4	จำนวนครั้งที่ต้องย้าย	47
4.4	อีกตัวอย่างหนึ่ง: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น	48
4.5	การนับจำนวนวิธีด้วยการแยกกรณี: จำนวนเซตย่อย	49
4.6	สรุปภาพรวมของการตั้งความสัมพันธ์เวียนเกิด	49
4.7	ข้อควรระวัง	50
4.7.1	มี recursive step แต่ไม่มี base case	50
4.7.2	ปัญหาไม่เล็กลงจริง	50
4.7.3	สูตรไม่สะท้อนโครงสร้างจริงของปัญหา	50
5	Mathematical Induction	53
5.1	Ordinary Induction	53
5.1.1	หลักการพื้นฐาน	53
5.1.2	สิ่งที่ $P(n)$ เป็น และสิ่งที่มันไม่ใช่	54
5.1.3	ข้อผิดพลาดที่พบบ่อย	55
5.2	ตัวอย่างแรก: ผลบวกของจำนวนเต็มตัวแรก	55
5.2.1	สิ่งที่ตัวอย่างนี้สอนเรา	56
5.2.2	ผลบวกกำลังสอง	56
5.3	อีกสองตัวอย่างของ ordinary induction	58
5.3.1	ผลบวกของจำนวนคี่	58
5.3.2	ตัวอย่างอสมการ	59

5.4	Induction กับสูตรปิดของลำดับที่นิยามแบบเวียนเกิด	60
5.4.1	ลำดับเลขคณิต	60
5.4.2	ลำดับเรขาคณิต	61
5.5	อีกตัวอย่างหนึ่งที่มีโครงสร้างมากขึ้น: การปูบอร์ดด้วย L-tromino	62
5.6	จาก ordinary induction สู่ structural induction	62
5.7	Structural Induction	63
5.7.1	หลักทั่วไป	63
5.7.2	การพิสูจน์ความเท่ากันของเซต $R = A$	63
5.7.3	ตัวอย่าง: เซตของกำลังของ 2	64
5.7.4	ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 จากกฎ $x \mapsto x + 5$	65
5.7.5	ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 ที่ปิดภายใต้การบวก	66
5.7.6	ตัวอย่าง: คุณสมบัติของสมาชิกทุกตัวในเซตที่สร้างแบบ recursion	68
5.7.7	ตัวอย่างกับ bit string	69
5.8	จาก structural induction สู่ strong induction	70
5.9	Strong Induction	70
5.9.1	หลักการ	70
5.9.2	เมื่อไรควรใช้ strong induction	71
5.9.3	ตัวอย่าง: recurrence ที่ย้อนสองขั้น	71
5.9.4	ตัวอย่าง: หักซื้อโกแลตบาร์	72
5.9.5	ตัวอย่าง: แพ็กเหรียญขนาด 4 และ 5	73
5.9.6	ตัวอย่างเพิ่ม: การแยกตัวประกอบเป็นจำนวนเฉพาะ	74
5.10	สรุปความสัมพันธ์ของสามแนวคิด	75
5.11	ความสมมูลกันของ ordinary and strong induction	75
5.11.1	Strong induction $\Rightarrow$ Ordinary induction	76
5.11.2	Ordinary induction $\Rightarrow$ Strong induction	76
5.11.3	แล้วเหตุใดเรายังแยก ordinary กับ strong induction ออกจากกัน?	78
5.11.4	ความสัมพันธ์กับ structural induction	78
5.12	บทสรุปของ chapter นี้	79
6	Recursive Correctness Proof	81
6.1	Specification, Termination, and Correctness	81
6.2	Divide-and-Conquer: Merge Sort	82
6.2.1	แนวคิด Divide--Conquer--Combine	82
6.2.2	นิยาม sorted และสเปกของ MergeSort	82
6.2.3	Pseudocode ของ MergeSort และ Merge	83
6.3	พิสูจน์ความถูกต้องของ Merge	83
6.4	พิสูจน์ความถูกต้องของ MergeSort	85

6.5	Recursive Binary Search	87
6.5.1	นิยามและสเปก	87
6.5.2	Pseudocode	88
6.5.3	Termination	88
6.5.4	Correctness	89
6.6	ตัวอย่าง recursive algorithm เพิ่มเติม	91
6.6.1	Power	91
6.6.2	SumList	92
6.7	Pattern Summary	94
7	Recurrent Relation	97
7.1	Introduction to Recurrence Relations	97
7.1.1	ตัวอย่างที่ 1: recurrence แบบง่าย	97
7.1.2	ตัวอย่างที่ 2: Merge Sort recurrence (กรณี n เป็นกำลังของ 2)	99
7.1.3	Plug-and-chug สำหรับ Merge Sort	100
7.2	Binary Search recurrence	102
7.2.1	การวิเคราะห์	102
7.2.2	Plug-and-chug	103
7.3	From Recurrence to Closed Form and Back Again	104
III	ทฤษฎีกราฟเบื้องต้น	107
IV	การนับเบื้องต้น	109

ภาค I

พื้นฐานการพิสูจน์และการให้เหตุผลทางคณิตศาสตร์

บทที่ 1

Mathematics as a Language for Computer Science

คณิตศาสตร์ในบริบทของวิทยาการคอมพิวเตอร์ไม่ได้ทำหน้าที่เป็นเพียงเครื่องมือสำหรับคำนวณ แต่ทำหน้าที่เป็น *ภาษา* สำหรับอธิบายโครงสร้าง ความสัมพันธ์ เงื่อนไข และความถูกต้องของสิ่งที่เราสนใจด้วย เมื่อเราพูดถึงอัลกอริทึม เราไม่ได้ต้องการเพียงคำตอบว่าโปรแกรม *ดูเหมือน* ทำงานได้ในตัวอย่างที่ลอง แต่เราต้องการอธิบายอย่างชัดเจนว่า *เพราะเหตุใด* มันจึงควรทำงานได้ถูกต้องในทุกกรณีที่อยู่ภายใต้เงื่อนไขที่กำหนด

ในแง่นี้ คณิตศาสตร์ทำหน้าที่คล้ายภาษาสำหรับเขียน *specification* ของความคิด เรานิยามคำศัพท์ให้ชัด เรายระบุเงื่อนไขให้ครบ เราแยกสิ่งที่อ้างออกจากเหตุผลที่ใช้รองรับ และเมื่อจำเป็น เราเขียนการพิสูจน์เพื่อแสดงให้เห็นว่าแต่ละบรรทัดของการให้เหตุผลนั้นตามมาจากอะไร กระบวนการทั้งหมดนี้เป็นหัวใจของการให้เหตุผลเชิงคณิตศาสตร์ และเป็นทักษะพื้นฐานของการเรียนคอมพิวเตอร์ในระดับที่จริงจัง

1.1 คณิตศาสตร์ในฐานะภาษา

เมื่อเราเขียนโปรแกรม เราใช้ภาษาโปรแกรมเพื่อสื่อสารกับเครื่อง แต่ก่อนจะไปถึงขั้นนั้น เรามักต้องสื่อสารกับมนุษย์ก่อนเสมอ: เราต้องอธิบายว่าปัญหาคืออะไร ข้อมูลมีลักษณะอย่างไร ข้อจำกัดมีอะไร คำตอบที่ถูกต้องควรมีคุณสมบัติแบบไหน และเหตุใดวิธีที่เราออกแบบจึงน่าเชื่อถือ

ภาษารธรรมชาติช่วยให้เราเล่าเรื่องได้ดี แต่ก็มักเปิดช่องให้คลุมเครือได้ง่าย คณิตศาสตร์จึงเข้ามาช่วยในฐานะภาษาที่เน้นความแม่นยำ มันบังคับให้เราระบุเรื่อกำลังพูดถึงวัตถุชนิดใด เงื่อนไขใดเป็นสมมติฐาน ข้อสรุปใดเป็นสิ่งที่ต้องพิสูจน์ และเส้นทางจากต้นทางไปปลายทางนั้นอาศัยเหตุผลอะไรบ้าง

ในรายวิชานี้ เราให้ความสำคัญกับความคิดแบบ *บอกให้ชัดว่าแต่ละประโยคมาจากไหน* กล่าวอีกแบบหนึ่งคือ เราจะไมยอมรับประโยคที่ “น่าจะจริง” ถ้ายังไม่สามารถชี้ได้ว่ามันตามมาจากนิยาม ข้อเท็จจริงเดิม หรือกฎตรรกะใด

1.2 ประพจน์และประโยคเปิด

1.2.1 ประพจน์

Definition 1.2.1. ประพจน์ (proposition) คือประโยคที่มีค่าความจริงแน่นอนว่าเป็น **จริง** หรือ **เท็จ** แต่ไม่เป็นทั้งสองอย่างพร้อมกัน

แนวคิดสำคัญของนิยามนี้อยู่ที่คำว่า “มีค่าความจริงแน่นอน” กล่าวคือ เมื่อประโยคหนึ่งเป็นประพจน์ เราต้องสามารถมองมันได้ว่าในท้ายที่สุดแล้ว มันมีสถานะเพียงสองแบบเท่านั้น คือจริงหรือเท็จ ไม่ใช่ประโยคกำกวม ไม่ใช่คำสั่ง และไม่ใช่ประโยคที่ยังขาดข้อมูลสำคัญบางอย่าง

Example 1.2.2. ประโยคต่อไปนี้นี้เป็นประพจน์

1. $2 + 3 = 5$
2. “กรุงเทพฯ เป็นเมืองหลวงของประเทศไทย”
3. “เลข 7 เป็นจำนวนเฉพาะ”

Example 1.2.3. ประโยคต่อไปนี้ *ไม่ใช่* ประพจน์

1. “กรุณาปิดประตู” (เป็นคำสั่ง)
2. “ $x^2 - 3x + 2 = 0$ ” (ยังไม่ทราบว่า x คืออะไร)
3. “ n เป็นจำนวนเฉพาะ” (ยังไม่ทราบว่า n คืออะไร)

ประเด็นที่ควรสังเกตก็คือ ประโยคบางชนิด ดูเหมือน จะเป็นข้อความทางคณิตศาสตร์ แต่ยังไม่ใช่ประพจน์ในทันที เพราะมันยังมีตัวแปรที่ไม่ได้ระบุความหมายอย่างสมบูรณ์

1.2.2 Predicate หรือประโยคเปิด

Definition 1.2.4. **Predicate** หรือ **ประโยคเปิด** (open sentence) คือประโยคที่มีตัวแปรอยู่ในนั้น และจะกลายเป็นประพจน์ได้ก็ต่อเมื่อเรา

1. กำหนดค่าของตัวแปรนั้นลงไป หรือ
2. ใส่ตัวบ่งปริมาณ เช่น “สำหรับทุก” หรือ “มีอย่างน้อยหนึ่ง”

ตัวอย่างเช่น ถ้าเราพิจารณา predicate

$$P(n) : \text{“}n \text{ เป็นจำนวนเฉพาะ”}$$

ประโยคนี้ยังไม่ใช่ประพจน์ เพราะยังไม่รู้ว่า n คือจำนวนใด แต่ถ้าเราแทนค่า $n = 4$ เราจะได้ประโยค

$$P(4) : \text{“}4 \text{ เป็นจำนวนเฉพาะ”}$$

ซึ่งตอนนี้เป็นประพจน์แล้ว และเป็นประพจน์เท็จ

อีกวิธีหนึ่งคือใส่ตัวบ่งปริมาณ เช่น

$$\forall n \in \mathbb{N}, P(n) \quad \text{หรือ} \quad \exists n \in \mathbb{N}, P(n)$$

ข้อความเหล่านี้เป็นประพจน์ เพราะมีการระบุชัดแล้วว่าเรากำลังพูดถึงสมาชิกทุกตัว หรือบางตัวของเซตที่กำหนด

1.2.3 จากประโยคเปิดสู่ประพจน์

การเปลี่ยน predicate ให้เป็นประพจน์มีสองแนวทางพื้นฐาน

1. แทนค่าตัวแปร

เช่นจาก `` $n$  เป็นจำนวนเฉพาะ" ไปเป็น ``13 เป็นจำนวนเฉพาะ"

2. ใส่ตัวบ่งปริมาณพร้อมระบุโดเมน

เช่น ``สำหรับทุก  $n \in \mathbb{Z}$ " หรือ ``มี $n \in \mathbb{N}$ บางตัว"

หากสิ่งที่เรากำลังพิจารณาอยู่เพียงไม่กี่ตัว เราอาจนึกภาพตัวบ่งปริมาณได้จากการเชื่อมประโยคหลายประโยคเข้าด้วยกัน เช่น ข้อความว่า

``ทุกจำนวนใน $\{2, 3, 4\}$ เป็นจำนวนเฉพาะ"

สามารถมองอย่างไม่เป็นทางการได้ว่าเทียบกับ

``2 เป็นจำนวนเฉพาะ และ 3 เป็นจำนวนเฉพาะ และ 4 เป็นจำนวนเฉพาะ"

ส่วนข้อความว่า

``มีจำนวนใน $\{2, 3, 4\}$ เป็นจำนวนเฉพาะ"

ก็เทียบกับ

``2 เป็นจำนวนเฉพาะ หรือ 3 เป็นจำนวนเฉพาะ หรือ 4 เป็นจำนวนเฉพาะ"

วิธีคิดแบบนี้ช่วยให้เห็นหน้าที่ของ ``ทุก" และ ``มี" แต่เมื่อเราต้องพูดถึงเซตที่มีสมาชิกอนันต์ตัว เราจำเป็นต้องใช้สัญลักษณ์ของตัวบ่งปริมาณอย่างจริงจังมากขึ้น ซึ่งจะเป็นหัวข้อสำคัญในบทถัดไป

1.3 เอกภพสัมพัทธ์และข้อตกลงพื้นฐาน

ข้อตกลงที่ใช้ตลอดเล่มช่วงต้น

$$\mathbb{N} = \{0, 1, 2, \dots\}$$

กล่าวคือ ในเอกสารชุดนี้เรานับ 0 เป็นจำนวนนับ

ข้อตกลงเรื่องโดเมนหรือเอกภพสัมพัทธ์เป็นเรื่องเล็กที่มักถูกมองข้าม แต่จริง ๆ แล้วเป็นส่วนสำคัญของการสื่อสารทางคณิตศาสตร์ เพราะข้อความเดียวกันอาจให้ค่าความจริงต่างกันได้ หากเปลี่ยนกลุ่มของวัตถุที่เรากำลังพูดถึง ตัวอย่างเช่น ข้อความว่า

$$\exists x, x^2 = 2$$

ยังไม่สมบูรณ์พอ เพราะเราไม่รู้ว่า x ถูกเลือกมาจากไหน ถ้า $x \in \mathbb{R}$ ข้อความนี้เป็นจริง แต่ถ้า $x \in \mathbb{Z}$ ข้อความนี้เป็นเท็จ ดังนั้น ทุกครั้งที่เราเขียนประโยคในรูป “สำหรับทุก x ” หรือ “มี x บางตัว” เราควรถามตัวเองทันทีว่า x มาจากไหน

ประโยคทางคณิตศาสตร์ที่ดีไม่เพียงต้อง “จริง” แต่ต้อง “ชัด” ด้วย หลายครั้งความผิดพลาดไม่ได้เกิดจากการคำนวณผิด แต่เกิดจากการไม่ระบุโดเมนให้ชัดเจนตั้งแต่ต้น

1.4 การพิสูจน์คืออะไร

Definition 1.4.1. การพิสูจน์ ในมุมมองของรายวิชานี้ คือชุดของบรรทัดข้อความที่เรียงต่อกันอย่างมีเหตุผล โดยแต่ละบรรทัดต้องมีสิ่งรองรับว่า ตามมาจากนิยาม ข้อเท็จจริงที่พิสูจน์แล้ว หรือการอ้างตรรกะที่ถูกต้อง และผู้อ่านต้องสามารถตรวจสอบย้อนกลับได้ว่า แต่ละบรรทัดมาจากอะไร

นิยามนี้สะท้อนมุมมองสำคัญข้อหนึ่ง: การพิสูจน์ไม่ใช่การเขียนให้ดูยาก และไม่ใช่ว่าเล่าเรื่องยาว ๆ โดยหวังว่าผู้อ่านจะ “รู้สึกเข้าใจ” แต่เป็นการจัดลำดับของเหตุผลให้ตรวจสอบได้

ในระดับเริ่มต้น เราจึงควรแยกให้ออกระหว่างสองบทบาทต่อไปนี้

1. สิ่งที่ย้ำ (claim)
คือประโยคที่เราต้องการบอกว่าจริง
2. เหตุผลหรือฐานรองรับ (reason/justification)
คือคำอธิบายว่าทำไมเราจึงมีสิทธิ์พูดประโยคนั้น

ตัวอย่างเช่น ถ้าเราพูดว่า

“ n เป็นจำนวนคู่”

ประโยคนี้นับเป็น claim แต่ถ้าจะเขียนมันลงไปในพิสูจน์ เราต้องตอบให้ได้ด้วยว่า เรารู้อย่างไร มาจากสมมติฐานหรือไม่ มาจากนิยามหรือไม่ หรือเพิ่งพิสูจน์ได้จากบรรทัดก่อนหน้านี้

1.4.1 ทำไมการผ่าน test cases ไม่ใช่การพิสูจน์

ในงานคอมพิวเตอร์ ผู้เรียนจำนวนมากคุ้นกับการตรวจสอบโปรแกรมด้วยการทดลองรัน ซึ่งเป็นสิ่งจำเป็นในทางปฏิบัติ แต่การผ่าน test cases ไม่ได้เท่ากับการพิสูจน์ความถูกต้อง

เหตุผลก็คือ test cases เป็นเพียงการตรวจบางกรณี ในขณะที่ข้ออ้างจำนวนมากในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ มีลักษณะเป็นข้อความแบบ “สำหรับทุกอินพุตที่เป็นไปตามเงื่อนไข” ซึ่งเป็นข้อความที่ครอบคลุมกรณีอนันต์จำนวนหรืออย่างน้อยก็จำนวนมหาศาลเกินกว่าจะทดสอบครบได้

ดังนั้นการพิสูจน์จึงเข้ามาทำหน้าที่ตอบคำถามที่การทดลองตอบไม่ได้: ไม่ใช่แค่ว่า ลองแล้วผ่าน แต่เป็นว่า เพราะอะไรจึงต้องผ่านเสมอ

1.5 นิยามพื้นฐานที่ใช้ฝึกพิสูจน์

เพื่อเริ่มฝึกการพิสูจน์อย่างเป็นรูปธรรม เราจะใช้นิยามของจำนวนคู่และจำนวนคี่ ซึ่งเป็นตัวอย่างที่เรียบง่ายแต่ทรงพลังมากในการฝึก ``แกะนิยาม``

Definition 1.5.1. ให้ $n \in \mathbb{Z}$

$$n \text{ เป็นจำนวนคู่} \iff \exists k \in \mathbb{Z} \text{ ที่ทำให้ } n = 2k,$$

$$n \text{ เป็นจำนวนคี่} \iff \exists k \in \mathbb{Z} \text{ ที่ทำให้ } n = 2k + 1.$$

จุดสำคัญของนิยามนี้ไม่ได้อยู่ที่การท่องจำรูป $2k$ หรือ $2k + 1$ แต่อยู่ที่การมองเห็นว่า คำว่า ``เป็นจำนวนคู่`` และ ``เป็นจำนวนคี่`` จริง ๆ แล้วเป็นข้อความเชิงการมีอยู่ ($\exists$) ซึ่งบอกว่า มี จำนวนเต็มตัวหนึ่งทำให้เขียน n อยู่ในรูปที่กำหนดได้

นี่เป็นเหตุผลว่าทำไมเวลาพิสูจน์เรื่องจำนวนคู่หรือคี่ เรามักเริ่มจากประโยคประเภท ``ตามนิยาม จะมี  $k \in \mathbb{Z}$  ที่ทำให้ ...`` เพราะเรากำลังเปิดใช้นิยามโดยตรง

เมื่อข้อความหนึ่งอยู่ในรูป ``มีบางสิ่งที่ทำให้ ...`` เรามักต้องแสดง พยาน (witness) ว่าสิ่งนั้นคืออะไร การตั้งชื่อพยานให้ชัด เช่น k, m, t ช่วยให้พิสูจน์อ่านง่ายขึ้นมาก

1.6 โครงสร้างมาตรฐานของการพิสูจน์แบบอิมพลีเคชัน

ข้อความในรูป

$$P \rightarrow Q$$

อ่านว่า ``ถ้า  $P$  แล้ว  $Q$ `` มักพิสูจน์ด้วยรูปแบบมาตรฐานดังนี้

แพตเทิร์นพื้นฐาน

เพื่อพิสูจน์ ``ถ้า  $P$  แล้ว  $Q$ ``

1. ระบุเอกภพหรือวัตถุที่กำลังพิจารณาให้ชัด
2. สมมติว่า P เป็นจริง
3. ใช้นิยามหรือข้อเท็จจริงที่เกี่ยวข้องผลักไปข้างหน้า
4. สรุปว่า Q เป็นจริง

โครงสร้างนี้สำคัญมาก เพราะมันเป็นต้นแบบของพิสูจน์จำนวนมากในวิชาตรรกศาสตร์ โดยเฉพาะข้อความที่มีทั้งตัวบ่งปริมาณและอิมพลิเคชัน เช่น

$$\forall n \in \mathbb{Z}, P(n) \rightarrow Q(n).$$

ในกรณีนี้ เรามักเริ่มด้วยการเลือก $n \in \mathbb{Z}$ แบบใด ๆ แล้วสมมติว่า $P(n)$ จริง ก่อนจะพาเหตุผลไปสู่ $Q(n)$

1.6.1 ตัวอย่างต้นแบบ: ถ้าจำนวนคี่แล้วยกกำลังสองยังคงเป็นคี่

Theorem 1.6.1. สำหรับทุกจำนวนเต็ม n ถ้า n เป็นจำนวนคี่ แล้ว n^2 เป็นจำนวนคี่

Proof idea. ใช้นิยามของจำนวนคี่: ถ้า n คี่ แปลว่ามี $k \in \mathbb{Z}$ บางตัวที่ทำให้ $n = 2k + 1$ จากนั้นคำนวณ n^2 และจัดรูปให้อยู่ในรูป $2(\dots) + 1$ เพื่อกลับไปใช้นิยามของจำนวนคี่อีกครั้ง

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคี่ ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k + 1.$$

ดังนั้น

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

ให้

$$m = 2k^2 + 2k.$$

เนื่องจาก $k \in \mathbb{Z}$ จึงได้ว่า $m \in \mathbb{Z}$ และจากสมการข้างต้นมีว่า

$$n^2 = 2m + 1.$$

จึงสรุปได้ตามนิยามว่า n^2 เป็นจำนวนคี่

□

1.6.2 อ่านพิสูจน์อย่างนักเขียนพิสูจน์

พิสูจน์ข้างต้นควรถูกอ่านอย่างมีโครงสร้าง ไม่ใช่อ่านเพียงตามบรรทัด

1. “ให้ $n \in \mathbb{Z}$ ” คือการระบุเอกภพสัมพัทธ์
2. “สมมติว่า n เป็นจำนวนคี่” คือการเริ่มต้นพิสูจน์อิมพลิเคชัน
3. “ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z} \dots$ ” คือการแคะนิยาม
4. การตั้ง $m = 2k^2 + 2k$ คือการเลือกพยานของข้อความเชิงการมีอยู่
5. บรรทัดสุดท้ายคือการย้อนกลับไปใช้นิยามเดิมเพื่อปิดพิสูจน์

การอ่านพิสูจน์ด้วยสายตาแบบนี้จะช่วยให้เห็นว่า พิสูจน์ไม่ได้เป็นการคำนวณลอย ๆ แต่เป็นการประกอบโครงสร้างตรรกะทีละขั้น

1.6.3 อีกหนึ่งตัวอย่างสั้น: ถ้าจำนวนคู่แล้วยกกำลังสองยังคงเป็นคู่

Theorem 1.6.2. สำหรับทุกจำนวนเต็ม n ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

Proof idea. ถ้า n เป็นจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้ $n = 2k$ เมื่อนำไปยกกำลังสองจะได้ $n^2 = 4k^2$ ซึ่งเขียนเป็น $2(\dots)$ ได้

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคู่ ตามนิยามของจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k.$$

ดังนั้น

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2).$$

ให้ $m = 2k^2$ ซึ่งเป็นจำนวนเต็ม จะได้ว่า $n^2 = 2m$ จึงสรุปว่า n^2 เป็นจำนวนคู่ตามนิยาม □

1.7 ตัวอย่างค้านและการปฏิเสธข้อความแบบ ``สำหรับทุก"

นอกจากการพิสูจน์ว่าอะไรจริงแล้ว อีกทักษะหนึ่งที่สำคัญไม่แพ้กันคือการชี้ว่าอะไรไม่จริง สำหรับข้อความในรูป

$$\forall x, P(x)$$

การปฏิเสธไม่ได้ต้องการการอธิบายยืดเยื้อ แต่ต้องการเพียงการหาตัวอย่างสักตัวที่ทำให้ $P(x)$ ล้มเหลว

Definition 1.7.1. ตัวอย่างค้าน (counterexample) คือค่าหรือตัวอย่างเฉพาะที่ทำให้ข้อความแบบ ``สำหรับทุก" เป็นเท็จ แนวคิดนี้ง่ายแต่ทรงพลังมาก: หากมีคนอ้างว่า

$$\forall p \in \mathbb{Z}, (p \text{ เป็นจำนวนเฉพาะ}) \rightarrow (p \text{ เป็นจำนวนคี่}),$$

เรายังไม่ต้องรีบพิสูจน์หรือโต้เถียงด้วยคำพูดยาว ๆ แต่ควรถามทันทีว่า มีตัวอย่างสักตัวไหมที่เป็นจำนวนเฉพาะแต่ไม่เป็นจำนวนคี่

คำตอบคือ $p = 2$ เพราะ 2 เป็นจำนวนเฉพาะ และ 2 ไม่เป็นจำนวนคี่ ดังนั้นข้อความข้างต้นเป็นเท็จ

สำหรับข้อความสากล ($\forall$) บางครั้งวิธีที่เร็วและชัดที่สุดไม่ใช่การคิดว่าจะพิสูจน์อย่างไร แต่คือการถามกลับว่า ``มีตัวอย่างค้านไหม" นิสัยนี้มีประโยชน์มากทั้งในคณิตศาสตร์และการออกแบบอัลกอริทึม

1.8 การเขียนพิสูจน์ให้เหมือนหนังสือ มากกว่าเหมือนแบบเดิมคำ

ในระยะเริ่มต้น ผู้เรียนมักคุ้นกับโครงร่างพิสูจน์แบบเดิมช่องว่าง ซึ่งมีประโยชน์มากสำหรับฝึกจับแพทเทิร์น แต่เมื่อเราจะเขียนเป็นหนังสือหรือเอกสารอ้างอิง เป้าหมายควรเปลี่ยนจาก ``เดิมให้ครบ" เป็น ``เล่าให้ชัด"

การเขียนที่ดีจึงควรมีคุณลักษณะดังนี้

1. ระบุว่าเรากำลังพิสูจน์ข้อความอะไร
2. บอกแนวคิดสั้น ๆ ว่าจะใช้นิยามหรือวิธีใด
3. เดินพิสูจน์ทีละบรรทัดอย่างมีที่มา
4. จบบทพิสูจน์ในจุดที่ข้อสรุปตามมาอย่างชัดเจน

รูปแบบ

Claim $\rightarrow$ **Proof idea** $\rightarrow$ **Proof** $\rightarrow$ **QED**

จึงเป็นรูปแบบที่ดีสำหรับการฝึกในช่วงต้น เพราะช่วยแยกบทบาทของแต่ละส่วนออกจากกันอย่างชัดเจน และส่งเสริมให้ผู้อ่านตรวจสอบโครงสร้างเหตุผลได้ง่าย

1.9 สรุปท้ายบท

บทนี้มีเป้าหมายเพื่อวางพื้นฐานด้านภาษาและการให้เหตุผลก่อนเข้าสู่หัวข้อทฤษฎีที่ลึกขึ้น ใจความสำคัญมีดังนี้

1. คณิตศาสตร์เป็นภาษาที่ช่วยให้เราสื่อสารอย่างแม่นยำ
2. ประพจน์คือประโยคที่มีค่าความจริงแน่นอน
3. predicate ยังไม่เป็นประพจน์จนกว่าจะกำหนดค่า หรือใส่ตัวบ่งปริมาณ
4. การระบุโดเมนให้ชัดเป็นส่วนหนึ่งของการเขียนอย่างถูกต้อง
5. การพิสูจน์คือการเรียงข้อความที่ตรวจสอบที่มาของแต่ละบรรทัดได้
6. การพิสูจน์อิมพลีเคชันมักเริ่มจากการสมมติส่วนหน้า แล้วผลักไปสู่ส่วนหลัง
7. การหักล้างข้อความแบบ “สำหรับทุก” มักทำได้ด้วยตัวอย่างค้านเพียงตัวเดียว

อ่านต่อ

เนื้อหาในบทนี้ปูทางไปสู่หัวข้อเรื่องตัวเชื่อมตรรกะ ตัวบ่งปริมาณ และวิธีพิสูจน์รูปแบบต่าง ๆ ในบทถัดไป โดยเฉพาะอย่างยิ่ง เมื่อเราเริ่มเขียนข้อความที่ซับซ้อนขึ้น ความสามารถในการแยกโดเมน สมมติฐาน ข้อสรุป และพยาน จะกลายเป็นทักษะสำคัญที่ต้องใช้ซ้ำตลอดทั้งรายวิชา

แบบฝึกหัดท้ายบท

Exercise 1.9.1. พิจารณาว่าข้อความต่อไปนี้ประพจน์หรือไม่ ถ้าเป็น ให้ระบุว่าเป็นจริงหรือเท็จ

1. $5^2 = 25$
2. “ $x + 1 = 3$ ”
3. “กรุณาส่งงานตรงเวลา”

4. ``มีจำนวนเต็มที่ยกกำลังสองแล้วได้ 9"

Exercise 1.9.2. อธิบายว่าข้อความ

$$\forall x, x^2 \geq 0$$

ยังไม่สมบูรณ์พออย่างไรหากไม่ระบุโดเมนของ x และจงเขียนใหม่ให้สมบูรณ์อย่างน้อยสองแบบ

Exercise 1.9.3. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า a และ b เป็นจำนวนคู่ แล้ว $a + b$ เป็นจำนวนคู่

Exercise 1.9.4. หาตัวอย่างค้านของข้อความต่อไปนี้

$$\forall n \in \mathbb{Z}, (n \text{ เป็นจำนวนคี่}) \rightarrow (n \text{ หารด้วย 3 ลงตัว})$$

และอธิบายสั้น ๆ ว่าทำไมตัวอย่างนั้นจึงใช้ได้

บทที่ 2

ตรรกศาสตร์ประพจน์และโครงสร้างของการพิสูจน์

ในบทก่อนหน้า เราเริ่มจากแนวคิดพื้นฐานว่า คณิตศาสตร์เป็นภาษาสำหรับเขียนข้ออ้างให้ชัดเจนและตรวจสอบได้ เราได้เห็นแล้วว่า การพิสูจน์ไม่ได้เป็นเพียงการคำนวณหรือการลองตัวอย่างหลาย ๆ ค่า แต่เป็นการเรียงเหตุผลทีละบรรทัด โดยแต่ละบรรทัดต้องมีที่มารองรับอย่างชัดเจน

บทนี้จะขยับจากระดับของ ``ประพจน์เดี่ยว" ไปสู่ระดับของ ``รูปประโยค" กล่าวคือ เราจะพิจารณาว่าถ้าเรานำประพจน์มาประกอบกันด้วยคำว่า ``ไม่", ``และ", ``หรือ", ``ถ้า...แล้ว...", และ ``ก็ต่อเมื่อ" แล้วรูปของประโยคที่ได้จะบอกอะไรเกี่ยวกับวิธีพิสูจน์ที่เหมาะสม

แนวคิดสำคัญของบทนี้คือ

รูปของข้อความ $\Rightarrow$ รูปแบบของการพิสูจน์

เมื่อเห็นข้อความในรูป $P \rightarrow Q$ เราควรถามทันทีว่าควรพิสูจน์ตรงดีหรือไม่ หรือควรเปลี่ยนไปพิสูจน์แบบ contrapositive หรือ contradiction เมื่อเห็นข้อความในรูป $P \leftrightarrow Q$ เราควรรู้ทันทีว่าต้องพิสูจน์สองทิศ และเมื่อเห็นข้อความที่มี ``หรือ" เราควรแยกให้ออกว่าเรากำลัง สร้าง ประโยค $\vee$ หรือกำลัง ใช้ ประโยค $\vee$ ที่มีอยู่แล้วเพื่อสรุปผล

2.1 ตัวเชื่อมตรรกะและสูตรตรรกะ

Definition 2.1.1. ให้ P, Q เป็นประพจน์ เราสามารถสร้างประพจน์ใหม่ได้จากตัวเชื่อมตรรกะดังต่อไปนี้

$$\neg P, \quad P \wedge Q, \quad P \vee Q, \quad P \rightarrow Q, \quad P \leftrightarrow Q.$$

ตัวเชื่อมเหล่านี้มีความหมายพื้นฐานดังนี้

- $\neg P$ อ่านว่า ``ไม่เป็นจริงที่  $P$ " หรือ ``ไม่ P "
- $P \wedge Q$ อ่านว่า `` P และ Q "
- $P \vee Q$ อ่านว่า `` $P$  หรือ  $Q$ " โดยในวิชานี้หมายถึง ``หรืออย่างน้อยหนึ่งอย่าง" และ อาจจริงทั้งคู่ได้

- $P \rightarrow Q$ อ่านว่า “ถ้า P แล้ว Q ”
- $P \leftrightarrow Q$ อ่านว่า “ P ก็ต่อเมื่อ Q ”

ค่าความจริงของประพจน์ที่ประกอบด้วยตัวเชื่อมเหล่านี้สามารถอธิบายได้ด้วยตารางค่าความจริง แต่ในรายวิชานี้ เป้าหมายหลักไม่ใช่การท่องตารางให้ได้ เป้าหมายคือการเข้าใจ *ความหมาย* ของแต่ละตัวเชื่อมให้ดีพอจนสามารถเลือกวิธีพิสูจน์ที่เหมาะสมได้

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

แม้ตารางนี้จะดูเป็นเรื่องของ “ค่าความจริง” แต่ในทางปฏิบัติ สิ่งที่เราสนใจกว่าคือความหมายเชิงโครงสร้างของมัน โดยเฉพาะคอลัมน์ของ $P \rightarrow Q$ และ $P \leftrightarrow Q$ เพราะสองตัวเชื่อมนี้สัมพันธ์กับการพิสูจน์โดยตรง

2.2 อิมพลีเคชัน: ความหมายของข้อความรูป $P \rightarrow Q$

2.2.1 อิมพลีเคชันในฐานะคำสัญญา

ประโยค $P \rightarrow Q$ ควรถูกอ่านว่า

“ทุกครั้งที่ P จริง จะต้องทำให้ Q จริงด้วย”

ในมุมมองของวิทยาการคอมพิวเตอร์ เราอาจมอง P เป็น *precondition* และ Q เป็น *postcondition* กล่าวคือ ถ้าเงื่อนไขตั้งต้น P เกิดขึ้น ระบบจะต้องรับผิดชอบทำให้ผลลัพธ์ Q เกิดขึ้นตามมา

ดังนั้น เมื่อเราเห็นข้อความในรูป $P \rightarrow Q$ สิ่งที่ต้องจับให้ได้ไม่ใช่เพียงคำอ่าน แต่คือคำถามว่า

“ข้อความนี้จะพึงได้ในกรณีใด?”

2.2.2 อิมพลีเคชันเป็นเท็จได้เมื่อใด

จากตารางค่าความจริง เราเห็นว่า $P \rightarrow Q$ เป็นเท็จได้เพียงกรณีเดียว คือ

P จริง แต่ Q เท็จ

นี่เป็นข้อสังเกตที่สำคัญมาก เพราะมันบอกเราทันทีว่า ถ้าต้องการ *หักล้าง* ข้อความในรูป $P \rightarrow Q$ เราต้องหากรณีที่ทำให้ “ส่วนหน้า” เป็นจริง แต่ “ส่วนหลัง” เป็นเท็จ

หลักคิดสำหรับการหากล้างอิมพลีเคชัน

ถ้าต้องการหากล้างข้อความ $P \rightarrow Q$ ให้มองหาตัวอย่างที่ทำให้

$$P \text{ จริง} \quad \text{และ} \quad Q \text{ เท็จ}$$

เพียงตัวอย่างเดียวก็พอที่จะทำให้ข้อความทั้งประโยคเป็นเท็จ

Example 2.2.1. พิจารณาข้อความ

$$\forall n \in \mathbb{Z}, (n^2 \text{ เป็นจำนวนคู่}) \rightarrow (n \text{ หารด้วย 4 ลงตัว})$$

ข้อความนี้เป็นเท็จ เพราะเราเลือก $n = 2$ ได้

เมื่อ $n = 2$ จะมี $n^2 = 4$ ซึ่งเป็นจำนวนคู่ ดังนั้นส่วนหน้าของอิมพลีเคชันเป็นจริง แต่ 2 ไม่หารด้วย 4 ลงตัว ดังนั้นส่วนหลังเป็นเท็จ จึงเป็นตัวอย่างค้านของข้อความนี้

2.2.3 ความจริงแบบว่าง

ข้อที่ผู้เรียนมักรู้สึกขัดใจในช่วงแรกคือ ถ้า P เป็นเท็จ แล้ว $P \rightarrow Q$ จะถือว่าเป็นจริง ไม่ว่า Q จะเป็นจริงหรือเท็จก็ตาม

แนวคิดนี้เรียกว่า **vacuous truth** หรือความจริงแบบว่าง

เหตุผลคือ อิมพลีเคชันสัญญาเฉพาะกรณีที่ P เกิดขึ้นเท่านั้น ถ้า P ไม่เกิดขึ้นเลย ก็ไม่มีกรณีใดที่สัญญาถูกละเมิด

Example 2.2.2. ประโยค

$$\text{``ถ้า 1 เป็นจำนวนคู่ แล้ว } 2 + 2 = 5\text{''}$$

เป็นจริงในเชิงตรรกะ ไม่ใช่เพราะ $2 + 2 = 5$ จริง แต่เพราะส่วนหน้า ``1 เป็นจำนวนคู่" เป็นเท็จ

2.2.4 ถ้อยคำที่มักแปลผิด

ข้อความรูป $P \rightarrow Q$ มักถูกเขียนในภาษาธรรมชาติได้หลายแบบ และผู้เรียนมักสับสนกับคำว่า *if*, *only if*, *sufficient*, และ *necessary*

ถ้อยคำที่เทียบเท่ากับ $P \rightarrow Q$

ข้อความต่อไปนี้ล้วนมีความหมายเท่ากับ $P \rightarrow Q$

- “ถ้า P แล้ว Q ”
- “ P only if Q ”
- “ Q if P ”
- “ P เป็นเงื่อนไขเพียงพอสำหรับ Q ”
- “ Q เป็นเงื่อนไขจำเป็นสำหรับ P ”

วิธีจำที่มีประโยชน์มากคือ

“ P only if Q ” หมายความว่า ถ้าไม่มี Q ก็ห้ามมี P

นั่นคือ Q เป็นสิ่งที่ P ต้องพึ่งพา จึงเป็น *necessary condition* ของ P

2.3 จากระบบประโยคสู่รูปแบบการพิสูจน์

ในบทก่อนหน้านี้ เราเห็นแล้วว่าการพิสูจน์ข้อความในรูป “ถ้า P แล้ว Q ” มักเริ่มด้วยการสมมติว่า P เป็นจริง แล้วค่อยผลักเหตุผลไปจนถึง Q ตอนนี้เราจะเขียนหลักคิดนี้ให้ชัดเจนในเชิงตรรกะ

โครงมาตรฐานของการพิสูจน์แบบ direct proof

เมื่อต้องการพิสูจน์ข้อความ $P \rightarrow Q$

1. ระบุวัตถุที่กำลังพิจารณาให้ชัด
2. สมมติว่า P เป็นจริง
3. ใช้นิยาม ข้อเท็จจริง หรือทฤษฎีบทที่เกี่ยวข้อง
4. สรุปว่า Q เป็นจริง

Theorem 2.3.1. สำหรับทุกจำนวนเต็ม n ถ้า n หารด้วย 4 ลงตัว แล้ว n เป็นจำนวนคู่

Proof idea. ใช้ความหมายของ “หารด้วย 4 ลงตัว” แล้วจัดรูปให้เห็นว่า n อยู่ในรูป $2(\dots)$

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n หารด้วย 4 ลงตัว ตามนิยาม จะมีจำนวนเต็ม k ที่ทำให้

$$n = 4k.$$

ดังนั้น

$$n = 4k = 2(2k).$$

เนื่องจาก $2k \in \mathbb{Z}$ จึงได้ว่า n อยู่ในรูป $2m$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n เป็นจำนวนคู่ □
ทฤษฎีบทนี้เป็นตัวอย่างสำคัญ เพราะมันแสดงรูปแบบพื้นฐานที่สุดของการพิสูจน์อิมพลีเคชัน: เริ่มจากสมมติฐาน เปิดนิยาม แล้วจัดรูปไปสู่ข้อสรุป

2.4 สมมูลตรรกะที่มีประโยชน์ต่อการพิสูจน์

บางครั้งการพิสูจน์ $P \rightarrow Q$ ตรง ๆ อาจไม่สะดวก ในกรณีเช่นนี้ เรามักเปลี่ยนรูปประโยคให้สมมูลกับประโยคเดิม แต่พิสูจน์ง่ายกว่า

2.4.1 Contrapositive

สมมูลที่สำคัญที่สุดอันหนึ่งคือ

$$P \rightarrow Q \quad \equiv \quad \neg Q \rightarrow \neg P.$$

ข้อความด้านขวาเรียกว่า **contrapositive** ของข้อความด้านซ้าย

ความสำคัญของ contrapositive ไม่ได้อยู่ที่การเล่นสัจลักษณ์ แต่อยู่ที่ข้อเท็จจริงว่า บางครั้งการพิสูจน์ “ถ้า P แล้ว Q ” ตรง ๆ นั้นยาก แต่การพิสูจน์ “ถ้าไม่ Q แล้วไม่ P ” กลับง่ายกว่ามาก

ข้อควรจำ

สำหรับข้อความ $P \rightarrow Q$

- **contrapositive** คือ $\neg Q \rightarrow \neg P$
- **converse** คือ $Q \rightarrow P$

ข้อความเดิมสมมูลกับ contrapositive แต่ไม่จำเป็นต้องสมมูลกับ converse

Theorem 2.4.1. สำหรับทุกจำนวนเต็ม n ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่

Proof idea. พิสูจน์แบบ contrapositive แทน กล่าวคือพิสูจน์ว่า ถ้า n ไม่เป็นจำนวนคู่ หรือกล่าวอีกแบบคือ n เป็นจำนวนคี่ แล้ว n^2 ไม่เป็นจำนวนคู่ หรือก็คือ n^2 เป็นจำนวนคี่

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคี่ ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k + 1.$$

ดังนั้น

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

จึงได้ว่า n^2 อยู่ในรูป $2m + 1$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n^2 เป็นจำนวนคี่ เราได้พิสูจน์แล้วว่า ถ้า n คี่ แล้ว n^2 คี่ ซึ่งเป็น contrapositive ของข้อความที่ต้องการ ดังนั้นสรุปได้ว่า ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่ $\square$

2.4.2 De Morgan

เมื่อใช้ contrapositive หรือ contradiction เรามักต้องปฏิเสธข้อความที่ประกอบด้วย ``และ" หรือ ``หรือ" จึงควรรู้สมมูลของ De Morgan ต่อไปนี้

$$\neg(A \wedge B) \equiv (\neg A \vee \neg B) \qquad \neg(A \vee B) \equiv (\neg A \wedge \neg B)$$

สมมูลคู่นี้ใช้บ่อยมาก เพราะมันบอกเราว่า การปฏิเสธประโยคประกอบไม่ได้เพียงแค่ใส่เครื่องหมาย $\neg$ ไว้ข้างหน้า แต่ต้องเปลี่ยนตัวเชื่อมและปฏิเสธองค์ประกอบภายในด้วย

Example 2.4.2. นิเสธของข้อความ `` $a$  เป็นจำนวนคู่ และ  $b$  เป็นจำนวนคู่" คือ `` a ไม่เป็นจำนวนคู่ หรือ b ไม่เป็นจำนวนคู่" นิเสธของข้อความ `` $a \geq 0$  หรือ  $b < 0$ " คือ `` $a < 0$ และ $b \geq 0$ "

2.4.3 อิมพลิเคชันในรูปดิสจังก์ชัน

อีกสมมูลหนึ่งที่ใช้กันได้บ่อยคือ

$$P \rightarrow Q \equiv \neg P \vee Q.$$

สมมูลนี้ช่วยอธิบายว่าทำไมอิมพลิเคชันจึงเป็นจริงเมื่อ P เท็จ และยังช่วยในการแปลงรูปข้อความให้มองเห็นโครงสร้างของการพิสูจน์ชัดเจน โดยเฉพาะเวลาที่ต้องพิสูจน์ข้อความในรูปดิสจังก์ชัน

2.5 การพิสูจน์แบบ contradiction

นอกจากการพิสูจน์ตรงและการพิสูจน์แบบ contrapositive แล้ว อีกวิธีสำคัญคือ **proof by contradiction** แนวคิดพื้นฐานมีสองแบบที่เกี่ยวข้องกัน

1. ถ้าต้องการพิสูจน์ประโยคเดี่ยว P เราอาจสมมติว่า $\neg P$ แล้วพาไปสู่ความขัดแย้ง
2. ถ้าต้องการพิสูจน์ $P \rightarrow Q$ เราอาจสมมติว่า P จริงและ Q เท็จพร้อมกัน

เชิงตรรกะแล้ว

$$P \rightarrow Q \equiv \neg(P \wedge \neg Q)$$

ดังนั้น ถ้าเราพิสูจน์ได้ว่า ``กรณี P จริงแต่ Q เท็จ" เป็นไปไม่ได้ เราก็พิสูจน์ $P \rightarrow Q$ ได้

Theorem 2.5.1. ไม่มีจำนวนเต็มใดที่เป็นทั้งจำนวนคู่และจำนวนคี่พร้อมกัน

Proof idea. สมมติว่ามีจำนวนเต็มดังกล่าวอยู่จริง แล้วใช้นิยามของจำนวนคู่และจำนวนคี่พร้อมกัน จะได้สมการที่เป็นไปไม่ได้

Proof. สมมติขัดแย้งว่า มีจำนวนเต็ม n ที่เป็นทั้งจำนวนคู่และจำนวนคี่พร้อมกัน

เพราะ n เป็นจำนวนคู่ จะมี $a \in \mathbb{Z}$ ที่ทำให้

$$n = 2a.$$

และเพราะ n เป็นจำนวนคี่ จะมี $b \in \mathbb{Z}$ ที่ทำให้

$$n = 2b + 1.$$

ดังนั้น

$$2a = 2b + 1.$$

จัดรูปได้ว่า

$$2(a - b) = 1.$$

แต่จำนวนทางซ้ายเป็นจำนวนคู่ ส่วนจำนวนทางขวาเป็นจำนวนคี่ ซึ่งเป็นไปไม่ได้ เกิดความขัดแย้ง

ดังนั้นสมมติฐานที่ว่า มีจำนวนเต็มซึ่งเป็นทั้งคู่และคี่พร้อมกันจึงเป็นเท็จ สรุปว่าไม่มีจำนวนเต็มดังกล่าว □

ตัวอย่างนี้แสดงให้เห็นว่า การพิสูจน์แบบ contradiction ไม่ได้แปลว่า เขียนคำว่า “ขัดแย้ง” ลงไปแล้วจบ แต่ต้องแสดงให้เห็นอย่างชัดเจนว่าความขัดแย้งเกิดขึ้นตรงไหน

2.6 การใช้อิมพลีเคชันที่พิสูจน์แล้ว: Modus Ponens

เมื่อเราพิสูจน์ข้อความอิมพลีเคชันได้แล้ว เรามักต้องนำมาไปใช้ต่อในพิสูจน์อื่น กฎการอ้างเหตุผลพื้นฐานที่สุดคือ **Modus Ponens**

Modus Ponens	
ถ้าเรารู้ว่า	$P \rightarrow Q$ และ P
แล้วเราสรุปได้ว่า	$Q.$

ในภาษาเรียบง่าย กฎนี้บอกว่า ถ้ามีกฎว่าทุกครั้งที่ P จริงจะได้ Q จริง และตอนนี้เรารู้แล้วว่า P จริง เราก็มีสิทธิ์สรุปว่า Q จริง

Example 2.6.1. สมมติว่าเราได้พิสูจน์มาแล้วว่า

$$(\forall n \in \mathbb{Z})(n \text{ เป็นจำนวนคู่} \rightarrow n^2 \text{ เป็นจำนวนคู่}).$$

เราต้องการพิสูจน์ว่า

$$(\forall x \in \mathbb{Z})(x \text{ เป็นจำนวนคู่} \rightarrow x^2 + 1 \text{ เป็นจำนวนคี่}).$$

ให้ $x \in \mathbb{Z}$ และสมมติว่า x เป็นจำนวนคู่ จากทฤษฎีบทที่พิสูจน์มาแล้ว สรุปได้ว่า x^2 เป็นจำนวนคู่ ดังนั้นจะมี $k \in \mathbb{Z}$ ที่ทำให้ $x^2 = 2k$ จึงได้ว่า

$$x^2 + 1 = 2k + 1$$

ซึ่งเป็นจำนวนคี่

ตัวอย่างนี้สะท้อนภาพสำคัญของคณิตศาสตร์ทั้งวิชา: เราไม่ได้พิสูจน์แต่ละข้อความแยกขาดจากกัน แต่สะสมข้อเท็จจริงไว้เป็นเครื่องมือสำหรับพิสูจน์ข้อความใหม่ต่อไป

2.7 ดิสจังก์ชันและการพิสูจน์แบบแยกกรณี

2.7.1 ความหมายของ $P \vee Q$

ในวิชานี้ $P \vee Q$ หมายถึง “ P หรือ Q หรือทั้งคู่” ดังนั้นมันเป็นจริงเมื่ออย่างน้อยหนึ่งประโยคในสองประโยคนี้เป็นจริง เมื่อเจอดิสจังก์ชัน เรามักพบอยู่ในสองสถานการณ์หลัก

1. เราต้องการพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง
2. เรารู้แล้วว่า $P \vee Q$ เป็นจริง และต้องการใช้มันเพื่อสรุปผลบางอย่าง

สองสถานการณ์นี้หน้าตาคล้ายกัน แต่ใช้เหตุผลคนละแบบ

2.7.2 การพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง

ถ้าต้องการพิสูจน์ $P \vee Q$ เรามีหลายแนวทาง เช่น

- พิสูจน์ P ให้ได้โดยตรง
- หรือพิสูจน์ Q ให้ได้โดยตรง
- หรือแปลงรูปข้อความเดิมให้อยู่ในรูปที่พิสูจน์ง่ายกว่า

Theorem 2.7.1. ให้ $a, b \in \mathbb{Z}$. ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

Proof idea. พิสูจน์แบบ contrapositive จะง่ายกว่า เพราะ contrapositive ของข้อความนี้คือ ถ้า a ไม่เป็นจำนวนคู่ และ b ไม่เป็นจำนวนคู่ แล้ว ab ไม่เป็นจำนวนคู่ ซึ่งเท่ากับการพิสูจน์ว่า ถ้า a, b เป็นจำนวนคี่ทั้งคู่ แล้ว ab เป็นจำนวนคี่

Proof. เราจะพิสูจน์ contrapositive

ให้ $a, b \in \mathbb{Z}$ และสมมติว่า a ไม่เป็นจำนวนคู่ และ b ไม่เป็นจำนวนคู่ สำหรับจำนวนเต็ม ข้อความ “ไม่เป็นจำนวนคู่” เท่ากับ “เป็นจำนวนคี่” ดังนั้นจะมี $r, s \in \mathbb{Z}$ ที่ทำให้

$$a = 2r + 1, \quad b = 2s + 1.$$

คูณกันได้

$$ab = (2r + 1)(2s + 1) = 4rs + 2r + 2s + 1 = 2(2rs + r + s) + 1.$$

จึงได้ว่า ab เป็นจำนวนคี่ นั่นคือ ab ไม่เป็นจำนวนคู่

ดังนั้น contrapositive เป็นจริง จึงสรุปได้ว่า ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ □

2.7.3 การใช้ $P \vee Q$: proof by cases

ถ้าเรารู้ว่า $P \vee Q$ เป็นจริง เรา *ยังเลือกไม่ได้ทันที* ว่า P จริงหรือ Q จริง สิ่งที่เรารู้มีเพียงว่า อย่างน้อยหนึ่งกรณีต้องเกิดขึ้น ดังนั้นถ้าต้องการสรุปผลลัพธ์เดียวกัน R เราต้องพิสูจน์ว่า

$$P \rightarrow R \quad \text{และ} \quad Q \rightarrow R.$$

นี่คือหลักของ **proof by cases** หรือ $\vee$ -elimination

Proof by Cases

ถ้ารู้ว่า $P \vee Q$ เป็นจริง และเราพิสูจน์ได้ว่า

$$P \rightarrow R \quad \text{และ} \quad Q \rightarrow R$$

แล้วสรุปได้ว่า R จริง

Theorem 2.7.2. ให้ $a, b \in \mathbb{Z}$. ถ้า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ แล้ว ab เป็นจำนวนคู่

Proof idea. แยกเป็นสองกรณีตามดิสจังก์ชันที่กำหนดมา

Proof. ให้ $a, b \in \mathbb{Z}$ และสมมติว่า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

เราพิจารณาเป็นสองกรณี

กรณีที่ 1: a เป็นจำนวนคู่

จะมี $k \in \mathbb{Z}$ ที่ทำให้ $a = 2k$ ดังนั้น

$$ab = (2k)b = 2(kb).$$

เนื่องจาก $kb \in \mathbb{Z}$ จึงได้ว่า ab เป็นจำนวนคู่

กรณีที่ 2: b เป็นจำนวนคู่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้ $b = 2\ell$ ดังนั้น

$$ab = a(2\ell) = 2(a\ell).$$

เนื่องจาก $a\ell \in \mathbb{Z}$ จึงได้ว่า ab เป็นจำนวนคู่

ในทุกกรณี เราสรุปได้ว่า ab เป็นจำนวนคู่ ดังนั้นถ้า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ แล้ว ab เป็นจำนวนคู่ □

ข้อผิดพลาดที่พบบ่อย

เมื่อมี $P \vee Q$ อยู่ในมือ เรา ห้าม เลือกเอาเองว่า “นั่นให้ P จริง” เพราะข้อมูลที่มีไม่ได้บอกว่า P จริงแน่ ๆ มันบอกเพียงว่าต้องมีอย่างน้อยหนึ่งใน P, Q ที่จริง ดังนั้นถ้าจะสรุปผลจาก $P \vee Q$ ให้ถูกต้อง ต้องครอบคลุมทุกกรณีที่เป็นไปได้

2.8 ไบคอนดิชันนัลและการพิสูจน์สองทิศ

ข้อความ $P \leftrightarrow Q$ อ่านว่า “ P ก็ต่อเมื่อ Q ” ซึ่งหมายความว่า P และ Q จริงเทียบเท่ากัน กล่าวคือ แต่ละข้อความเป็นทั้งเงื่อนไขจำเป็นและเงื่อนไขเพียงพอของอีกข้อความหนึ่ง

เชิงตรรกะ เรามีสมมูล

$$P \leftrightarrow Q \quad \equiv \quad (P \rightarrow Q) \wedge (Q \rightarrow P).$$

ดังนั้น เมื่อต้องการพิสูจน์ข้อความแบบ iff สิ่งขั้นต่ำที่ต้องทำคือพิสูจน์ให้ครบสองทิศ

Theorem 2.8.1. สำหรับทุกจำนวนเต็ม n ,

$$n \text{ หารด้วย 4 ลงตัว} \leftrightarrow (n \text{ เป็นจำนวนคู่ และ } n/2 \text{ เป็นจำนวนคู่})$$

Proof idea. พิสูจน์ทีละทิศ ทิศแรกเปิดนิยามของการหารด้วย 4 ลงตัว ส่วนทิศกลับใช้ข้อมูลว่า $n/2$ เป็นจำนวนคู่ แล้วคูณกลับด้วย 2

Proof. เราจะแยกพิสูจน์สองทิศ

($\rightarrow$) สมมติว่า n หารด้วย 4 ลงตัว ดังนั้นมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 4k.$$

จึงได้ว่า

$$n = 2(2k),$$

ดังนั้น n เป็นจำนวนคู่

อีกทั้ง

$$n/2 = 2k,$$

ซึ่งเป็นจำนวนคู่เช่นกัน จึงได้ว่า n เป็นจำนวนคู่ และ $n/2$ เป็นจำนวนคู่

($\leftarrow$) สมมติว่า n เป็นจำนวนคู่ และ $n/2$ เป็นจำนวนคู่ จากที่ $n/2$ เป็นจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n/2 = 2k.$$

คูณทั้งสองข้างด้วย 2 จะได้

$$n = 4k.$$

ดังนั้น n หาดด้วย 4 ลงตัว

เมื่อพิสูจน์ครบทั้งสองทิศแล้ว สรุปได้ว่า

$$n \text{ หาดด้วย 4 ลงตัว} \leftrightarrow (n \text{ เป็นจำนวนคู่ และ } n/2 \text{ เป็นจำนวนคู่})$$

เป็นจริงสำหรับทุกจำนวนเต็ม n

□

2.9 ข้อผิดพลาดที่พบบ่อยในการเขียนพิสูจน์

เมื่อเริ่มใช้ตรรกศาสตร์เป็นเครื่องมือวางโครงพิสูจน์ ผู้เรียนมักผิดพลาดในรูปแบบเดิม ๆ ไม่ก็แบบ การรู้จักข้อผิดพลาดเหล่านี้จะช่วยให้ตรวจงานของตนเองได้ดีขึ้นมาก

2.9.1 พิสูจน์ converse แทนที่จะพิสูจน์ข้อความเดิม

ถ้าต้องการพิสูจน์ $P \rightarrow Q$ แต่กลับเริ่มจากสมมติว่า Q จริง แล้วสรุปว่า P จริง สิ่งที่กำลังพิสูจน์อยู่คือ $Q \rightarrow P$ ซึ่งเป็น converse ไม่ใช่ข้อความเดิม

ตัวอย่างเช่น ถ้าต้องการพิสูจน์

$$\text{ถ้า } n \text{ เป็นจำนวนคู่ แล้ว } n^2 \text{ เป็นจำนวนคู่}$$

แต่กลับเริ่มจากสมมติว่า n^2 เป็นจำนวนคู่ นั้นไม่ใช่การพิสูจน์ตรงของข้อความนี้

2.9.2 พิสูจน์ iff เพียงทิศเดียว

ถ้าต้องการพิสูจน์ $P \leftrightarrow Q$ แต่พิสูจน์ได้แค่ $P \rightarrow Q$ ก็ยังถือว่าไม่จบ เพราะยังเหลืออีกทิศหนึ่งที่ต้องแสดง

2.9.3 ใช้ proof by cases แบบไม่ครบทุกกรณี

ถ้ามีเพียง $P \vee Q$ เราไม่อาจเลือกข้างเองว่ากรณี P เกิดขึ้น เพราะข้อมูลยังไม่พอ สิ่งที่ต้องทำคือแสดงให้ครบว่า ไม่ว่าจะเป็กรณิ P หรือกรณี Q ผลลัพธ์ที่ต้องการก็ยังคงตามมา

2.9.4 พิสูจน์แบบ contradiction โดยสมมติผิดตัว

ถ้าต้องการพิสูจน์ $P \rightarrow Q$ ด้วย contradiction สิ่งที่เราควรสมมติคือ $P \wedge \neg Q$ ไม่ใช่สมมติ $\neg Q$ อย่างลอย ๆ และไม่ใช่สมมติสิ่งเดียวกับที่ต้องการพิสูจน์แล้วประกาศว่าขัดแย้งทันที
ความขัดแย้งต้องเป็นสิ่งที่เกิดขึ้นจากเหตุผล ไม่ใช่เพียงคำบอกว่า “ขัดแย้ง”

2.10 สรุป

ใจความของบทนี้สามารถย่อได้ดังนี้

1. ตัวเชื่อมตรรกะไม่ได้มีไว้เพื่อเขียนตารางค่าความจริงเท่านั้น แต่มันบอกเราด้วยว่าพิสูจน์อย่างไร
2. ข้อความ $P \rightarrow Q$ เป็นเท็จได้เพียงกรณีเดียว คือ P จริงแต่ Q เท็จ
3. ถ้าพิสูจน์ $P \rightarrow Q$ ตรง ๆ ยาก เราอาจเปลี่ยนไปพิสูจน์ contrapositive $\neg Q \rightarrow \neg P$
4. การพิสูจน์แบบ contradiction คือการแสดงว่ากรณีตรงข้ามนำไปสู่สิ่งที่เป็นไปไม่ได้
5. ดิสจังก์ชัน $P \vee Q$ ต้องแยกให้ออกว่าเรากำลังสร้างมัน หรือกำลังใช้มัน
6. ข้อความ $P \leftrightarrow Q$ ต้องพิสูจน์สองทิศเสมอ
7. การพิสูจน์ที่ดีต้องไม่เพียงถูก แต่ต้องชัดด้วยว่าแต่ละบรรทัดมาจากอะไร

มุมมองที่ควรติดตัวไปต่อ

เมื่อเห็นข้อความทางคณิตศาสตร์ อย่าเพิ่งรีบคำนวณ ให้ถามก่อนว่า “รูปของข้อความนี้คืออะไร” เพราะบ่อยครั้ง วิธีพิสูจน์ที่เหมาะสมถูกซ่อนอยู่ในรูปของประโยคนั้นเอง

แบบฝึกหัดท้ายบท

Exercise 2.10.1. พิจารณาว่าข้อความต่อไปนี้จริงหรือเท็จ และถ้าเป็นเท็จให้หาตัวอย่างค้าน

1. $\forall n \in \mathbb{Z}, (n \text{ หารด้วย } 6 \text{ ลงตัว}) \rightarrow (n \text{ เป็นจำนวนคู่})$
2. $\forall n \in \mathbb{Z}, (n \text{ เป็นจำนวนคู่}) \rightarrow (n \text{ หารด้วย } 4 \text{ ลงตัว})$
3. $\forall p \in \mathbb{Z}, (p \text{ เป็นจำนวนเฉพาะ}) \rightarrow (p \text{ เป็นจำนวนคี่})$

Exercise 2.10.2. เขียนข้อความต่อไปนี้เป็นสัญลักษณ์ โดยให้ P, Q เป็นประพจน์

1. “ P only if Q ”
2. “ Q เป็นเงื่อนไขจำเป็นสำหรับ P ”
3. “ P เป็นเงื่อนไขเพียงพอสำหรับ Q ”
4. “ P ก็ต่อเมื่อ Q ”

Exercise 2.10.3. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า $a + b$ เป็นจำนวนคี่ แล้วอย่างน้อยหนึ่งใน a, b เป็นจำนวนคี่

Exercise 2.10.4. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า a และ b เป็นจำนวนคี่ทั้งคู่ แล้ว ab เป็นจำนวนคี่ แล้วใช้ผลลัพธ์นี้เพื่อสรุปอีกครั้งว่า ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

Exercise 2.10.5. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม n ,

$$n^2 \text{ เป็นจำนวนคู่ } \vee n^2 \text{ เป็นจำนวนคี่}$$

โดยใช้การแยกกรณีจากข้อเท็จจริงที่ว่า จำนวนเต็มทุกตัวเป็นจำนวนคู่หรือจำนวนคี่

Exercise 2.10.6. อธิบายว่าทำไมการพิสูจน์ข้อความ $P \leftrightarrow Q$ ด้วยการพิสูจน์เพียงทิศเดียวจึงยังไม่เพียงพอ พร้อมยกตัวอย่างข้อความจริงสักหนึ่งข้อความในรูป iff แล้วเขียนโครงพิสูจน์สองทิศให้ครบ

บทที่ 3

ตัวบ่งปริมาณและแพตเทิร์นพื้นฐานของการพิสูจน์

บทก่อนหน้าเราศึกษาตรรกศาสตร์ประพจน์ในฐานะ โครง ของการพิสูจน์ ใจความสำคัญคือ เมื่อเรามองเห็นรูปของข้อความ เรา มักมองเห็นวิธีพิสูจน์ที่เหมาะสมตามไปด้วย ข้อความในรูป ``ถ้า ... แล้ว ..." ชี้แนะให้เราเริ่มจากสมมติส่วนหน้า ข้อความในรูป ``ก็ต่อเมื่อ" บอกเราว่าต้องเดินเหตุผลให้ครบสองทิศ และข้อความที่มี ``หรือ" บังคับให้เราะวังว่าคำว่า ``หรือ" กำลังปรากฏ อยู่ในฐานะ ข้อมูลที่มีอยู่แล้ว หรือเป็น ข้อสรุปที่เราต้องการสร้าง

ในบทนี้เราจะเดินต่อจากจุดนั้นไปสู่ประโยคที่มีตัวบ่งปริมาณ ได้แก่ ``สำหรับทุก" และ ``มีอยู่" ซึ่งเป็นภาษามาตรฐานในการกล่าวอ้างเกี่ยวกับวัตถุจำนวนมาก โดยเฉพาะข้อความที่พบอยู่เสมอในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ เช่น

$$\forall n \in \mathbb{Z}, P(n), \quad \exists x \in U, Q(x), \quad \forall x \in U \exists y \in V, R(x, y).$$

เมื่อมีตัวบ่งปริมาณเข้ามา การพิสูจน์จะต้องระวังมากขึ้นอีกหนึ่งระดับ: เราไม่เพียงต้องดูว่าโครงของข้อความเป็นอิมพลีเคชันหรือไม่ แต่ต้องดูด้วยว่าเรากำลังพูดถึงวัตถุ *ทุกตัว* หรือเพียง *บางตัว* และหากเป็น ``บางตัว" เราต้องแสดงพยานอย่างไร หากเป็น ``ทุกตัว" เราต้องเลือกสมาชิกแบบใดจึงจะถูกต้อง

3.1 ทบทวน: รูปของข้อความบอกวิธีตั้งสมมติฐาน

ก่อนเข้าสู่ตัวบ่งปริมาณอย่างจริงจัง เราควรสรุปให้ชัดอีกครั้งว่า *assumption* หรือสมมติฐานของการพิสูจน์ ขึ้นกับ *รูปของประโยคที่กำลังพิสูจน์* ไม่ใช่ขึ้นกับความสะดวกของผู้เขียน

3.1.1 อิมพลีเคชัน

ถ้าเป้าหมายอยู่ในรูป

$$A \rightarrow B,$$

แล้วการพิสูจน์ตรง (*direct proof*) เริ่มจากการสมมติว่า A เป็นจริง และพาเหตุผลไปจนได้ B

โครงพื้นฐานของ direct proof

เมื่อจะพิสูจน์ $A \rightarrow B$

1. สมมติว่า A เป็นจริง
2. ใช้นิยาม ข้อเท็จจริง หรือผลลัพธ์ที่พิสูจน์แล้ว
3. สรุปว่า B เป็นจริง

ถ้าจะพิสูจน์แบบ *contrapositive* เราจะสมมติ A แต่จะสมมติ $\neg B$ แล้วพยายามสรุป $\neg A$ เพราะ

$$A \rightarrow B \quad \text{สมมูลกับ} \quad \neg B \rightarrow \neg A.$$

ถ้าจะพิสูจน์แบบ *contradiction* เราจะสมมติกรณีที่ทำให้ $A \rightarrow B$ ล้มเหลว นั่นคือ

$$A \wedge \neg B,$$

แล้วผลไปจนเกิดสิ่งที่เป็นไปไม่ได้

ข้อควรจำ

เวลาพิสูจน์ข้อความรูป $A \rightarrow B$

- direct: เริ่มจาก A
- contrapositive: เริ่มจาก $\neg B$
- contradiction: เริ่มจาก A และ $\neg B$

ดังนั้น ถ้าเริ่มพิสูจน์ผิดจุดตั้งแต่ต้น แม้บรรทัดถัด ๆ ไปจะดูสมเหตุสมผล พิสูจน์นั้นก็ยิ่งผิดโครงสร้างอยู่ดี

3.1.2 ไบคอนดิชันนัล

ข้อความ

$$A \leftrightarrow B$$

ไม่ได้หมายถึงการพิสูจน์เพียงทิศเดียว แต่หมายถึงการพิสูจน์ว่า

$$(A \rightarrow B) \wedge (B \rightarrow A).$$

ดังนั้น เมื่อจะพิสูจน์ข้อความในรูป iff เราต้องทำให้ครบทั้งสองส่วน

โครงพื้นฐานของ iff

เมื่อต้องการพิสูจน์ $A \leftrightarrow B$

1. พิสูจน์ $A \rightarrow B$
2. พิสูจน์ $B \rightarrow A$

การพิสูจน์เพียงทิศเดียวไม่เพียงพอ

3.1.3 คำว่า "หรือ" ปรากฏในบทบาทใด

ประโยคที่มี $\vee$ ต้องระวังเป็นพิเศษ เพราะรูปแบบการใช้ต่างกันตามตำแหน่งของมัน

ถ้า $P \vee Q$ เป็น ข้อมูลที่มีอยู่แล้ว และเราต้องการสรุป R เราต้องทำ *proof by cases* กล่าวคือพิสูจน์ให้ครบว่า

$$P \rightarrow R \quad \text{และ} \quad Q \rightarrow R.$$

แต่ถ้า $P \vee Q$ เป็น ข้อสรุปที่เราต้องการ เรามักเพียงพิสูจน์ให้ได้อย่างน้อยหนึ่งข้าง เช่น พิสูจน์ P ให้ได้โดยตรง หรือพิสูจน์ Q ให้ได้โดยตรง

สองบทบาทของ $\vee$

- ถ้ามี $P \vee Q$ เป็นข้อมูล และต้องการ R : แยกกรณี
- ถ้าต้องการพิสูจน์ $P \vee Q$: พิสูจน์ให้ได้สักข้างหนึ่งก็พอ

3.2 ตัวบ่งปริมาณและเอกภพสัมพัทธ์

เมื่อเราเขียนข้อความที่มีตัวแปร เรายังไม่ได้ประพจน์ที่มีค่าความจริงแน่นอนเสมอไป หลายครั้งเราจะได้เพียง *predicate* หรือ ประโยคเปิด ซึ่งจะกลายเป็นประพจน์ก็ต่อเมื่อกำหนดตัวแปรให้ชัด หรือครอบมันด้วยตัวบ่งปริมาณ

Definition 3.2.1. ให้ U เป็นเอกภพสัมพัทธ์

1. ตัวบ่งปริมาณสากล (universal quantifier)

$$\forall x \in U, P(x)$$

หมายความว่า $P(x)$ เป็นจริงสำหรับทุก x ใน U

2. ตัวบ่งปริมาณการมี (existential quantifier)

$$\exists x \in U, P(x)$$

หมายความว่าอย่างน้อยหนึ่ง x ใน U ที่ทำให้ $P(x)$ เป็นจริง

สิ่งที่ห้ามมองข้ามคือ เอกภพสัมพัทธ์ หรือโดเมน เพราะข้อความเดียวกันอาจเปลี่ยนค่าความจริงได้ทันทีเมื่อเปลี่ยนโดเมน

Example 3.2.2. ข้อความ

$$\exists x, x^2 = 2$$

ยังไม่สมบูรณ์พอถ้าไม่บอกว่า x มาจากไหน

ถ้า $x \in \mathbb{R}$ ข้อความนี้เป็นจริง แต่ถ้า $x \in \mathbb{Z}$ ข้อความนี้เป็นเท็จ

ดังนั้นการเขียนว่า “สำหรับทุก x ” โดยไม่บอกขอบเขตของ x มักทำให้ข้อความกำกวมโดยไม่จำเป็น โดยเฉพาะในบทพิสูจน์ที่ต้องตรวจสอบละเอียดที่ละบรรทัด

หลักการเขียนที่ควรยึด

เมื่อเขียนข้อความที่มีตัวแปร ให้พยายามระบุโดเมนให้ชัดเจน เช่น

$$\forall n \in \mathbb{Z}, \quad \exists x \in \mathbb{R}, \quad \forall user \in U.$$

3.3 เมื่อเป้าหมายเป็น $\forall$: เลือกสมาชิกแบบ arbitrary

การพิสูจน์ข้อความสากลมีรูปแบบที่ค่อนข้างตายตัว

แพตเทิร์นของการพิสูจน์ $\forall$

เมื่อต้องการพิสูจน์

$$\forall x \in U, P(x),$$

ให้เริ่มโดยเขียนว่า

$$\text{“ให้ } x \in U \text{ เป็นใด ๆ”}$$

จากนั้นพิสูจน์ว่า $P(x)$ เป็นจริง โดยใช้เพียงข้อเท็จจริงที่เป็นจริงสำหรับสมาชิกทั่วไปของ U

คำว่า “เป็นใด ๆ” สำคัญมาก เพราะมันเตือนเราว่า x ต้องเป็นสมาชิก *ทั่วไป* ไม่ใช่สมาชิกพิเศษที่เราเลือกเพราะสะดวกต่อการพิสูจน์

Example 3.3.1. ถ้าต้องการพิสูจน์

$$\forall n \in \mathbb{Z}, n^2 + n \text{ เป็นจำนวนคู่}$$

เราไม่อาจเริ่มด้วย “ให้ $n = 0$ ” แล้วพิสูจน์ว่าจริงสำหรับ 0 เพราะนั่นพิสูจน์เพียงกรณีเดียว ไม่ใช่ทุกกรณี

Theorem 3.3.2. สำหรับทุกจำนวนเต็ม n , ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

Proof idea. ข้อความนี้เป็นอิมพลีเคชันภายในตัวบ่งปริมาณสากล ดังนั้นให้ $n \in \mathbb{Z}$ เป็นใด ๆ แล้วสมมติส่วนหน้าของอิมพลีเคชัน จากนั้นแคะนิยามของจำนวนคู่

Proof. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ และสมมติว่า n เป็นจำนวนคู่ ตามนิยามของจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k.$$

ดังนั้น

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2).$$

เนื่องจาก $2k^2 \in \mathbb{Z}$ จึงได้ว่า n^2 อยู่ในรูป $2m$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n^2 เป็นจำนวนคู่ □

3.4 เมื่อเป้าหมายเป็น $\exists$: ระบุพยาน

การพิสูจน์ข้อความเชิงการมีมีแนวคิดต่างออกไป เราจะไม่เริ่มจากการเลือกสมาชิกทั่วไป แต่ต้อง *เสนอพยาน* ว่าใครคือวัตถุที่ทำให้เงื่อนไขเป็นจริง

แพตเทิร์นของการพิสูจน์ $\exists$

เมื่อต้องการพิสูจน์

$$\exists x \in U, P(x),$$

ให้เลือกหรือกำหนดสมาชิกตัวหนึ่ง $w \in U$ แล้วตรวจสอบว่า $P(w)$ เป็นจริง

คำว่า *witness* หรือพยาน ไม่ได้หมายถึงวัตถุที่สลับซับซ้อน แต่มักเป็นเพียงค่าที่เราเสนอออกมาอย่างชัดเจนว่า “ตัวนี้แหละใช้ได้”

Example 3.4.1. ข้อความ

$$\exists n \in \mathbb{Z}, n^2 = 4$$

พิสูจน์ได้โดยเลือกพยาน $n = 2$ เพราะ $2 \in \mathbb{Z}$ และ $2^2 = 4$

Example 3.4.2. ข้อความ

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z}, z + m = m$$

พิสูจน์ได้โดยเลือกพยาน $z = 0$ แล้วตรวจว่า $0 + m = m$ จริงสำหรับทุก $m \in \mathbb{Z}$

ในการเขียนพิสูจน์แบบมีอยู่ เรามักไม่จำเป็นต้องเขียนกระบวนการค้นหาพยานอย่างละเอียด สิ่งสำคัญกว่าคือ เมื่อเลือกพยานแล้ว ต้องตรวจสอบให้ชัดว่าพยานนั้นทำงานจริง

3.5 หลายตัวบ่งปริมาณ: ลำดับมีผล

เมื่อข้อความมีตัวบ่งปริมาณมากกว่าหนึ่งตัว ลำดับของมันมีผลโดยตรงต่อความหมาย

$$\forall x \exists y R(x, y) \quad \text{ต่างจาก} \quad \exists y \forall x R(x, y)$$

ประโยคสองประโยคนี้นี้อาจคล้ายกันมาก แต่จริง ๆ แล้วต่างกันอย่างมากมีนัยสำคัญ

3.5.1 แนวคิดเชิงภาษา

ข้อความ

$$\forall x \exists y R(x, y)$$

แปลว่า

“สำหรับทุก x เราสามารถหาค่า y ที่เหมาะกับ x ตัวนั้นได้”

ส่วนข้อความ

$$\exists y \forall x R(x, y)$$

แปลว่า

“มี y ตัวเดียวสักตัวหนึ่งที่ใช้ได้พร้อมกันสำหรับทุก x ”

ประโยคแบบแรกอนุญาตให้ y เปลี่ยนไปตาม x แต่ประโยคแบบหลังไม่อนุญาต เพราะ y ถูกเลือกก่อนแล้วต้องใช้ตัวเดิมกับทุก x

ข้อแตกต่างที่ต้องเห็นให้ชัด

ในประโยค

$$\forall x \exists y P(x, y),$$

พยาน y อาจขึ้นกับ x

แต่ในประโยค

$$\exists y \forall x P(x, y),$$

พยาน y ต้องเป็น ค่าคงที่ตัวเดียว ที่ใช้ได้กับทุก x

3.5.2 ตัวอย่างเชิงจำนวน

พิจารณาข้อความสองประโยคต่อไปนี้ใน $\mathbb{Z}$

$$\forall x \in \mathbb{Z} \exists y \in \mathbb{Z} : y > x$$

และ

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : y > x.$$

ข้อความแรกเป็นจริง เพราะเมื่อให้ $x \in \mathbb{Z}$ มาใด ๆ เราเลือก

$$y = x + 1$$

ได้เสมอ และแน่นอนว่า $y > x$

แต่ข้อความที่สองเป็นเท็จ เพราะไม่มีจำนวนเต็มตัวใดที่มากกว่าจำนวนเต็มทุกตัวพร้อมกัน ไม่ว่าคุณจะเสนอ y ตัวไหน เราก็เลือก $x = y + 1$ เพื่อให้ $y > x$ ล้มเหลวได้ทันที

3.5.3 ตัวอย่างเชิงระบบคอมพิวเตอร์

ให้ U เป็นเซตผู้ใช้งานทั้งหมด และ R เป็นเซตทรัพยากรทั้งหมดในระบบ โดย $allowed(user, resource)$ หมายความว่า ผู้ใช้งานคนนั้นเข้าถึงทรัพยากรนั้นได้

ประโยค

$$\forall user \in U \exists resource \in R, allowed(user, resource)$$

หมายความว่า

“ผู้ใช้งานทุกคนมีทรัพยากรอย่างน้อยหนึ่งอย่างที่เข้าถึงได้”

แต่ประโยค

$$\exists resource \in R \forall user \in U, allowed(user, resource)$$

หมายความว่า

“มีทรัพยากรชิ้นหนึ่งที่ผู้ใช้งานทุกคนเข้าถึงได้”

ประโยคหลังเข้มกว่าอย่างชัดเจน เพราะต้องการทรัพยากร *ร่วม* ตัวเดียว ไม่ใช่ทรัพยากรคนละตัวกัน

3.6 การปฏิเสธตัวบ่งปริมาณ

การหาตัวอย่างค้านหรือการพิสูจน์โดยวิธีแย้ง มักบังคับให้เราปฏิเสธข้อความที่มีตัวบ่งปริมาณ ดังนั้นสมมูลต่อไปนี้จึงสำคัญมาก

กฎปฏิเสธตัวบ่งปริมาณ

$$\neg(\forall x \in U, P(x)) \equiv \exists x \in U, \neg P(x),$$

$$\neg(\exists x \in U, P(x)) \equiv \forall x \in U, \neg P(x).$$

หลักจำคือ

“ปฏิเสธ $\forall$ แล้วกลายเป็น $\exists$, ปฏิเสธ $\exists$ แล้วกลายเป็น $\forall$ ”

พร้อมกับต้องผลัดเครื่องหมายปฏิเสธเข้าไปที่เงื่อนไขภายในด้วย

Example 3.6.1. นิเสธของข้อความ

$$\forall x \in U, P(x)$$

คือ

$$\exists x \in U, \neg P(x),$$

ซึ่งแปลว่า ``มีอย่างน้อยหนึ่งตัวที่ไม่ทำให้ P จริง"

ไม่ใช่

$$\forall x \in U, \neg P(x),$$

ซึ่งเป็นข้อความที่แรงกว่ามาก เพราะแปลว่า ``ไม่มีตัวไหนเลยที่ทำให้ P จริง"

Example 3.6.2. ถ้าเราปฏิเสธข้อความ

$$\forall n \in \mathbb{Z}, \exists k \in \mathbb{Z} : n < k$$

เราจะได้

$$\exists n \in \mathbb{Z}, \forall k \in \mathbb{Z} : n \geq k.$$

กล่าวคือ ``มีจำนวนเต็มตัวหนึ่งที่มีมากกว่าหรือเท่ากับจำนวนเต็มทุกตัว" ซึ่งใน $\mathbb{Z}$ เป็นไปไม่ได้

3.7 ตัวอย่างทฤษฎีบทสม $\forall \exists$

ข้อความที่มีทั้ง $\forall$ และ $\exists$ มักต้องใช้สองแพดเทิร์นพร้อมกัน: เริ่มจากเลือกตัวทั่วไปสำหรับ $\forall$ แล้วจึงสร้างพยานสำหรับ $\exists$

Theorem 3.7.1. สำหรับทุกจำนวนเต็ม n จะมีจำนวนเต็ม k ที่ทำให้

$$n(n+1) = 2k.$$

Proof idea. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ แล้วแยกกรณีว่า n เป็นจำนวนคู่หรือจำนวนคี่ ในแต่ละกรณีจะสามารถจัดรูป $n(n+1)$ ให้อยู่ในรูป $2k$ ได้

Proof. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ เราพิจารณาเป็นสองกรณี

กรณีที่ 1: n เป็นจำนวนคู่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้

$$n = 2\ell.$$

ดังนั้น

$$n(n+1) = (2\ell)(2\ell+1) = 2(\ell(2\ell+1)).$$

ให้

$$k = \ell(2\ell+1).$$

เนื่องจาก $\ell \in \mathbb{Z}$ จึงได้ว่า $k \in \mathbb{Z}$ และมี

$$n(n+1) = 2k.$$

กรณีที่ 2: n เป็นจำนวนคี่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้

$$n = 2\ell + 1.$$

ดังนั้น

$$n(n+1) = (2\ell+1)(2\ell+2) = 2((2\ell+1)(\ell+1)).$$

ให้

$$k = (2\ell+1)(\ell+1).$$

แล้ว $k \in \mathbb{Z}$ และ

$$n(n+1) = 2k.$$

ในทุกกรณี เราหาจำนวนเต็ม k ได้ที่ทำให้ $n(n+1) = 2k$ ดังนั้น

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n(n+1) = 2k.$$

เป็นจริง

□

จุดที่พลาดบ่อย

เวลาเห็นข้อความ

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n(n+1) = 2k$$

ห้ามเลือก n แบบเฉพาะเจาะจง เช่น $n = 0$ แล้วสรุปทันทีว่า “จริงสำหรับทุก n ” เพราะสิ่งที่ต้องพิสูจน์คือ ทุก n ไม่ใช่เพียงกรณีตัวอย่าง

3.8 ตัวอย่างทฤษฎีบทแบบ $\exists\forall$

Theorem 3.8.1. มีจำนวนเต็ม z ตัวหนึ่งที่ทำให้สำหรับทุกจำนวนเต็ม m ,

$$z + m = m.$$

Proof idea. เดาพยานก่อนว่าควรเป็นใคร แล้วจึงตรวจว่าพยานนั้นใช้ได้กับทุก m

Proof. เลือก

$$z = 0.$$

ชัดเจนว่า $z \in \mathbb{Z}$

ต่อไปให้ $m \in \mathbb{Z}$ เป็นใด ๆ จะได้ว่า

$$z + m = 0 + m = m.$$

ดังนั้น $z = 0$ ใช้ได้กับทุก $m \in \mathbb{Z}$ จึงสรุปว่า

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z} : z + m = m.$$

เป็นจริง □

ตัวอย่างนี้เรียบง่ายมาก แต่มีประโยชน์เชิงโครงสร้างสูง เพราะมันบังคับให้เห็นชัดว่าในประโยคแบบ $\exists \forall$ เราต้องเลือกพยานเพียงครั้งเดียว แล้วใช้พยานตัวเดิมกับทุกค่าในภายหลัง

3.9 การหักล้างข้อความที่มีตัวบ่งปริมาณ

การหักล้างข้อความที่มีตัวบ่งปริมาณ มักเริ่มจากการปฏิเสธมันอย่างถูกต้องก่อน แล้วจึงมองหาว่าจะให้ตัวอย่างค้านหรือเหตุผลแบบใด

Theorem 3.9.1. ข้อความ

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x < y$$

เป็นเท็จ

Proof idea. ถ้าข้อความนี้จริง จะมีจำนวนเต็มตัวหนึ่งที่มีมากกว่าจำนวนเต็มทุกตัว แต่ไม่ว่าคุณจะเสนอ y ตัวไหน เราสามารถเลือก $x = y + 1$ เพื่อให้เงื่อนไข $x < y$ ล้มเหลวได้

Proof. สมมติว่า

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x < y$$

เป็นจริง ดังนั้นจะมีจำนวนเต็ม y ตัวหนึ่งที่ทำให้

$$x < y \quad \text{สำหรับทุก } x \in \mathbb{Z}.$$

แต่เมื่อ $y \in \mathbb{Z}$ เราสามารถเลือก

$$x = y + 1$$

ซึ่งเป็นจำนวนเต็มเช่นกัน แล้วจะได้ว่า

$$y + 1 < y,$$

ซึ่งเป็นไปไม่ได้

เกิดข้อขัดแย้ง จึงสรุปได้ว่าข้อความเดิมเป็นเท็จ □

ในอีกมุมหนึ่ง เราอาจหักล้างได้แบบตรงไปตรงมาว่า ``สำหรับทุก $y \in \mathbb{Z}$ ถ้าเสนอ y มา เราเลือก $x = y + 1$ ได้" ซึ่งเป็นรูปแบบการคิดที่มีประโยชน์มากเมื่อเจอข้อความเชิงการมีที่ดูน่าสงสัย

3.10 คลินิกข้อผิดพลาดที่พบบ่อย

ส่วนนี้รวบรวมข้อผิดพลาดจากงานลักษณะ workshop แต่เขียนใหม่ในรูปที่อ่านแบบหนังสือได้

3.10.1 สมมติผิดด้านของอิมพลิเคชัน

ถ้าต้องการพิสูจน์

$$A \rightarrow B$$

แต่กลับเริ่มด้วย ``สมมติว่า B '' นั้นไม่ใช่ direct proof ของข้อความเดิม สิ่งที่กำลังทำอยู่ใกล้เคียงกับการพิสูจน์ converse หรือข้อความอีกอันหนึ่งแทน

Example 3.10.1. ถ้าต้องการพิสูจน์ว่า

ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

แต่เริ่มจากสมมติว่า n^2 เป็นจำนวนคู่ นั้นไม่ใช่ direct proof ของข้อความนี้

3.10.2 พิสูจน์ iff เพียงทิศเดียว

ถ้าต้องการพิสูจน์

$$n \text{ เป็นจำนวนคู่} \leftrightarrow n^2 \text{ เป็นจำนวนคู่}$$

แล้วพิสูจน์ได้แค่

$$n \text{ คู่} \rightarrow n^2 \text{ คู่}$$

ก็ยังไม่จบ เพราะยังเหลืออีกทิศหนึ่ง

$$n^2 \text{ คู่} \rightarrow n \text{ คู่}$$

ซึ่งอาจพิสูจน์ได้ด้วย contrapositive จากข้อความ ``ถ้า n คู่ แล้ว n^2 คู่"

3.10.3 ใช้ contradiction แต่สมมติผิด

เมื่อต้องการพิสูจน์ $A \rightarrow B$ ด้วย contradiction สิ่งที่ต้องสมมติไม่ใช่ $\neg B$ อย่างลอย ๆ และไม่ใช่สิ่งเดียวกับที่ต้องการพิสูจน์ แต่ต้องสมมติกรณีที่ทำให้ข้อความเดิมล้มเหลว นั่นคือ

$$A \wedge \neg B.$$

ตัวอย่างเช่น ถ้าจะพิสูจน์ว่า

ถ้า n เป็นจำนวนคู่ แล้ว n ไม่เป็นจำนวนคู่

โดย contradiction เราต้องเริ่มจากสมมติว่า

n เป็นจำนวนคู่ และ n เป็นจำนวนคี่

แล้วพาไปสู่ข้อขัดแย้ง ไม่ใช่เริ่มจาก ``สมมติว่า n ไม่เป็นจำนวนคี่"

3.10.4 เลือกค่าคงที่ตัวเดียวแทนที่จะให้ขึ้นกับตัวแปรก่อนหน้า

ข้อผิดพลาดที่พบบ่อยมากในข้อความแบบ

$$\forall x \exists y P(x, y)$$

คือรับเลือก y เป็นค่าคงที่ค่าหนึ่งทันที ราวกับกำลังพิสูจน์

$$\exists y \forall x P(x, y)$$

ซึ่งเป็นคนละข้อความ

Example 3.10.2. ข้อความ

$$\forall x \in \mathbb{Z} \exists y \in \mathbb{Z} : y > x$$

จะพิสูจน์ไม่ได้ถ้าเริ่มจาก ``ให้ $y = 0$ " เพราะ 0 ไม่ได้มากกว่า x ทุกตัว วิธีที่ถูกต้องคือต้องปล่อยให้ y ขึ้นกับ x เช่นเลือก $y = x + 1$

3.11 ตัวอย่างเสริมเกี่ยวกับการแปลความหมาย

ก่อนจะพิสูจน์ข้อความที่มี quantifier การแปลเป็นประโยคภาษาไทยให้ชัดมักช่วยได้มาก

Example 3.11.1. ให้ $R(x, t)$ แปลว่า ``โปรแกรมบนอินพุต x จบภายในเวลาไม่เกิน t " โดย $t \in \mathbb{N}$
ข้อความ

$$\forall x \exists t \in \mathbb{N}, R(x, t)$$

แปลว่า

``สำหรับทุกอินพุต มีเวลาบางค่าที่ทำให้โปรแกรมจบบนอินพุตนั้น"

ส่วนข้อความ

$$\exists t \in \mathbb{N} \forall x, R(x, t)$$

แปลว่า

``มีเวลาค่าหนึ่งที่ใช้ได้กับทุกอินพุต"

ข้อความหลังแรงกว่ามาก เพราะต้องการ bound เดียวที่ครอบคลุมทุกอินพุต

ตัวอย่างเช่นนี้มีความสำคัญมากในวิทยาการคอมพิวเตอร์ เพราะมันสัมพันธ์โดยตรงกับการแยกความต่างระหว่าง ``เวลาอาจเปลี่ยนไปตามอินพุต" กับ ``มี worst-case bound เดียวที่ใช้กับทุกอินพุต"

3.12 สรุปแพตเทิร์นของบทนี้

เราสามารถย่อใจความของบทนี้ให้อยู่ในแพตเทิร์นหลักไม่กี่ข้อได้ดังนี้

Pattern Summary

1. ถ้าเป้าหมายเป็น $A \rightarrow B$: เริ่มจาก A
2. ถ้าเป้าหมายเป็น $A \leftrightarrow B$: พิสูจน์สองทิศ
3. ถ้ามี $P \vee Q$ เป็นข้อมูลและต้องการ R : ทำ proof by cases
4. ถ้าเป้าหมายเป็น $\forall x \in U, P(x)$: ให้ $x \in U$ เป็นใด ๆ
5. ถ้าเป้าหมายเป็น $\exists x \in U, P(x)$: ระบุพยานแล้วตรวจให้ชัด
6. ระวังลำดับตัวบ่งปริมาณ เพราะ

$$\forall x \exists y R(x, y) \neq \exists y \forall x R(x, y)$$

7. ปฏิเสธ quantifier ให้ถูก:

$$\neg \forall = \exists \neg, \quad \neg \exists = \forall \neg$$

แบบฝึกหัดท้ายบท

Exercise 3.12.1. จงอธิบายความแตกต่างของข้อความต่อไปนี้เป็นภาษาไทยอย่างชัดเจน

$$\forall \text{student} \in U \exists \text{problem} \in H, \text{solved}(\text{student}, \text{problem})$$

และ

$$\exists \text{problem} \in H \forall \text{student} \in U, \text{solved}(\text{student}, \text{problem}).$$

Exercise 3.12.2. จงเขียนปฏิเสธของข้อความต่อไปนี้ให้ถูกต้อง

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n < k.$$

Exercise 3.12.3. จงพิสูจน์ว่า

$$\forall n \in \mathbb{Z}, \exists m \in \mathbb{Z} : m > n.$$

Exercise 3.12.4. จงพิสูจน์ว่า

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z} : m + z = m.$$

Exercise 3.12.5. จงหักล้างข้อความต่อไปนี้

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x \leq y.$$

Exercise 3.12.6. จงพิสูจน์ว่า

$$\forall n \in \mathbb{Z}, \exists k \in \mathbb{Z} : n^2 + n = 2k.$$

โดยเขียนในรูป

Claim $\rightarrow$ **Proof idea** $\rightarrow$ **Proof** $\rightarrow$ **QED**

ให้ครบ

ภาค II

การแก้ปัญหาเชิงกระบวนการและการเวียนเกิด

บทที่ 4

แนวคิดพื้นฐานของการเวียนเกิด

หัวข้อนี้ต่างจากบทก่อนหน้าอยู่พอสมควร เพราะเป้าหมายหลักไม่ใช่การพิสูจน์ข้อความเชิงตรรกะโดยตรง แต่เป็นการฝึก *ออกแบบวิธีคิด* สำหรับปัญหาที่มีโครงสร้างซ้ำ กล่าวอย่างง่ายที่สุด การคิดแบบเวียนเกิดคือการพยายามตอบคำถามต่อไปนี้เสมอว่า

ถ้าฉันแก้ปัญหขนาดเล็กลงได้แล้ว ฉันจะเอาคำตอบนั้นมาประกอบเป็นคำตอบของปัญหาปัจจุบันได้อย่างไร

ในมุมมองของการเขียนโปรแกรม ความคิดนี้มีปรากฏในรูปของฟังก์ชันที่เรียกใช้ตัวเอง แต่แก่นแท้ของ *recursion* ไม่ได้อยู่ที่การ ``เรียกตัวเอง`` แต่อยู่ที่การนิยามปัญหาหนึ่งด้วยปัญหา *ชนิดเดิม* แต่มีขนาดเล็กลง พร้อมกับมีกรณีฐานที่เล็กจนตอบได้ทันทีโดยไม่ต้องย้อนลงไปอีก

4.1 Recursion

การเวียนเกิด (recursion) คือรูปแบบการแก้ปัญหา (หรือการคำนวณ) ปัญหาหนึ่ง ๆ ที่อาศัยผลลัพธ์จากการแก้ปัญหาเดียวกันแต่ *input* เล็กกว่านำมาประกอบขึ้นเป็นผลลัพธ์ของปัญหาที่กำลังแก้

ซึ่งในทางการเขียนโปรแกรมนั้น *input* ที่เล็กกว่ามักจะเป็นส่วนย่อยของ *input* ที่อยากแก้ปัญหา ตัวอย่างเช่น

- อยากแก้ปัญหาที่รับ array $A[1 \dots n]$ เป็น *input* โดยใช้ผลลัพธ์จากโจทย์เดียวกันแต่แก้กับ $A[1 \dots (n - 1)]$ (กล่าวคือลองดูกับแค่ตัวแรกจนถึงตัวรองสุดท้ายก่อนว่าได้ผลลัพธ์เป็นอะไร แล้วค่อยนำคนสุดท้ายมาพิจารณาพร้อมผลลัพธ์ที่ได้มา)
- อยากคำนวณค่าของโจทย์หนึ่งที่รับ n เข้าไป โดยใช้ผลลัพธ์จากโจทย์เดียวกันที่คำนวณกับ $n - 1$ และ $n - 2$ มาแล้ว

การแก้ปัญหาทาง computational บนโลกนี้มี 2 แนวคิดหลัก

1. **Iterative programming:** ใช้ *loop* ที่ผู้พัฒนาตั้ง *logic* การแก้ปัญหาในแต่ละรอบไว้แบบชัดเจนแล้ว -- อ่านง่าย(สำหรับคนส่วนใหญ่) วิธีการแก้ปัญหาเหมือนพาจับมือทำ ภาษาโปรแกรมส่วนใหญ่ออกแบบมาเพื่อเขียนแบบนี้อยู่แล้ว แต่การได้วิธีการแก้บางโจทย์ค่อนข้างยากและไม่ตรงไปตรงมา และตรวจสอบความถูกต้องทางทฤษฎีได้ลำบาก

2. **Recursive programming:** ฟังก์ชันเรียกใช้ตัวเองเพื่อเอาคำตอบจากปัญหาที่เล็กกว่า -- บางคนบอกว่าอ่านยาก(แต่สำหรับผม code คลีนกว่า) วิธีการได้มาของผลลัพธ์เหมือนต้องอาศัยความรู้เก่าเลยไม่รู้ว่าได้มาอย่างไร ภาษาแบบ **functional programming** ถูกพัฒนาเพื่อเขียนแบบนี้ (เช่น Haskell, Scala) แต่ภาษาที่ **iterative** ก็ทำได้ แต่ว่าการคิดวิธีการแก้ไขที่ยากก่อนข้างชัดเจนและเป็นธรรมชาติกับวิธีคิดมากกว่า และที่สำคัญ ตรวจสอบความถูกต้องของการทำงานของโปรแกรมด้วยการพิสูจน์ทางคณิตศาสตร์ได้ (อาจารย์คิดว่าวิธีคิดของมนุษย์ใกล้เคียงการแก้ปัญหาแบบ **recursive** มากกว่า **iterative** เพราะปกติมนุษย์มักใช้ประสบการณ์ในการลองแก้ปัญหาที่ง่ายกว่าก่อนแล้วค่อยพัฒนาผลลัพธ์ปัญหาง่ายให้เป็นผลลัพธ์ของปัญหาที่ยากขึ้น)

4.1.1 Recursion ในฐานรูปแบบของการแก้ปัญหา

Definition 4.1.1. การเวียนเกิด (recursion) คือวิธีนิยามหรือแก้ปัญหาปัญหาหนึ่ง โดยอาศัยผลลัพธ์ของปัญหาเดียวกันในกรณีที่เล็กกว่า ร่วมกับ **กรณีฐาน** (base case) ที่ตอบได้โดยตรง

นิยามนี้มีองค์ประกอบสำคัญอยู่สามส่วน

1. **ปัญหขนาดเล็กลง**
ต้องมีความหมายชัดเจนว่า ``เล็กกว่า" อย่างไร เช่น ใช้ข้อมูลน้อยลงหนึ่งตัว ใช้จำนวนเต็มที่เล็กลงหนึ่งหน่วย หรือใช้โครงสร้างย่อยของวัตถุเดิม
2. **กฎสำหรับประกอบคำตอบ**
เมื่อรู้คำตอบของกรณีเล็กกว่าแล้ว ต้องอธิบายให้ได้ว่าจะประกอบเป็นคำตอบของกรณีปัจจุบันอย่างไร
3. **กรณีฐาน**
ถ้าไม่มีกรณีฐาน การนิยามแบบ recursion จะไม่หยุด และจึงไม่ให้ความหมายที่ใช้งานได้จริง

สิ่งที่ต้องมีเสมอ

ถ้าจะนิยามสิ่งใดด้วย recursion ให้ตรวจสอบอย่างนี้เสมอ:

1. ปัญหาเล็กลงจริงหรือไม่
2. จากคำตอบเดิมประกอบคำตอบใหม่ได้จริงหรือไม่
3. มีกัณฑ์หลุมหรือยัง กล่าวคือมี base case หรือยัง

4.1.2 เปรียบเทียบกับการทำแบบวนซ้ำ

ในทางการเขียนโปรแกรม เรามักพบสองแนวคิดหลักในการแก้ปัญหาเชิงคำนวณ

1. **การทำแบบวนซ้ำ (iteration):**
ผู้เขียนกำหนดลำดับการทำงานที่ละรอบผ่าน loop อย่างชัดเจน
2. **การทำแบบเวียนเกิด (recursion):**
ผู้เขียนนิยามคำตอบของปัญหาปัจจุบันจากคำตอบของปัญหาย่อยชนิดเดิม

ทั้งสองวิธีสามารถให้ผลลัพธ์เดียวกันได้ในหลายโจทย์ แต่ให้ “มุมมอง” ต่อปัญหาต่างกัน iteration เน้นลำดับคำสั่ง ส่วน recursion เน้นโครงสร้างของปัญหา

ในรายวิชานี้ เราให้ความสำคัญกับ recursion เพราะมันช่วยบังคับให้เราเห็นว่า ปัญหาหนึ่งแตกออกเป็นปัญหาลึกกว่าอย่างไร และในหลายกรณี ความคิดแบบนี้เป็นธรรมชาติอย่างมากสำหรับทั้งคณิตศาสตร์และวิทยาการคอมพิวเตอร์

4.2 ตัวอย่างอุปกรณ์: การนิยามลำดับแบบเวียนเกิด

ก่อนจะไปสู่ปัญหาประเภทอัลกอริทึม เราจะเริ่มจากตัวอย่างที่คุ้นเคยอย่างลำดับเชิงตัวเลข เพราะตัวอย่างเหล่านี้ทำให้เห็นโครงสร้างของ recursion ได้ชัดที่สุด

4.2.1 ลำดับเลขคณิต

ลำดับเลขคณิตคือ ลำดับที่ผลต่างระหว่างพจน์ติดกันมีค่าคงที่เสมอ ถ้าเราใช้ $a(n)$ แทนพจน์ที่ n และให้ d เป็นผลต่างร่วม เราจะเขียนนิยามแบบ recursion ได้ว่า

$$a(0) = a_0, \quad a(n) = a(n-1) + d \quad \text{สำหรับทุก } n \geq 1.$$

ประโยคนี้น่าจะอ่านว่า พจน์ที่ n ไม่ได้เกิดขึ้นอย่างลอย ๆ แต่ได้มาจากพจน์ก่อนหน้า แล้วบวกด้วยค่าคงที่เดิม d

Example 4.2.1. ถ้า

$$a(0) = 5, \quad d = 3$$

แล้วลำดับนี้เริ่มเป็น

$$5, 8, 11, 14, 17, \dots$$

เพราะ

$$a(1) = a(0) + 3 = 8, \quad a(2) = a(1) + 3 = 11, \quad a(3) = a(2) + 3 = 14.$$

ดังนั้น

$$a(7) = 26.$$

จุดสำคัญของตัวอย่างนี้ไม่ได้อยู่ที่การหาคำตอบ 26 แต่อยู่ที่การเห็นว่าการนิยามแบบ recursion มีลักษณะเป็น

“รู้ค่าหนึ่งค่าเริ่มต้น แล้วเดินไปข้างหน้าทีละขั้น”

4.2.2 ลำดับฟีโบนัชชี

อีกตัวอย่างมาตรฐานหนึ่งคือลำดับฟีโบนัชชี ซึ่งแต่ละพจน์ได้มาจากผลบวกของสองพจน์ก่อนหน้า

$$Fib(0) = 1, \quad Fib(1) = 1,$$

และสำหรับทุก $n \geq 2$,

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2).$$

ในที่นี้เราต้องมีกรณีฐาน สองค่า เพราะความสัมพันธ์เวียนเกิดอ้างอิงถึงสองพจน์ก่อนหน้า ดังนั้นถ้ามีเพียง $\text{Fib}(0)$ ค่าเดียว เรายังเริ่มคำนวณต่อไม่ได้

Example 4.2.2. จากนิยามข้างต้นจะได้ว่า

$$\text{Fib}(2) = \text{Fib}(1) + \text{Fib}(0) = 1 + 1 = 2,$$

$$\text{Fib}(3) = \text{Fib}(2) + \text{Fib}(1) = 2 + 1 = 3,$$

$$\text{Fib}(4) = \text{Fib}(3) + \text{Fib}(2) = 3 + 2 = 5,$$

$$\text{Fib}(5) = \text{Fib}(4) + \text{Fib}(3) = 5 + 3 = 8.$$

ข้อสังเกต

จากตัวอย่างลำดับเลขคณิตและฟีโบนัชชี เราเริ่มเห็นโครงซ้ำของ recursion แล้ว:

1. มีค่าฐาน
2. มีสูตรที่บอกว่าจะสร้างค่าถัดไปจากค่าก่อนหน้าอย่างไร
3. ยิ่งสูตรอ้างอิงย้อนหลังหลายชั้น ก็ยิ่งต้องมีฐานมากพอ

4.3 Recursion ในฐานะขั้นตอนวิธี: Tower of Hanoi

ตัวอย่างต่อไปนี้อาจสำคัญมาก เพราะแสดงให้เห็น recursion ไม่ใช่แค่การนิยามลำดับ แต่เป็นการออกแบบ วิธีทำงาน อย่างเป็นขั้นตอน

4.3.1 กติกาของปัญหา

เกม Tower of Hanoi มีเสาสามต้น ซึ่งเราจะเรียกว่า A, B, C และมีแผ่นดิสก์หลายแผ่นวางซ้อนกันอยู่บนเสาต้นหนึ่ง โดยแผ่นเล็กกว่าจะอยู่บนแผ่นใหญ่กว่าเสมอ เราต้องการย้ายกองแผ่นทั้งหมดจากเสาต้นเริ่มต้นไปยังเสาปลายทาง ภายใต้กติกาสองข้อคือ

1. ย้ายได้ครั้งละหนึ่งแผ่นเท่านั้น
2. ห้ามวางแผ่นใหญ่บนแผ่นเล็ก

4.3.2 เริ่มจากกรณี 4 แผ่น

สมมติว่าต้องการย้ายดิสก์ 4 แผ่นจากเสา A ไปเสา C โดยใช้เสา B เป็นตัวช่วย ถ้าเรามองเฉพาะแผ่นล่างสุด ซึ่งเป็นแผ่นใหญ่ที่สุด จะพบว่ามันย้ายได้เพียงครั้งเดียว และจะย้ายได้ก็ต่อเมื่อแผ่นเล็กอีก 3 แผ่นด้านบนถูกย้ายออกไปหมดแล้ว

นี่คือจุดที่ recursion โผล่ขึ้นมาอย่างเป็นธรรมชาติ: ถ้าเรามี “ผู้ช่วย” ที่สามารถย้ายกอง 3 แผ่นได้สำเร็จเสมอ เราก็สามารถใช้ผู้ช่วยคนนั้นเป็นส่วนหนึ่งของแผนสำหรับ 4 แผ่นได้ทันที

โครงของวิธีแก้จึงเป็นดังนี้

1. ย้ายแผ่น 3 แผ่นบนสุดจาก A ไป B
2. ย้ายแผ่นใหญ่สุดจาก A ไป C
3. ย้ายแผ่น 3 แผ่นจาก B ไป C

สิ่งที่น่าสนใจคือ ขั้นตอนที่ 1 และ 3 เป็น “ปัญหาชนิดเดิม” เพียงแต่ขนาดเล็กลงจาก 4 แผ่นเหลือ 3 แผ่นเท่านั้น

4.3.3 รูปแบบทั่วไปสำหรับ n แผ่น

จากตัวอย่าง 4 แผ่น เราเห็นรูปแบบทั่วไปทันที ถ้าต้องการย้าย n แผ่นจากเสาด้านทางไปยังเสาปลายทาง โดยใช้อีกเสาหนึ่งเป็นตัวช่วย เราต้องทำ 3 ขั้นตอน

ขั้นตอนวิธีแบบ recursion ของ Tower of Hanoi

เพื่อย้าย n แผ่นจากเสา X ไปเสา Y โดยใช้เสา Z เป็นตัวช่วย:

1. ย้าย $n - 1$ แผ่นบนสุดจาก X ไป Z
2. ย้ายแผ่นใหญ่สุดจาก X ไป Y
3. ย้าย $n - 1$ แผ่นจาก Z ไป Y

ส่วนกรณีฐานคือ $n = 1$ เพราะถ้ามีเพียงแผ่นเดียว เราเพียงย้ายมันตรงจากด้านทางไปปลายทางได้ทันที

4.3.4 จำนวนครั้งที่ต้องย้าย

ให้ $T(n)$ แทนจำนวนครั้งน้อยที่สุดที่ต้องใช้ในการย้าย Tower of Hanoi n แผ่น จากขั้นตอนวิธีข้างต้น เราจะได้ว่า

$$T(1) = 1,$$

และสำหรับทุก $n \geq 2$,

$$T(n) = T(n - 1) + 1 + T(n - 1) = 2T(n - 1) + 1.$$

นี่เป็นความสัมพันธ์เวียนเกิดที่ได้จากการวิเคราะห์ปัญหาโดยตรง: ย้าย $n - 1$ แผ่นครั้งหนึ่ง, ย้ายแผ่นใหญ่สุดอีกครั้งหนึ่ง, แล้วจึงย้าย $n - 1$ แผ่นอีกครั้ง

Example 4.3.1. จากสมการ

$$T(1) = 1, \quad T(n) = 2T(n-1) + 1$$

จะได้ว่า

$$T(2) = 2T(1) + 1 = 3,$$

$$T(3) = 2T(2) + 1 = 7,$$

$$T(4) = 2T(3) + 1 = 15.$$

ในบทนี้ เรายังเน้นที่การ *ตั้งสมการเวียนเกิด* จากโครงสร้างของปัญหาเป็นหลัก ส่วนการหาสูตรปิด เช่น

$$T(n) = 2^n - 1$$

และการพิสูจน์ว่าสูตรดังกล่าวถูกต้อง จะเป็นหัวข้อที่สัมพันธ์กับบทอุปนัยต่อไป

4.4 อีกตัวอย่างหนึ่ง: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น

ปัญหานี้เป็นตัวอย่างที่ติดมากของการคิดว่า ``เมื่อเพิ่มวัตถุใหม่เข้ามาอีกหนึ่งชิ้น มันสร้างผลกระทบเพิ่มขึ้นเท่าไร"

Example 4.4.1. ให้ $A(n)$ แทนจำนวนชิ้นมากที่สุดที่สามารถเกิดขึ้นได้เมื่อเราใช้เส้นตรง n เส้นแบ่งระนาบ จงหาความสัมพันธ์เวียนเกิดของ $A(n)$

ถ้ายังไม่วางเส้นเลย ระนาบทั้งแผ่นเป็นเพียงชิ้นเดียว ดังนั้น

$$A(0) = 1.$$

ที่นี้สมมติว่าเราได้วางเส้นไว้แล้ว $n-1$ เส้น และแบ่งระนาบได้มากที่สุดเท่าที่จะทำได้ คำถามคือ เมื่อเติมเส้นที่ n ลงไปอีกหนึ่งเส้น จะเพิ่มจำนวนชิ้นได้มากที่สุดกี่ชิ้น

เพื่อให้ได้มากที่สุด เส้นที่ n ควรตัดกับเส้นเดิมทุกเส้น และไม่ควรผ่านจุดตัดเดิมใด ๆ ดังนั้นบนเส้นใหม่จะมีจุดตัด $n-1$ จุด ซึ่งแบ่งเส้นใหม่นี้ออกเป็น n ท่อนย่อย

แต่ละท่อนย่อยอยู่ในชิ้นเดิมชิ้นหนึ่งของระนาบ และเมื่อเราใช้ท่อนย่อยนั้นตัดผ่านชิ้นเดิม มันจะเพิ่มจำนวนชิ้นขึ้นอีกหนึ่งชิ้น ดังนั้นเส้นที่ n จะเพิ่มจำนวนชิ้นได้มากที่สุด n ชิ้น

สรุปว่า

$$A(n) = A(n-1) + n \quad \text{สำหรับทุก } n \geq 1.$$

Example 4.4.2. จาก

$$A(0) = 1, \quad A(n) = A(n-1) + n$$

จะได้ว่า

$$A(1) = 2, \quad A(2) = 4, \quad A(3) = 7, \quad A(4) = 11.$$

จุดเด่นของปัญหานี้คือ recursion ไม่ได้มาจากการ “เรียกตัวเอง” แบบ Tower of Hanoi แต่มาจากการถามว่า การเปลี่ยนจากกรณี $n - 1$ ไปเป็นกรณี n สร้างของใหม่เพิ่มขึ้นเท่าไร

4.5 การนับจำนวนวิธีด้วยการแยกกรณี: จำนวนเซตย่อย

ให้ $S(n)$ แทนจำนวนเซตย่อยทั้งหมดของเซต

$$\{1, 2, 3, \dots, n\}.$$

เราต้องการหาความสัมพันธ์เวียนเกิดของ $S(n)$

เริ่มจากกรณีฐานก่อน ถ้า $n = 0$ เรากำลังพูดถึงเซตว่าง ซึ่งมีเซตย่อยเพียงเซตเดียว คือเซตว่างเอง ดังนั้น

$$S(0) = 1.$$

ทีนี้ให้ $n \geq 1$ พิจารณาเซตย่อยใด ๆ ของ $\{1, 2, \dots, n\}$ เซตย่อยนั้นจะอยู่ในหนึ่งในสองกรณี และไม่มีกรณีอื่นนอกจากนั้น

1. ไม่มีสมาชิก n
2. มี สมาชิก n

ถ้าไม่มีก็เทียบได้พอดีกับเซตย่อยของ $\{1, 2, \dots, n - 1\}$

ถ้ามี n อยู่ด้วย เมื่อเอา n ออก จะเหลือเซตย่อยหนึ่งของ $\{1, 2, \dots, n - 1\}$

ดังนั้นจำนวนเซตย่อยในสองกรณีนี้เท่ากันทั้งคู่ คือเท่ากับ $S(n - 1)$ จึงได้ว่า

$$S(n) = S(n - 1) + S(n - 1) = 2S(n - 1) \quad \text{สำหรับทุก } n \geq 1.$$

Example 4.5.1. จาก

$$S(0) = 1, \quad S(n) = 2S(n - 1)$$

จะได้ว่า

$$S(1) = 2, \quad S(2) = 4, \quad S(3) = 8, \quad S(4) = 16.$$

นี่เป็นอีกตัวอย่างหนึ่งที่มีความสำคัญมาก เพราะรูปแบบการนับแบบ “แยกเป็นกรณีที่มีวัตถุชิ้นสุดท้าย” กับ “ไม่มีวัตถุชิ้นสุดท้าย” จะกลับมาปรากฏซ้ำอีกหลายครั้งในคณิตศาสตร์ discrete

4.6 สรุปภาพรวมของการตั้งความสัมพันธ์เวียนเกิด

จากตัวอย่างทั้งหมดในบทนี้ เราจะเห็นว่าแม้โจทย์จะหน้าตาแตกต่างกันมาก แต่รูปแบบการคิดมีแกนร่วมกันอยู่

1. ระบุปริมาณที่ต้องการหา

เช่น $a(n)$, $Fib(n)$, $T(n)$, $A(n)$, $S(n)$

2. ถามว่ากรณีขนาด n สัมพันธ์กับกรณีขนาดเล็กกว่าอย่างไร
บางครั้งคือ $n - 1$, บางครั้งคือ $n - 2$, และบางครั้งต้องใช้มากกว่าหนึ่งกรณี
3. ระบุกรณีฐาน
เพื่อให้การนิยามเริ่มต้นได้จริง
4. ตรวจสอบว่าความสัมพันธ์มีเหตุผลเชิงโครงสร้าง
กล่าวคือ สูตรไม่ได้เดามาลอย ๆ แต่สะท้อนการประกอบคำตอบจากปัญหาย่อยจริง

Pattern Summary

การตั้ง recursion ที่ดีมักตอบคำถามสี่ข้อให้ได้:

1. เรากำลังหาค่าอะไร
2. ค่าของขนาด n สร้างจากขนาดเล็กกว่าอย่างไร
3. ต้องมี base case อะไรบ้าง
4. เหตุผลของสูตรนี้มาจากโครงสร้างของปัญหาใด

4.7 ข้อควรระวัง

ผู้เรียนมักพลาดอยู่ไม่กี่แบบเมื่อเริ่มตั้ง recursion

4.7.1 มี recursive step แต่ไม่มี base case

ถ้ามีเพียงสูตรเช่น

$$F(n) = F(n - 1) + 3$$

แต่ไม่บอกว่าเริ่มต้นที่ค่าใด สูตรนี้ยังคำนวณจริงไม่ได้

4.7.2 ปัญหาไม่เล็กลงจริง

ถ้าเขียนนิยามที่ยังย้อนกลับไปหาปัญหขนาดเดิม หรือขนาดไม่ได้ลดลงอย่างแน่นอน นิยามนั้นจะไม่สิ้นสุด

4.7.3 สูตรไม่สะท้อนโครงสร้างจริงของปัญหา

บางครั้งผู้เรียนเดาสมการจากคำตอบสองสามค่าตัวแรก แต่ไม่สามารถอธิบายได้ว่าทำไมจึงต้องเป็นสูตรนั้น ในรายวิชานี้เราสนใจ recursion ที่มาจาก เหตุผลของปัญหา ไม่ใช่แค่การทายรูปแบบของตัวเลข

แบบฝึกหัดท้ายบท

Exercise 4.7.1. กำหนดลำดับโดย

$$b(0) = 2, \quad b(n) = b(n-1) + 5 \text{ สำหรับทุก } n \geq 1.$$

จงหาค่า $b(1), b(2), b(3), b(6)$

Exercise 4.7.2. กำหนดลำดับโดย

$$G(0) = 2, \quad G(1) = 3, \quad G(n) = G(n-1) + G(n-2) \text{ สำหรับทุก } n \geq 2.$$

จงหาค่า $G(2), G(3), G(4), G(5)$

Exercise 4.7.3. ให้ $T(n)$ แทนจำนวนครั้งน้อยที่สุดในการย้าย Tower of Hanoi n แผ่น จงอธิบายด้วยภาษาของตนเองว่าทำไม

$$T(n) = 2T(n-1) + 1$$

จึงเป็นสมการที่สมเหตุสมผล

Exercise 4.7.4. ให้ $A(n)$ แทนจำนวนชิ้นมากที่สุดที่ได้จากการใช้เส้นตรง n เส้นแบ่งระนาบ จงหาค่า $A(1), A(2), A(3), A(4), A(5)$ เมื่อกำหนดว่า

$$A(0) = 1, \quad A(n) = A(n-1) + n.$$

Exercise 4.7.5. ให้ $S(n)$ แทนจำนวนเซตย่อยของ $\{1, 2, \dots, n\}$ จงอธิบายว่าทำไมเซตย่อยทุกเซตจึงถูกแบ่งได้เป็นสองชนิด คือ เซตที่มี n และเซตที่ไม่มี n แล้วใช้แนวคิดนี้เพื่อเสนอ recursion ของ $S(n)$

บทที่ 5

Mathematical Induction

ในบทก่อนหน้านี้ เราใช้แนวคิดแบบเวียนเกิดเพื่อ นิยาม ลำดับ นิยามเซตบางชนิด และอธิบายวิธีคิดที่สร้างคำตอบจากปัญหาที่ย่อยลง แต่เมื่อได้สูตรหรือได้วัตถุที่นิยามแบบ recursion มาแล้ว ยังมีคำถามสำคัญอีกข้อหนึ่งเสมอคือ

เราจะพิสูจน์ได้อย่างไรว่าสิ่งที่นิยามไว้นั้นมีคุณสมบัติที่ต้องการจริงสำหรับทุกขนาด?

ตัวอย่างเช่น ถ้าเราคาดเดาว่า

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

หรือถ้านิยามลำดับด้วยความสัมพันธ์เวียนเกิดแล้วคาดเดาสูตรปิดบางอย่างขึ้นมา เราจะไม่พอใจเพียงการลองแทนค่า $n = 1, 2, 3, 4$ แล้วเห็นว่า “น่าจะจริง” เพราะในคณิตศาสตร์ เราต้องการเหตุผลที่ครอบคลุมทุกค่าที่เป็นไปได้ ไม่ใช่เพียงตัวอย่างไม่กี่ค่า

เครื่องมือพื้นฐานที่สุดสำหรับคำถามลักษณะนี้คือ **อุปนัยทางคณิตศาสตร์** ซึ่งเหมาะสมอย่างยิ่งกับข้อความที่เรียงตามขั้นของจำนวนนับ เช่นข้อความตระกูล

$$P(0), P(1), P(2), \dots$$

โดยที่ $P(n)$ เป็นประโยคที่ขึ้นกับดัชนี n

ภาพเปรียบเทียบที่ใช้กันบ่อยคือแถวโดมิโน: ถ้าเราทำให้ตัวแรกเริ่มล้มได้ และถ้าทุกครั้งที่ตัวที่ k ล้มแล้วตัวที่ $k+1$ จะล้มตามเสมอ เราก็สรุปได้ว่าโดมิโนทุกตัวในแถวจะล้มทั้งหมด ในบริบทของคณิตศาสตร์ สิ่ง “ล้ม” ไม่ใช่แผ่นโดมิโน แต่คือความจริงของข้อความ $P(n)$

5.1 Ordinary Induction

5.1.1 หลักการพื้นฐาน

Theorem 5.1.1 (Principle of Mathematical Induction). ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนนับตั้งแต่ค่าเริ่มต้น n_0 เป็นต้นไป ถ้าเราพิสูจน์ได้ว่า

1. ขั้นฐาน (base step): $P(n_0)$ เป็นจริง

2. **ขั้นอุปนัย (inductive step):** สำหรับทุก $k \geq n_0$, ถ้า $P(k)$ เป็นจริง แล้ว $P(k + 1)$ เป็นจริง

จะสรุปได้ว่า

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

แก่นของหลักการนี้อยู่ตรงความคิดว่า เราไม่จำเป็นต้องพิสูจน์ทีละค่า $n = 0, 1, 2, 3, \dots$ แบบไม่สิ้นสุด แต่เราพิสูจน์เพียงสองอย่างแทน: เริ่มได้ถูกต้องหนึ่งครั้ง และเดินต่อจากขั้นหนึ่งไปขั้นถัดไปได้เสมอ เมื่อสองอย่างนี้ครบ จึงสรุปได้ว่าทุกขั้นเป็นจริงทั้งหมด

ในทางปฏิบัติ เรามักเขียนการพิสูจน์แบบอุปนัยในโครงมาตรฐานดังนี้

โครงสร้างมาตรฐานของการพิสูจน์แบบอุปนัย

ให้ $P(n)$ แทนข้อความที่ต้องการพิสูจน์

1. กำหนดข้อความ ให้ชัดว่า $P(n)$ คืออะไร
2. **ขั้นฐาน** พิสูจน์ว่า $P(n_0)$ เป็นจริง
3. **ขั้นอุปนัย** ให้ $k \geq n_0$ เป็นค่าทั่วไป และสมมติว่า $P(k)$ เป็นจริง
4. ใช้สมมติฐานขั้นอุปนัยนั้นพิสูจน์ว่า $P(k + 1)$ เป็นจริง
5. สรุปด้วยหลักอุปนัยว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

สมมติฐานที่ว่า $P(k)$ เป็นจริง เรียกว่า **สมมติฐานขั้นอุปนัย (induction hypothesis)**

5.1.2 สิ่งที่ $P(n)$ เป็น และสิ่งที่มันไม่ใช่

จุดที่นักศึกษามักสับสนตั้งแต่ต้นคือ ไม่เห็นความต่างระหว่าง $P(n)$, $P(k)$, และ $P(k + 1)$

- $P(n)$ คือ รูปแบบของข้อความ ที่ยังเปิดตัวแปร n ไว้
- $P(k)$ คือข้อความเดียวกัน แต่แทน n ด้วยค่าทั่วไป k แล้ว
- $P(k + 1)$ คือข้อความที่อยากพิสูจน์ในขั้นถัดไป

ดังนั้น เมื่อเข้าสู่ขั้นอุปนัย เราจะไม่สามารถพูดว่า “จาก $P(k)$ แทน k เป็น $k + 1$ จึงได้ $P(k + 1)$ ” เพราะนั่นไม่ใช่เหตุผล แต่เป็นเพียงการเปลี่ยนสัญลักษณ์ สิ่งที่ต้องทำจริง ๆ คือ ใช้เนื้อหาของ $P(k)$ เป็นข้อมูลตั้งต้น แล้วเดินเหตุผลไปให้ถึง $P(k + 1)$

ข้อควรจำ

อุปนัยไม่ใช่การ “เดาแพตเทิร์นแล้วเปลี่ยน k เป็น $k + 1$ ” แต่อุปนัยคือการพิสูจน์ว่า

$$\text{ถ้าข้อความจริงที่ระดับ } k \implies \text{ข้อความจริงที่ระดับ } k + 1$$

โดยมีเหตุผลรองรับจริง

5.1.3 ข้อผิดพลาดที่พบบ่อย

ก่อนจะไปดูตัวอย่าง ควรแยกข้อผิดพลาดที่พบบ่อยที่สุดออกมาก่อน เพราะถ้าไม่ระวัง จุดเหล่านี้จะทำให้พิสูจน์ผิดแม้เข้าใจโอเคโดยรวมนแล้ว

1. เริ่มขั้นฐานผิดค่า โจทย์บางข้อเริ่มที่ $n \geq 1$ บางข้อเริ่มที่ $n \geq 0$ ถ้าเลือกฐานผิดตั้งแต่แรก บทพิสูจน์ทั้งหมดอาจพังทันที
2. สมมติฐานขั้นอุปนัยแรงเกินจำเป็น ถ้ากำลังใช้อุปนัยธรรมดา ควรสมมติเพียง $P(k)$ ไม่ใช่แบบสมมติ $P(0), P(1), \dots, P(k)$ ทั้งหมด เพราะนั่นคือ strong induction
3. เริ่มจากสิ่งที่อยากพิสูจน์แล้วจัดรูปย้อนกลับ วิธีเขียนที่ดีควรเริ่มจากสิ่งที่รู้ว่าเป็นจริง เช่น สมมติฐานขั้นอุปนัย นิยาม หรือข้อเท็จจริงที่พิสูจน์แล้ว แล้วค่อยเดินไปหาสิ่งที่ต้องการ
4. สันนิษฐานว่า k เป็นค่าทั่วไป ในขั้นอุปนัย k ไม่ใช่ค่าคงที่เฉพาะตัว แต่เป็นตัวแทนของทุกค่าที่อยู่ในโดเมนที่โจทย์กำหนด

5.2 ตัวอย่างแรก: ผลบวกของจำนวนเต็มตัวแรก

ตัวอย่างแรกของบทนี้ควรเป็นสูตรที่เรียบง่ายพอให้เห็นโครงสร้างของอุปนัยชัด โดยยังไม่ถูกรบกวนด้วยการจัดรูปที่ซับซ้อนเกินไป สูตรมาตรฐานที่สุดคือ

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Theorem 5.2.1. สำหรับทุก $n \geq 0$,

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2},$$

โดยตกลงว่าเมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่างและมีค่าเป็น 0

Proof idea. กำหนด

$$P(n) : 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}.$$

ขั้นฐานตรงตรง ๆ ที่ $n = 0$ ส่วนขั้นอุปนัยใช้สมมติฐานสำหรับผลบวกถึง k แล้วเติม $(k+1)$ เข้าไปอีกหนึ่งพจน์

Proof. ให้

$$P(n) : 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}.$$

ขั้นฐาน: เมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่าง จึงมีค่าเป็น 0 และด้านขวาเป็น

$$\frac{0(0+1)}{2} = 0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า $P(k)$ เป็นจริง นั่นคือ

$$1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2}.$$

เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง กล่าวคือ

$$1 + 2 + 3 + \cdots + k + (k+1) = \frac{(k+1)(k+2)}{2}.$$

เริ่มจากด้านซ้ายของข้อความที่ต้องการพิสูจน์:

$$1 + 2 + 3 + \cdots + k + (k+1).$$

ใช้สมมติฐานขั้นอุปนัยแทนผลบวกส่วนแรก จะได้

$$\frac{k(k+1)}{2} + (k+1).$$

จัดรูปต่อเป็น

$$\frac{k(k+1)}{2} + \frac{2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

ซึ่งตรงกับด้านขวาของ $P(k+1)$ ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วยหลักอุปนัยว่า

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

เป็นจริงสำหรับทุก $n \geq 0$

□

5.2.1 สิ่งที่ว่าอย่างนี้สอนเรา

ตัวอย่างนี้สำคัญไม่ใช่เพราะสูตรใหม่ แต่เพราะมันแสดงรูปแบบการทำงานของ induction อย่างชัดเจน: เรามีข้อความที่ระดับ k แล้วใช้มันเป็น “สะพาน” พาเราไปยังข้อความที่ระดับ $k+1$

กล่าวอีกแบบหนึ่ง อุปนัยไม่ได้พิสูจน์ทุกค่าพร้อมกัน แต่มันพิสูจน์ว่า “ถ้าจริงถึงตรงนี้ ก็จริงต่อไปได้อีกหนึ่งก้าว” ซึ่งเพียงพอเมื่อจับคู่กับขั้นฐาน

5.2.2 ผลบวกกำลังสอง

หลังจากเห็นสูตรผลบวกของจำนวนเต็มตัวแรกแล้ว ตัวอย่างถัดไปที่เป็นธรรมชาติคือผลบวกกำลังสอง ซึ่งยังอยู่ในโลกของ ordinary induction แต่ต้องจัดรูปมากขึ้นอีกเล็กน้อย จึงช่วยให้เห็นชัดว่า induction ไม่ได้พึ่งเพียงการท่องแพตเทิร์น แต่ต้องอาศัยการจัดรูปอย่างมีเป้าหมายด้วย

Theorem 5.2.2. สำหรับทุก $n \geq 0$,

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

Proof idea. กำหนด

$$P(n) : 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

ขั้นฐานตรวจได้ตรง ๆ ส่วนขั้นอุปนัยใช้สมมติฐานสำหรับผลบวกถึง k แล้วบวก $(k+1)^2$ เข้าไปอีกหนึ่งพจน์ ก่อนจัดรูปให้กลายเป็นสูตรของ $P(k+1)$

Proof. ให้

$$P(n) : 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

ขั้นฐาน: เมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่างจึงมีค่าเป็น 0 และด้านขวาเป็น

$$\frac{0(0+1)(2 \cdot 0 + 1)}{6} = 0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$1^2 + 2^2 + \cdots + k^2 = \frac{k(k+1)(2k+1)}{6}.$$

ต้องการพิสูจน์ว่า

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}.$$

เริ่มจากด้านซ้ายของข้อความที่ต้องการพิสูจน์:

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2.$$

ใช้สมมติฐานขั้นอุปนัย จะได้

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2.$$

ดึง $(k+1)$ ออกเป็นตัวร่วม:

$$(k+1) \left(\frac{k(2k+1)}{6} + k+1 \right).$$

เขียน $k+1$ ให้อยู่ในส่วน 6:

$$(k+1) \left(\frac{k(2k+1) + 6(k+1)}{6} \right).$$

จัดรูปในวงเล็บ:

$$(k+1) \left(\frac{2k^2 + k + 6k + 6}{6} \right) = (k+1) \left(\frac{2k^2 + 7k + 6}{6} \right).$$

แยกตัวประกอบได้เป็น

$$2k^2 + 7k + 6 = (2k+3)(k+2).$$

จึงได้ว่า

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วยหลักอุปนัยว่า

$$1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

สำหรับทุก $n \geq 0$

□

5.3 อีกสองตัวอย่างของ ordinary induction

หลังจากเห็นตัวอย่างแรกแล้ว เราควรดูอีกสองแบบที่มีลักษณะแตกต่างกัน: แบบหนึ่งเป็นผลบวกที่มีแพตเทิร์นเฉพาะ อีกแบบหนึ่งเป็นอสมการ เพื่อให้เห็นว่า induction ไม่ได้ใช้เฉพาะกับสมการผลบวกเท่านั้น

5.3.1 ผลบวกของจำนวนคี่

Theorem 5.3.1. สำหรับทุก $n \geq 0$,

$$1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

Proof idea. กำหนด

$$P(n) : 1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

ประเด็นสำคัญคือ เมื่อเพิ่มจำนวนคี่ตัวถัดไป ผลรวมจะขยับจากกำลังสองหนึ่งไปยังกำลังสองถัดไปพอดี

Proof. ให้

$$P(n) : 1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$1 = (0+1)^2.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$1 + 3 + 5 + \cdots + (2k+1) = (k+1)^2.$$

ต้องการพิสูจน์ว่า

$$1 + 3 + 5 + \cdots + (2k + 1) + (2k + 3) = (k + 2)^2.$$

ใช้สมมติฐานขั้นอุปนัยกับด้านซ้าย จะได้

$$(k + 1)^2 + (2k + 3) = k^2 + 2k + 1 + 2k + 3 = k^2 + 4k + 4 = (k + 2)^2.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปได้ว่า

$$1 + 3 + 5 + \cdots + (2n + 1) = (n + 1)^2$$

สำหรับทุก $n \geq 0$

□

5.3.2 ตัวอย่างอสมการ

Theorem 5.3.2. สำหรับทุก $n \geq 1$,

$$3^n > 2^n.$$

Proof idea. ในกรณีของอสมการ เรามักต้องระวังเรื่องการคูณหรือการบวกว่ารักษาทิศของอสมการหรือไม่ ในที่นี้ประเด็นง่ายมาก เพราะเราคูณด้วยจำนวนบวกเท่านั้น

Proof. ให้

$$P(n) : 3^n > 2^n.$$

ขั้นฐาน: เมื่อ $n = 1$,

$$3^1 = 3 > 2 = 2^1.$$

ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$3^k > 2^k.$$

ต้องการพิสูจน์ว่า

$$3^{k+1} > 2^{k+1}.$$

จากสมมติฐานขั้นอุปนัย เมื่อคูณทั้งสองข้างด้วย 3 จะได้

$$3^{k+1} > 3 \cdot 2^k.$$

และเพราะ $3 > 2$ จึงมี

$$3 \cdot 2^k > 2 \cdot 2^k = 2^{k+1}.$$

ดังนั้น

$$3^{k+1} > 2^{k+1}.$$

จึงได้ว่า $P(k+1)$ เป็นจริง

สรุปว่า

$$3^n > 2^n$$

สำหรับทุก $n \geq 1$

□

5.4 Induction กับสูตรปิดของลำดับที่นิยามแบบเวียนเกิด

ถึงตรงนี้ ภาพสำคัญเริ่มปรากฏชัด: *recursion* ใช้บอกเราว่าวัตถุหรือค่าถัดไปเกิดจากอะไร ส่วน *induction* ใช้พิสูจน์ว่าคำอธิบายแบบปิดที่เราคาดเดาไว้นั้นถูกต้องจริง

ตรงนี้เป็นจุดเชื่อมธรรมชาติระหว่างบท *recursion* กับบท *induction* เพราะเมื่อนิยามลำดับด้วยความสัมพันธ์เวียนเกิดแล้ว สิ่งที่เราอยากทำต่อคือหาสูตรที่ไม่ต้องย้อนกลับหลายครั้ง แล้วใช้ *induction* รับรองว่าสูตรนั้นถูกต้อง

5.4.1 ลำดับเลขคณิต

Theorem 5.4.1. ให้ลำดับ $a(n)$ นิยามโดย

$$a(0) = s, \quad a(n) = a(n-1) + d \quad \text{สำหรับทุก } n \geq 1.$$

แล้วสำหรับทุก $n \geq 0$,

$$a(n) = s + nd.$$

Proof idea. นี่เป็นตัวอย่างมาตรฐานของความสัมพันธ์ระหว่าง *recursion* กับ *induction*: นิยามเวียนเกิดบอกว่า “แต่ละขั้นเพิ่ม d ” ส่วนสูตรปิดบอกว่า “หลังเดินมา n ขั้น จะเพิ่มรวม nd ”

Proof. ให้

$$P(n) : a(n) = s + nd.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$a(0) = s = s + 0d.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$a(k) = s + kd.$$

จากนิยามเวียนเกิด

$$a(k+1) = a(k) + d.$$

แทนสมมติฐานขั้นอุปนัย จะได้

$$a(k+1) = s + kd + d = s + (k+1)d.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า

$$a(n) = s + nd$$

สำหรับทุก $n \geq 0$

□

5.4.2 ลำดับเรขาคณิต

ตัวอย่างลำดับเลขคณิตแสดงให้เห็นว่า recursion แบบ “บวกค่าคงที่ทีละขั้น” นำไปสู่สูตรปิดเชิงเส้นใน n ตัวอย่างถัดไปคือกรณีที่แต่ละขั้นไม่ได้บวกค่าคงที่ แต่คูณด้วยค่าคงที่แทน ซึ่งนำไปสู่สูตรปิดในรูปเลขยกกำลัง

Theorem 5.4.2. ให้ลำดับ $g(n)$ นิยามโดย

$$g(0) = s, \quad g(n) = g(n-1)r \quad \text{สำหรับทุก } n \geq 1.$$

แล้วสำหรับทุก $n \geq 0$,

$$g(n) = sr^n.$$

Proof. ให้

$$P(n) : g(n) = sr^n.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$g(0) = s = sr^0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$g(k) = sr^k.$$

จากนิยามเวียนเกิด

$$g(k+1) = g(k)r.$$

แทนสมมติฐานขั้นอุปนัย จะได้

$$g(k+1) = sr^k \cdot r = sr^{k+1}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า

$$g(n) = sr^n$$

สำหรับทุก $n \geq 0$

□

สองตัวอย่างนี้ชี้ให้เห็น pattern ที่สำคัญมาก: นิยามแบบ recursion มักทำหน้าที่เป็น “กฎอัปเดตจากขั้นก่อนหน้า” ส่วน induction ใช้พิสูจน์ว่า “กฎอัปเดตนั้นสะสมออกมาเป็นสูตรทั่วไปที่เราคาดเดาได้”

5.5 อีกตัวอย่างหนึ่งที่มีโครงสร้างมากขึ้น: การปูบอร์ดด้วย L-tromino

ตัวอย่างก่อนหน้านี้นั้นส่วนใหญ่เป็นสูตรผลบวก อสมการ และสูตรปิดของลำดับ ซึ่งล้วนเป็นข้อความที่มีหน้าตาเชิงพีชคณิตค่อนข้างชัด แต่ induction ไม่ได้มีประโยชน์เฉพาะกับสูตรเชิงตัวเลขเท่านั้น มันยังใช้กับปัญหาเชิงโครงสร้างได้ด้วย และตัวอย่างคลาสสิกที่สุดตัวอย่างหนึ่งคือการปูบอร์ดด้วยบล็อกรูปตัว L

Theorem 5.5.1. ให้ $n \geq 1$ พิจารณาบอร์ดขนาด $2^n \times 2^n$ ที่มีช่องหายไปหนึ่งช่อง แล้วจะสามารถปูช่องที่เหลือทั้งหมดได้พอดีด้วยบล็อกรูปตัว L ซึ่งแต่ละบล็อกปิดได้ 3 ช่อง

Proof idea. กรณี 2×2 ที่หายไปหนึ่งช่องปูได้ทันทีด้วยบล็อกตัว L หนึ่งชิ้น สำหรับบอร์ดขนาดใหญ่ขึ้น ให้แบ่งบอร์ดออกเป็น 4 ควอดแรนต์เท่า ๆ กัน ช่องที่หายไปจริงจะอยู่ในหนึ่งควอดแรนต์ จากนั้นวางบล็อกตัว L หนึ่งชิ้นตรงกลาง เพื่อให้ควอดแรนต์อีกสามส่วน "มีช่องหายไป" ส่วนละหนึ่งช่องเช่นกัน แล้วใช้สมมติฐานขั้นอุปนัยกับทั้งสี่ควอดแรนต์

Proof. ให้

$P(n)$: บอร์ดขนาด $2^n \times 2^n$ ที่มีช่องหายไปหนึ่งช่อง สามารถปูที่เหลือด้วยบล็อกตัว L ได้พอดี.

ขั้นฐาน: เมื่อ $n = 1$ บอร์ดมีขนาด 2×2 ถ้าหายไปหนึ่งช่อง จะเหลือ 3 ช่องพอดี ซึ่งปิดได้ด้วยบล็อกตัว L หนึ่งชิ้น ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า $P(k)$ เป็นจริง เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาบอร์ดขนาด $2^{k+1} \times 2^{k+1}$ ที่มีช่องหายไปหนึ่งช่อง แบ่งบอร์ดนี้ออกเป็น 4 ควอดแรนต์ แต่ละควอดแรนต์มีขนาด $2^k \times 2^k$

ช่องที่หายไปเดิมจะอยู่ในหนึ่งควอดแรนต์แน่นอน ตอนนี้พิจารณาช่อง 4 ช่องที่อยู่ติดจุดกึ่งกลางของบอร์ด ซึ่งแต่ละช่องอยู่คนละควอดแรนต์ วางบล็อกตัว L หนึ่งชิ้นปิดช่องกึ่งกลาง 3 ช่องที่อยู่ในสามควอดแรนต์ ซึ่งเดิมไม่มีช่องหายไป

ผลคือ:

- ควอดแรนต์ที่มีช่องหายไปเดิมอยู่แล้ว ยังคงเป็นบอร์ด $2^k \times 2^k$ ที่มีช่องหายไปหนึ่งช่อง
- อีกสามควอดแรนต์ ก็กลายเป็นบอร์ด $2^k \times 2^k$ ที่มีช่องหายไปหนึ่งช่องเช่นกัน

ดังนั้นจากสมมติฐานขั้นอุปนัย เราสามารถปูแต่ละควอดแรนต์ที่เหลือได้พอดีด้วยบล็อกตัว L เมื่อรวมกับบล็อกตรงกลางที่วางไว้ก่อนหน้า จึงปูทั้งบอร์ดได้พอดี

ดังนั้น $P(k+1)$ เป็นจริง และสรุปด้วยอุปนัยว่า ข้อความเป็นจริงสำหรับทุก $n \geq 1$ □

5.6 จาก ordinary induction สู่ structural induction

ordinary induction ทำงานได้ดีเมื่อวัตถุหรือข้อความถูกจัดเรียงตามเลข n ทีละขั้น แต่ในหลายกรณี สิ่งที่เราสนใจไม่ได้ถูกอธิบายว่า "เป็นกรณีที่ n " แต่อธิบายว่า "ถูกสร้างขึ้นจากกฎบางอย่าง"

ตัวอย่างเช่น เราอาจนิยามเซต R ว่า

1. มีสมาชิกตั้งต้นบางตัวอยู่ใน R
2. ถ้านำสมาชิกเดิมใน R มาประกอบตามกฎที่กำหนด จะได้สมาชิกใหม่ที่ยังอยู่ใน R

ถ้าเราต้องการพิสูจน์ว่า “สมาชิกทุกตัวของ R มีคุณสมบัติบางอย่าง” วิธีที่เป็นธรรมชาติที่สุดคือพิสูจน์ตาม *วิธีที่สมาชิกเหล่านั้นถูกสร้างขึ้น* นั่นคือพิสูจน์ว่าฐานมีคุณสมบัติ และพิสูจน์ว่าถ้าของเดิมมีคุณสมบัติแล้ว ของใหม่ที่สร้างจากมันก็ยังมีคุณสมบัตินั้น แนวคิดนี้เรียกว่า **structural induction**

5.7 Structural Induction

5.7.1 หลักทั่วไป

โครงทั่วไปของ structural induction

สมมติว่าเซต R ถูกนิยามแบบเวียนเกิดด้วย

1. กฎฐาน ที่ระบุสมาชิกตั้งต้นของ R
2. กฎสร้าง ที่บอกว่าถ้ามีสมาชิกบางตัวอยู่ใน R แล้ว จะสร้างสมาชิกใหม่ตัวใดให้อยู่ใน R ได้อีก

ถ้าต้องการพิสูจน์ว่า “สมาชิกทุกตัวของ R มีคุณสมบัติ Q ” ให้ทำสองอย่าง:

1. พิสูจน์ว่าสมาชิกฐานทุกตัวมีคุณสมบัติ Q
2. พิสูจน์ว่าถ้าสมาชิกเดิมที่ใช้ในกฎสร้างมีคุณสมบัติ Q แล้วสมาชิกใหม่ที่สร้างขึ้นก็มีคุณสมบัติ Q ด้วย

หัวใจของมันคือ: แทนที่จะพิสูจน์ตามเลข n , เราพิสูจน์ตาม *ประวัติการสร้างวัตถุ*

5.7.2 การพิสูจน์ความเท่ากันของเซต $R = A$

ในโจทย์หลายข้อ เรามีเซตสองชุดดังนี้

- R : เซตที่นิยามแบบ recursion
- A : เซตที่นิยามด้วยคุณสมบัติที่เราเชื่อว่า R ควรเท่ากับมัน

ถ้าต้องการพิสูจน์ว่า

$$R = A$$

มักแยกเป็นสองส่วน:

แพตเทิร์นของการพิสูจน์ $R = A$

1. พิสูจน์ว่า $R \subseteq A$
กล่าวคือ ทุกสิ่งที่สร้างได้จากกฎของ R ล้วนมีคุณสมบัติของสมาชิกใน A
2. พิสูจน์ว่า $A \subseteq R$
กล่าวคือ ทุกสิ่งที่ควรอยู่ใน A สามารถสร้างได้จริงจากกฎของ R

โดยทั่วไป ทิศ $R \subseteq A$ มักใช้ structural induction เพราะเรากำลังวิ่งตามกฎการสร้างของ R ส่วนทิศ $A \subseteq R$ มักใช้ ordinary induction หรือวิธีอื่นที่เหมาะสมกับรูปของ A

5.7.3 ตัวอย่าง: เซตของกำลังของ 2

Theorem 5.7.1. ให้เซต R นิยามโดย

1. $2 \in R$
2. ถ้า $x \in R$ แล้ว $2x \in R$

แล้ว

$$R = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทิศ $R \subseteq A$ แปลว่า “ทุกสิ่งที่สร้างจากกฎนี้เป็นกำลังของ 2” จึงเหมาะกับ structural induction ส่วนทิศ $A \subseteq R$ แปลว่า “กำลังของ 2 ทุกตัวสร้างได้จริง” จึงใช้อุปนัยธรรมดาบนเลขชี้กำลัง n

Proof. ให้

$$A = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction ตามกฎของ R

ขั้นฐาน:

$$2 = 2^1 \in A.$$

ขั้นอุปนัย: ให้ $x \in R$ และสมมติว่า $x \in A$ ดังนั้นมี $m \geq 1$ ที่ทำให้

$$x = 2^m.$$

จากกฎสร้างของ R จะได้ว่า $2x \in R$ และ

$$2x = 2^{m+1} \in A.$$

จึงได้ว่า $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 2^n \in R.$$

ขั้นฐาน:

$$2^1 = 2 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า $2^k \in R$ จากกฎของ R จะได้ว่า

$$2^{k+1} = 2 \cdot 2^k \in R.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมสองทิศเข้าด้วยกัน จึงได้

$$R = A.$$

□

5.7.4 ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 จากกฎ $x \mapsto x + 5$

ตัวอย่างก่อนหน้านี้แสดงให้เห็นว่า เมื่อเซต R ถูกนิยามด้วยสมาชิกฐานและกฎสร้างแบบง่าย เรามักพิสูจน์ว่า $R = A$ โดยแยกเป็นสองทิศ ตัวอย่างถัดไปมีรูปแบบคล้ายกัน แต่เปลี่ยนจากการคูณด้วย 2 เป็นการบวกด้วย 5

Theorem 5.7.2. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $x \in R$ แล้ว $x + 5 \in R$

แล้ว

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทิศ $R \subseteq A$ ใช้ structural induction ตามกฎการสร้างของ R ส่วนทิศ $A \subseteq R$ ใช้อุปนัยธรรมดาบน n

Proof. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction

ขั้นฐาน:

$$5 = 5 \cdot 1 \in A.$$

ขั้นอุปนัย: ให้ $x \in R$ และสมมติว่า $x \in A$ ดังนั้นมี $m \geq 1$ ที่ทำให้

$$x = 5m.$$

จากกฎสร้างของ R จะได้ว่า

$$x + 5 \in R.$$

และ

$$x + 5 = 5m + 5 = 5(m + 1) \in A.$$

ดังนั้น $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 5n \in R.$$

ขั้นฐาน:

$$5 \cdot 1 = 5 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$5k \in R.$$

จากกฎของ R จะได้ว่า

$$5k + 5 = 5(k + 1) \in R.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมสองทิศ จะได้ว่า

$$R = A.$$

□

5.7.5 ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 ที่ปิดภายใต้การบวก

ตัวอย่างนี้สำคัญเพราะช่วยขยายภาพของ structural induction ออกไปอีกชั้น: กฎสร้างไม่จำเป็นต้องรับสมาชิกเดิมเพียงตัวเดียวแล้วสร้างสมาชิกใหม่ แต่อาจรับสมาชิกเดิมหลายตัวพร้อมกัน แล้วประกอบเป็นสมาชิกใหม่ตัวหนึ่งก็ได้ นั่นคือในขั้นอุปนัย เราอาจต้องสมมติคุณสมบัติของวัตถุเดิมมากกว่าหนึ่งตัวพร้อมกัน แล้วพิสูจน์ว่าของใหม่ที่สร้างจากวัตถุเหล่านั้นยังคงมีคุณสมบัติเดิมอยู่

Theorem 5.7.3. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $a \in R$ และ $b \in R$ แล้ว $a + b \in R$

แล้ว

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทศ $R \subseteq A$ ใช้ structural induction ตามกฎของ R

สิ่งที่ต่างจากตัวอย่างก่อนคือ ในกฎสร้างข้อที่สอง เราไม่ได้เริ่มจากสมาชิกเดิมเพียงตัวเดียว แต่เริ่มจากสมาชิกสองตัว $a, b \in R$ ดังนั้นในขั้นอุปนัยเราจะสมมติว่า a และ b ต่างก็เป็นพหุคูณของ 5 แล้วตรวจว่า $a + b$ ก็ยังเป็นพหุคูณของ 5 ส่วนทศ $A \subseteq R$ คราวนี้ยิ่งง่ายขึ้น เพราะเมื่อมี $5 \in R$ แล้ว เราสามารถสร้าง

$$10 = 5 + 5, \quad 15 = 10 + 5, \quad 20 = 15 + 5, \dots$$

ได้ต่อไปเรื่อย ๆ จึงยังใช้อุปนัยธรรมดาบน n ได้เหมือนเดิม

Proof. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction ตามกฎของ R

ขั้นฐาน:

$$5 = 5 \cdot 1 \in A.$$

ขั้นอุปนัย: ให้ $a, b \in R$ และสมมติว่า

$$a \in A, \quad b \in A.$$

ดังนั้นมีจำนวนเต็มบวก m, n ที่ทำให้

$$a = 5m, \quad b = 5n.$$

จากกฎสร้างของ R จะได้ว่า

$$a + b \in R.$$

และ

$$a + b = 5m + 5n = 5(m + n) \in A.$$

ดังนั้นสมาชิกใหม่ที่ได้จากการบวกสมาชิกเดิมสองตัว ก็ยังอยู่ใน A จึงสรุปได้ว่า $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 5n \in R.$$

ขั้นฐาน:

$$5 \cdot 1 = 5 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$5k \in R.$$

เพราะ $5 \in R$ จากขั้นฐาน และกฎของ R บอกว่าถ้า $a, b \in R$ แล้ว $a + b \in R$ จึงได้ว่า

$$5k + 5 = 5(k + 1) \in R.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมกับ $R \subseteq A$ จะได้ว่า

$$R = A.$$

□

5.7.6 ตัวอย่าง: คุณสมบัติของสมาชิกทุกตัวในเซตที่สร้างแบบ recursion

structural induction ไม่ได้ใช้เฉพาะเวลาจะพิสูจน์ความเท่ากันของเซต แต่ยังใช้พิสูจน์ “คุณสมบัติร่วมของสมาชิกทุกตัวที่สร้างได้” ด้วย

Theorem 5.7.4. ให้เซต R นิยามโดย

1. $(0, 0) \in R$
2. ถ้า $(x, y) \in R$ แล้ว

$$(x, y+1), (x+1, y+1), (x+2, y+1) \in R$$

แล้วสำหรับทุก $(x, y) \in R$ จะมี

$$x \leq 2y.$$

Proof idea. สิ่งที่ต้องพิสูจน์ไม่ได้เกี่ยวกับ “สมาชิกตัวที่ n ” เลย แต่เกี่ยวกับคู่ลำดับทุกตัวที่สร้างได้จากกฎ ดังนั้น structural induction จึงเป็นภาษาที่ตรงกับโจทย์มากที่สุด

Proof. ให้

$$Q(x, y) : x \leq 2y.$$

เราจะพิสูจน์ว่า สมาชิกทุกตัวของ R มีคุณสมบัติ Q

ขั้นฐาน: สมาชิกฐานคือ $(0, 0)$ และ

$$0 \leq 2 \cdot 0.$$

ดังนั้น $Q(0, 0)$ เป็นจริง

ขั้นอุปนัย: ให้ $(u, v) \in R$ และสมมติว่า

$$u \leq 2v.$$

เราต้องพิสูจน์ว่าสมาชิกใหม่ทั้งสามแบบยังสอดคล้องกับสมการนี้

สำหรับ $(u, v+1)$,

$$u \leq 2v \leq 2v+2 = 2(v+1).$$

จึงได้ว่า

$$u \leq 2(v+1).$$

สำหรับ $(u + 1, v + 1)$, จาก $u \leq 2v$ จะได้

$$u + 1 \leq 2v + 1 \leq 2v + 2 = 2(v + 1).$$

สำหรับ $(u + 2, v + 1)$, จาก $u \leq 2v$ จะได้

$$u + 2 \leq 2v + 2 = 2(v + 1).$$

ดังนั้นสมาชิกใหม่ทุกแบบยังมีคุณสมบัติ $x \leq 2y$ สรุปด้วย structural induction ได้ว่า ทุก $(x, y) \in R$ มีคุณสมบัติ

$$x \leq 2y.$$

□

5.7.7 ตัวอย่างกับ bit string

Theorem 5.7.5. ให้เซตของ bit string R นิยามโดย

1. string ว่าง ϵ อยู่ใน R
2. ถ้า $x \in R$ แล้ว $0x1$ และ $1x0$ อยู่ใน R
3. ถ้า $x, y \in R$ แล้ว $xy \in R$

แล้วทุก string ใน R มีจำนวนเลข 0 เท่ากับจำนวนเลข 1

Proof idea. ข้อนี้ทำให้เห็น structural induction ในรูปที่เป็นธรรมชาติที่สุด: ฐานคือ string ว่าง กฎสร้างแต่ละข้อบอกชัดว่าจะสร้าง string ใหม่จาก string เดิมอย่างไร ดังนั้นเราก็พิสูจน์ตามกฎทั้งสามข้อทีละข้อ

Proof. ให้ $\#_0(w)$ และ $\#_1(w)$ แทนจำนวนเลข 0 และ 1 ใน string w และให้

$$Q(w) : \#_0(w) = \#_1(w).$$

ขั้นฐาน: สำหรับ string ว่าง ϵ ,

$$\#_0(\epsilon) = 0 = \#_1(\epsilon).$$

ดังนั้น $Q(\epsilon)$ เป็นจริง

ขั้นอุปนัยจากกฎข้อ (2): ให้ $x \in R$ และสมมติว่า

$$\#_0(x) = \#_1(x).$$

พิจารณา $0x1$ จะมี

$$\#_0(0x1) = \#_0(x) + 1, \quad \#_1(0x1) = \#_1(x) + 1.$$

จึงได้ว่า

$$\#_0(0x1) = \#_1(0x1).$$

ในทำนองเดียวกัน

$$\#_0(1x0) = \#_0(x) + 1, \quad \#_1(1x0) = \#_1(x) + 1,$$

จึงได้ว่า

$$\#_0(1x0) = \#_1(1x0).$$

ขั้นอุปนัยจากกฎข้อ (3): ให้ $x, y \in R$ และสมมติว่า

$$\#_0(x) = \#_1(x), \quad \#_0(y) = \#_1(y).$$

สำหรับ string ต่อกัน xy จะมี

$$\#_0(xy) = \#_0(x) + \#_0(y), \quad \#_1(xy) = \#_1(x) + \#_1(y).$$

ดังนั้น

$$\#_0(xy) = \#_1(xy).$$

สรุปด้วย structural induction ว่า ทุก string ใน R มีจำนวนเลข 0 เท่ากับจำนวนเลข 1

□

5.8 จาก structural induction สู่ strong induction

ตอนนี้เรามีภาพอยู่สองแบบแล้ว: ordinary induction ใช้กับข้อความที่เดินจาก n ไป $n + 1$, ส่วน structural induction ใช้กับวัตถุที่สร้างตามกฎ

แต่ยังมีอีกสถานการณ์หนึ่งที่พบบ่อยมาก: ข้อความที่ระดับ $k + 1$ อาจไม่ได้อาศัยเพียงระดับ k อย่างเดียว แต่อาศัยหลายชั้นก่อนหน้า หรืออาศัยการแตกปัญหาออกเป็นหลายส่วนย่อยที่ล้วนเล็กกว่าเดิม

ในสถานการณ์เช่นนี้ การสมมติเพียง $P(k)$ อาจไม่พอ เราจึงต้องใช้ **strong induction**

5.9 Strong Induction

5.9.1 หลักการ

Theorem 5.9.1 (Principle of Strong Induction). ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนนับตั้งแต่ค่าเริ่มต้น n_0 เป็นต้นไป ถ้า

1. ตรวจสอบพื้นฐานที่จำเป็นครบ

2. สำหรับทุก $k \geq n_0$, ถ้าสมมติว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แล้วพิสูจน์ได้ว่า $P(k + 1)$ เป็นจริง

จะสรุปได้ว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

ความต่างจาก ordinary induction จึงไม่ใช่การเปลี่ยนข้อสรุป แต่เป็นการเปลี่ยน “สิ่งที่ได้รับอนุญาตให้สมมติในขั้นอุปนัย”

เปรียบเทียบ ordinary induction กับ strong induction

ordinary induction:

$$\boxed{P(k)} \implies P(k + 1)$$

strong induction:

$$\boxed{P(n_0) \wedge P(n_0 + 1) \wedge \dots \wedge P(k)} \implies P(k + 1)$$

5.9.2 เมื่อไรควรใช้ strong induction

โดยคร่าวที่สุด strong induction เหมาะกับโจทย์ที่มีลักษณะอย่างน้อยหนึ่งข้อในต่อไปนี้

1. ขึ้นใหม่ขึ้นกับหลายขั้นก่อนหน้า เช่น recurrence ที่อ้างถึง $n - 1$ และ $n - 2$
2. ปัญหาถูกแบ่งเป็นหลายปัญหาย่อยที่มีขนาดเล็กกว่าเดิม
3. เราารู้เพียงว่าขั้นใหม่จะย้อนกลับไปได้ “บางขั้นก่อนหน้า” แต่ยังไม่แน่ว่าเป็นขั้นใด

5.9.3 ตัวอย่าง: recurrence ที่ย้อนสองขั้น

Theorem 5.9.2. ให้ลำดับ (a_n) นิยามโดย

$$a_0 = 1, \quad a_1 = 2, \quad a_n = 5a_{n-1} - 6a_{n-2} \quad (n \geq 2).$$

แล้วสำหรับทุก $n \in \mathbb{N}$,

$$a_n = 2^n.$$

Proof idea. เพราะนิยามของ a_n ใช้ทั้ง a_{n-1} และ a_{n-2} บทพิสูจน์จึงต้องมีสิทธิ์เรียกใช้ผลของหลายขั้นก่อนหน้า ซึ่ง strong induction เหมาะโดยตรง

Proof. ให้

$$P(n) : a_n = 2^n.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$a_0 = 1 = 2^0.$$

เมื่อ $n = 1$,

$$a_1 = 2 = 2^1.$$

ดังนั้น $P(0)$ และ $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

จากสมมติฐาน จะได้โดยเฉพาะว่า

$$a_k = 2^k, \quad a_{k-1} = 2^{k-1}.$$

ดังนั้น

$$a_{k+1} = 5a_k - 6a_{k-1} = 5 \cdot 2^k - 6 \cdot 2^{k-1} = 5 \cdot 2^k - 3 \cdot 2^k = 2 \cdot 2^k = 2^{k+1}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วย strong induction ว่า

$$a_n = 2^n$$

สำหรับทุก $n \in \mathbb{N}$

□

5.9.4 ตัวอย่าง: หักช็อกโกแลตบาร์

Theorem 5.9.3. ถ้ามีช็อกโกแลตบาร์ขนาด $n \times 1$ และต้องการหักให้เหลือเป็นชิ้น 1×1 ทั้งหมด จะต้องใช้การหักพอดี $n - 1$ ครั้งเสมอ

Proof idea. จำนวนครั้งที่ต้องหักไม่ได้ขึ้นกับกรณี $n - 1$ เพียงกรณีเดียว เพราะการหักครั้งแรกอาจแบ่งบาร์ออกเป็นท่อนย่อยขนาด u_1 และ u_2 ซึ่งต่างก็เล็กกว่า n และต้องเอาผลของทั้งสองท่อนมารวมกัน

Proof. ให้

$P(n)$: ช็อกโกแลตบาร์ขนาด $n \times 1$ ต้องหัก $n - 1$ ครั้งเพื่อแยกเป็นชิ้น 1×1 ทั้งหมด.

ขั้นฐาน: เมื่อ $n = 1$ บาร์เป็นชิ้น 1×1 อยู่แล้ว จึงไม่ต้องหักเลย และ

$$0 = 1 - 1.$$

ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(1), P(2), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาบาร์ขนาด $(k + 1) \times 1$ การหักครั้งแรกจะแบ่งมันออกเป็นสองท่อน ขนาด $u_1 \times 1$ และ $u_2 \times 1$ โดยที่

$$u_1, u_2 \geq 1, \quad u_1 + u_2 = k + 1.$$

เพราะ $u_1, u_2 \leq k$ จึงใช้สมมติฐานขั้นอุปนัยได้กับทั้งสองท่อน

ดังนั้นท่อนแรกต้องหักอีก $u_1 - 1$ ครั้ง ท่อนที่สองต้องหักอีก $u_2 - 1$ ครั้ง รวมกับการหักครั้งแรกอีก 1 ครั้ง จำนวนครั้งทั้งหมดจึงเป็น

$$1 + (u_1 - 1) + (u_2 - 1) = u_1 + u_2 - 1 = (k + 1) - 1 = k.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า บาร์ขนาด $n \times 1$ ต้องหัก $n - 1$ ครั้งเสมอ

□

5.9.5 ตัวอย่าง: แพ็กเหรียญขนาด 4 และ 5

Theorem 5.9.4. สำหรับทุกจำนวนเต็ม $n \geq 12$ จะมีจำนวนเต็มไม่ลบ a, b ที่ทำให้

$$n = 4a + 5b.$$

Proof idea. โจทย์นี้เป็นตัวอย่างที่ดีของกรณีที่มีการสร้าง $k + 1$ ไม่ได้ย้อนไปที่ k โดยตรงเสมอ แต่ย้อนกลับไปยัง $k - 3$ แทน strong induction จึงสะดวกกว่า induction แบบพื้นฐาน

Proof. ให้

$$P(n) : \exists a, b \in \mathbb{N} \cup \{0\} \text{ ที่ทำให้ } n = 4a + 5b.$$

ขั้นฐาน:

$$12 = 4 + 4 + 4, \quad 13 = 4 + 4 + 5, \quad 14 = 4 + 5 + 5, \quad 15 = 5 + 5 + 5.$$

ดังนั้น $P(12), P(13), P(14), P(15)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 15$ และสมมติว่า

$$P(12), P(13), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

เพราะ $k \geq 15$ จึงมี

$$k + 1 - 4 = k - 3 \geq 12.$$

จากสมมติฐานขั้นอุปนัย $P(k - 3)$ เป็นจริง จึงมี $a, b \geq 0$ ที่ทำให้

$$k - 3 = 4a + 5b.$$

บวก 4 ทั้งสองข้าง จะได้

$$k + 1 = 4(a + 1) + 5b.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า สำหรับทุก $n \geq 12$, สามารถเขียน n ให้อยู่ในรูป $4a + 5b$ ได้

□

5.9.6 ตัวอย่างเพิ่ม: การแยกตัวประกอบเป็นจำนวนเฉพาะ

ตัวอย่างก่อนหน้าของ strong induction ทำให้เห็นกรณีที่ขั้นใหม่อาศัยหลายขั้นก่อนหน้า หรืออาศัยค่าที่ไม่ใช่เพียง k โดยตรง ตัวอย่างคลาสสิกอีกข้อหนึ่งคือทฤษฎีบทพื้นฐานว่า จำนวนเต็มทุกตัวที่มากกว่า 1 สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้ จุดเด่นของตัวอย่างนี้คือ เมื่อ n ไม่เป็นจำนวนเฉพาะ เราจะเขียน $n = ab$ ได้โดยที่ทั้ง a และ b เล็กกว่า n แล้วใช้สมมติฐานขั้นอุปนัยกับทั้งสองตัวพร้อมกัน

Theorem 5.9.5. สำหรับทุกจำนวนเต็ม $n \geq 2$, สามารถเขียน n ให้อยู่ในรูปผลคูณของจำนวนเฉพาะได้

Proof idea. ถ้า n เป็นจำนวนเฉพาะ ก็จบในทันทีเพราะ n เองคือผลคูณที่มีเพียงหนึ่งตัวประกอบ แต่ถ้า n ไม่เป็นจำนวนเฉพาะ จะมี

$$n = ab$$

โดยที่ $2 \leq a < n$ และ $2 \leq b < n$ จากนั้นใช้ strong induction กับทั้ง a และ b

Proof. ให้

$P(n)$: จำนวนเต็ม $n \geq 2$ สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้.

ขั้นฐาน: เมื่อ $n = 2$, จำนวน 2 เป็นจำนวนเฉพาะ จึงเขียนเป็นผลคูณของจำนวนเฉพาะได้แล้ว ดังนั้น $P(2)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 2$ และสมมติว่า

$$P(2), P(3), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

พิจารณา $k + 1$

ถ้า $k + 1$ เป็นจำนวนเฉพาะ ก็จบในทันที เพราะ $k + 1$ เองเป็นผลคูณของจำนวนเฉพาะหนึ่งตัว

ถ้า $k + 1$ ไม่เป็นจำนวนเฉพาะ จะมีจำนวนเต็ม a, b ที่ทำให้

$$k + 1 = ab, \quad 2 \leq a < k + 1, \quad 2 \leq b < k + 1.$$

ดังนั้น $a \leq k$ และ $b \leq k$ จึงใช้สมมติฐานขั้นอุปนัยได้กับทั้ง a และ b

จากสมมติฐาน a เขียนเป็นผลคูณของจำนวนเฉพาะได้ และ b ก็เขียนเป็นผลคูณของจำนวนเฉพาะได้เช่นกัน ดังนั้นเมื่อคูณสองผลคูณนี้เข้าด้วยกัน จะได้ว่า

$$k + 1 = ab$$

เขียนเป็นผลคูณของจำนวนเฉพาะได้

ดังนั้น $P(k+1)$ เป็นจริง สรุปด้วย strong induction ว่า ทุกจำนวนเต็ม $n \geq 2$ สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้ $\square$

5.10 สรุปความสัมพันธ์ของสามแนวคิด

ถึงตรงนี้ เราไม่ได้มีเพียง “อุปนัยแบบหนึ่ง” แต่มีมุมมองอยู่สามแบบที่สัมพันธ์กัน

สามรูปแบบของ induction

1. ordinary induction

ใช้เมื่อข้อความเดินจากระดับ n ไป $n+1$ โดยตรง

2. structural induction

ใช้เมื่อวัตถุถูกนิยามแบบ recursion และเราต้องการพิสูจน์ตามวิธีที่วัตถุนั้นถูกสร้างขึ้น

3. strong induction

ใช้เมื่อขั้นใหม่ต้องอาศัยหลายขั้นก่อนหน้า หรืออาศัยปัญหาย่อยที่เล็กกว่าหลายกรณีพร้อมกัน

ถ้ามองย้อนกลับไปจะเห็นว่า สามแนวคิดนี้ไม่ได้แยกขาดจากกัน แต่เป็นการขยายวิธีคิดเดียวกันให้เหมาะกับบริบทที่ต่างกัน:

ordinary induction $\rightarrow$ structural induction $\rightarrow$ strong induction

ordinary induction เป็นจุดเริ่มต้น structural induction คือการปรับอุปนัยให้สอดคล้องกับ “โครงสร้างการสร้างวัตถุ” และ strong induction คือการขยายสิ่งที่เราสมมติได้ในขั้นอุปนัย เพื่อรับมือกับปัญหาที่ซับซ้อนเกินไปไม่ได้อาศัยเพียงขั้นเดียวก่อนหน้า

5.11 ความสมมูลกันของ ordinary and strong induction

เมื่อเรียน strong induction เป็นครั้งแรก ผู้เรียนจำนวนมากมักรู้สึกที่ strong induction “แรงกว่า” เพราะในขั้นอุปนัย เราได้รับอนุญาตให้สมมติว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แทนที่จะสมมติเพียง

$$P(k)$$

เหมือนใน ordinary induction

ความรู้สึกนี้มีเหตุผลในเชิงการใช้งาน: เวลาลงมือพิสูจน์จริง strong induction มักดูสะดวกกว่า โดยเฉพาะเมื่อข้อความที่ระดับ $k+1$ ต้องอาศัยหลายขั้นก่อนหน้า หรืออาศัยขั้นก่อนหน้าบางขั้นที่ยังไม่รู้แน่ชัดว่าเป็นขั้นใด

อย่างไรก็ตาม ในเชิงหลักการแล้ว ordinary induction และ strong induction สมมูลกัน กล่าวคือ ถ้ายอมรับหลักอุปนัยแบบหนึ่ง เราสามารถพิสูจน์อีกแบบหนึ่งได้ ดังนั้น strong induction ไม่ได้เป็น “หลักใหม่ที่ทรงพลังกว่าเดิม” แต่เป็นเพียงอีกรูปแบบหนึ่งของแนวคิดเดียวกัน

Theorem 5.11.1. บทจำนวนนับ หลักอุปนัยธรรมดา (ordinary induction) และหลักอุปนัยแบบเข้ม (strong induction) สมมูลกัน

เราจะพิสูจน์สมมูลนี้เป็นสองทิศ

5.11.1 Strong induction $\Rightarrow$ Ordinary induction

ทิศนี้ค่อนข้างตรงไปตรงมา เพราะถ้าเราใช้งาน strong induction ได้ เราก็ย่อมใช้งาน ordinary induction ได้ทันที เหตุผลคือ ใน ordinary induction ขั้นตอนต้องการเพียงว่า

$$P(k) \Rightarrow P(k+1).$$

แต่ใน strong induction เราสมมติได้แรงกว่านั้น คือ

$$P(n_0), P(n_0+1), \dots, P(k)$$

เป็นจริงทั้งหมด ดังนั้นโดยเฉพาะ เราก็รู้ว่า $P(k)$ เป็นจริงด้วย จึงใช้เหตุผลแบบ ordinary induction ต่อได้ทันที

บทพิสูจน์. สมมติว่าเรายอมรับ strong induction และต้องการพิสูจน์ ordinary induction

ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนเต็ม $n \geq n_0$ และสมมติว่า

1. $P(n_0)$ เป็นจริง
2. สำหรับทุก $k \geq n_0$, ถ้า $P(k)$ เป็นจริง แล้ว $P(k+1)$ เป็นจริง

เราต้องการสรุปว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

เพื่อใช้ strong induction ให้ $k \geq n_0$ และสมมติว่า

$$P(n_0), P(n_0+1), \dots, P(k)$$

เป็นจริงทั้งหมด จากสมมติฐานนี้ เราได้โดยเฉพาะว่า $P(k)$ เป็นจริง ดังนั้นจากเงื่อนไขข้อ (2) จะได้ว่า $P(k+1)$ เป็นจริง จึงตรวจสอบอุปนัยในรูปของ strong induction ได้ครบ และเมื่อมีขั้นฐาน $P(n_0)$ อยู่แล้ว สรุปได้ด้วย strong induction ว่า

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

ดังนั้น strong induction ให้ ordinary induction ได้

□□

5.11.2 Ordinary induction $\Rightarrow$ Strong induction

ทิศนี้เป็นทิศที่สำคัญกว่า เพราะมันแสดงให้เห็นว่า strong induction แม้อูเหมือนจะอนุญาตให้สมมติได้มากกว่า แต่จริง ๆ แล้ว สามารถสร้างขึ้นจาก ordinary induction ได้

แนวคิดหลักคือ: ถ้า strong induction ต้องการให้เรารู้ว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

ทั้งหมดเป็นจริงพร้อมกัน เราก็สร้างข้อความใหม่ขึ้นมาหนึ่งข้อความ ให้มันรวบสิ่งเหล่านี้ไว้เป็นก้อนเดียว

ความคิดหลักของบทพิสูจน์

แทนที่จะพิสูจน์ $P(n)$ ตรง ๆ ให้กำหนดข้อความใหม่

$$Q(n) : P(n_0) \wedge P(n_0 + 1) \wedge \dots \wedge P(n).$$

ถ้าพิสูจน์ $Q(n)$ ได้ด้วย ordinary induction ก็ย่อมได้ว่า $P(n)$ จริงสำหรับทุก $n \geq n_0$

นี่คือเทคนิคที่สำคัญมาก: ordinary induction พิสูจน์ทีละขั้นก็จริง แต่เราสามารถทำให้ “หนึ่งขั้น” บรรจุข้อมูลสะสมของทุกขั้นก่อนหน้านี้ไว้ด้วยกันได้

บทพิสูจน์. สมมติว่าเรายอมรับ ordinary induction และต้องการพิสูจน์ strong induction

ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนเต็ม $n \geq n_0$ และสมมติว่า

1. $P(n_0)$ เป็นจริง
2. สำหรับทุก $k \geq n_0$, ถ้า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แล้ว $P(k + 1)$ เป็นจริง

เราต้องการสรุปว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

กำหนดข้อความใหม่

$$Q(n) : P(n_0) \wedge P(n_0 + 1) \wedge \dots \wedge P(n).$$

เราจะพิสูจน์ว่า $Q(n)$ เป็นจริงสำหรับทุก $n \geq n_0$ ด้วย ordinary induction

ขั้นฐาน: เมื่อ $n = n_0$, ข้อความ $Q(n_0)$ ก็คือเพียง

$$P(n_0),$$

ซึ่งเป็นจริงตามสมมติฐานข้อ (1)

ขั้นอุปนัย: ให้ $k \geq n_0$ และสมมติว่า $Q(k)$ เป็นจริง นั่นคือ

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด

จากสมมติฐานข้อ (2) จึงได้ว่า

$$P(k+1)$$

เป็นจริงด้วย

ดังนั้นเราสามารถรวมข้อความเดิมทั้งหมดกับ $P(k+1)$ ได้เป็น

$$P(n_0) \wedge P(n_0+1) \wedge \cdots \wedge P(k) \wedge P(k+1),$$

ซึ่งก็คือ $Q(k+1)$

จึงได้ว่า $Q(k) \Rightarrow Q(k+1)$

สรุปด้วย ordinary induction ว่า

$$Q(n)$$

เป็นจริงสำหรับทุก $n \geq n_0$

แต่เมื่อ $Q(n)$ เป็นจริง ย่อมได้โดยเฉพาะว่า $P(n)$ เป็นจริง ดังนั้น

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

จึงสรุปได้ว่า ordinary induction ให้ strong induction ได้

□□

5.11.3 แล้วเหตุใดเรายังแยก ordinary กับ strong induction ออกจากกัน?

เมื่อรู้แล้วว่าสองหลักนี้สมมูลกัน อาจเกิดคำถามตามมาว่า แล้วทำไมหนังสือเรียนจึงยังแยก ordinary induction กับ strong induction ออกจากกันอยู่

คำตอบคือ แม้มันจะสมมูลกันในเชิงตรรกะ แต่มัน *ไม่เท่ากัน* ในเชิงการใช้งาน

ordinary induction เหมาะมากเมื่อโครงของปัญหาชัดเจนว่า ขั้นที่ $k+1$ สร้างจากขั้นที่ k โดยตรง เช่นสูตรผลบวก ลำดับเลขคณิต หรือลำดับเรขาคณิต

ส่วน strong induction เหมาะกว่าเมื่อขั้นใหม่อาศัยหลายขั้นก่อนหน้า หรือเมื่อปัญหาถูกแตกออกเป็นหลายปัญหาย่อยที่เล็กกว่า เช่นการแยกตัวประกอบเป็นจำนวนเฉพาะ หรือการให้เหตุผลเกี่ยวกับการแบ่งปัญหา

ดังนั้น strong induction จึงไม่ได้ “พิสูจน์ได้มากกว่า” แต่หลายครั้งมัน *เขียนพิสูจน์ได้ตรงธรรมชาติของปัญหามากกว่า*

ข้อสรุปสำคัญ

ordinary induction และ strong induction สมมูลกันในเชิงหลักการ แต่ strong induction มัก สะดวกกว่าในเชิงการใช้งาน เมื่อข้อความที่ระดับใหม่ต้องอาศัยหลายกรณีก่อนหน้า

5.11.4 ความสัมพันธ์กับ structural induction

ควรระวังว่า ข้อสรุปเรื่อง “ordinary และ strong induction สมมูลกัน” พูดยถึงเฉพาะหลักอุปนัยบนจำนวนนับเป็นหลัก

ส่วน structural induction เป็นการนำความคิดแบบอุปนัย ไปเขียนในภาษาที่สอดคล้องกับ กฎการสร้างของวัตถุ ดังนั้นบทบาทของมันจึงต่างออกไปเล็กน้อย: มันไม่ใช่เพียงการเลือกว่า “จะสมมติหนึ่งขั้นก่อนหน้า หรือทุกขั้นก่อนหน้า” แต่เป็นการเลือกพิสูจน์ตามโครงสร้างการสร้างวัตถุโดยตรง

อย่างไรก็ตาม ในเชิงลึก structural induction ก็ยังอยู่ในกรอบคร่าวเดียวกับแนวคิดอุปนัยทั้งหมด: เราพิสูจน์ฐาน แล้วพิสูจน์ว่ากฎการสร้างไม่ทำให้คุณสมบัติที่ต้องการสูญหายไป

5.12 บทสรุปของ chapter นี้

ถ้าจะสรุปบทนี้เป็นประโยคเดียว ใจความสำคัญคือ:

recursion บอกว่าสิ่งหนึ่งถูกสร้างขึ้นอย่างไร และ **induction** บอกว่าเราจะพิสูจน์คุณสมบัติของสิ่งนั้นอย่างไร

สำหรับลำดับ เราเห็นว่า recursion ใช้นิยามค่าถัดไป แล้ว ordinary induction ใช้พิสูจน์สูตรทั่วไป สำหรับเซตหรือ bit string ที่สร้างจากกฎ เราใช้ structural induction เดินตามกฎการสร้าง และเมื่อขั้นถัดไปขึ้นกับหลายขั้นก่อนหน้า เราใช้ strong induction แทน

จุดนี้เองที่ทำให้บทอุปนัยเป็นสะพานสำคัญไปยังบทถัดไป: เมื่อเราเริ่มศึกษาความถูกต้องของอัลกอริทึมแบบ recursion โดยเฉพาะอัลกอริทึมที่แตกปัญหาเป็นปัญหาย่อย เครื่องมือหลักที่กลับมาอีกครั้งก็คือ induction โดยเฉพาะ strong induction เพียงแต่คราวนี้สิ่งที่เราพิสูจน์ไม่ใช่สูตรของลำดับ แต่เป็นความถูกต้องของ algorithm

Exercises

Exercise 5.12.1. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 0$,

$$2 + 4 + 6 + \cdots + 2n = n(n + 1).$$

Exercise 5.12.2. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 1$,

$$2^n \geq n + 1.$$

Exercise 5.12.3. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 0$,

$$1 + 2 + 2^2 + \cdots + 2^n = 2^{n+1} - 1.$$

Exercise 5.12.4. ให้ลำดับ $b(n)$ นิยามโดย

$$b(0) = 4, \quad b(n) = b(n - 1) + 7 \quad (n \geq 1).$$

จงคาดเดาสูตรปิดของ $b(n)$ และพิสูจน์ด้วย induction

Exercise 5.12.5. ให้ลำดับ $h(n)$ นิยามโดย

$$h(0) = 5, \quad h(n) = 3h(n-1) \quad (n \geq 1).$$

จงคาดเดาสสูตรปิดของ $h(n)$ และพิสูจน์ด้วย induction

Exercise 5.12.6. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $x \in R$ แล้ว $x + 5 \in R$

จงพิสูจน์ว่า

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Exercise 5.12.7. ให้เซต R นิยามโดย

1. $\epsilon \in R$
2. ถ้า $x \in R$ แล้ว $0x1$ และ $1x0 \in R$
3. ถ้า $x, y \in R$ แล้ว $xy \in R$

จงพิสูจน์ว่า string ทุกตัวใน R มีความยาวเป็นจำนวนคู่

Exercise 5.12.8. ให้ลำดับ (b_n) นิยามโดย

$$b_0 = 3, \quad b_1 = 6, \quad b_n = 4b_{n-1} - 4b_{n-2} \quad (n \geq 2).$$

จงคาดเดาสสูตรของ b_n และพิสูจน์ด้วย strong induction

Exercise 5.12.9. จงพิสูจน์ด้วย strong induction ว่า สำหรับทุกจำนวนเต็ม $n \geq 2$, สามารถเขียน n ให้อยู่ในรูปผลคูณของจำนวนเฉพาะได้

บทที่ 6

Recursive Correctness Proof

บทนี้แยกออกมาเพื่อศึกษาคำถามที่ต่างจากบทอุปนัยโดยตรง ในบทอุปนัย เรามักพิสูจน์ว่า *ข้อความ* หนึ่งเป็นจริงสำหรับทุกจำนวนเต็มบวก หรือพิสูจน์ว่าคุณสมบัติบางอย่างเป็นจริงสำหรับวัตถุที่นิยามแบบเวียนเกิด แต่ในบทนี้ สิ่งที่เราต้องการพิสูจน์ไม่ใช่เพียงข้อความคณิตศาสตร์ แต่เป็น *ความถูกต้องของอัลกอริทึม* โดยเฉพาะอัลกอริทึมที่เรียกตัวเอง

เมื่อเราออกแบบ recursive algorithm เรามักเริ่มจากการลดปัญหาใหญ่ให้กลายเป็นปัญหาย่อยที่เล็กลง แล้วใช้คำตอบของปัญหาย่อยเหล่านั้นมาประกอบเป็นคำตอบของปัญหาเดิม แนวคิดนี้ทำให้เราได้ algorithm ที่ดูสมเหตุสมผล แต่ในทางคณิตศาสตร์ เรายังต้องตอบอีกคำถามหนึ่งให้ชัดว่า

ทำไมอัลกอริทึมนี้จึงให้คำตอบที่ถูกต้องจริงสำหรับทุก input ในโดเมนที่กำหนด?

การตอบคำถามนี้มักประกอบด้วย 3 ส่วน:

1. ระบุ **specification** ให้ชัดว่า output ที่ถูกต้องต้องมีคุณสมบัติอะไร
2. พิสูจน์ว่า algorithm **หยุดทำงาน** (termination)
3. พิสูจน์ว่าเมื่อมันหยุดแล้ว output ที่ได้ **ตรงตาม spec** (correctness)

ตัวอย่างเปิดบทนี้จะเป็น *Merge Sort* ซึ่งเป็น recursive algorithm แบบ divide-and-conquer เพราะตัวอย่างนี้แสดงโครงสร้างของ recursive correctness proof ได้ชัดมาก: เราต้องพิสูจน์ทั้งการเรียกตัวเองในส่วนย่อย และต้องพิสูจน์ด้วยว่าขั้น *combine* ไม่ทำลายความถูกต้องของคำตอบย่อย

6.1 Specification, Termination, and Correctness

ก่อนเริ่มพิสูจน์อัลกอริทึมใด ๆ เราควรแยกสามคำนี้ออกจากกันให้ชัดเจน

Definition 6.1.1 (Specification). spec ของอัลกอริทึมคือคำอธิบายว่า เมื่อป้อน input ที่อยู่ในโดเมนที่อนุญาตเข้าไปแล้ว output ที่ถือว่าถูกต้องต้องมีคุณสมบัติอะไร

Definition 6.1.2 (Termination). อัลกอริทึม **terminate** ถ้ามันหยุดทำงานหลังจากทำไปจำนวนขั้นจำกัด ไม่เกิดการเรียกตัวเองต่อไปเรื่อย ๆ อย่างไม่มีที่สิ้นสุด

Definition 6.1.3 (Correctness). อัลกอริทึม **correct** ถ้าสำหรับทุก input ในโดเมนที่กำหนด เมื่ออัลกอริทึมหยุดทำงานแล้ว output ที่ได้ตรงตาม spec

ข้อควรระวัง

การรู้ว่า base case ถูกต้อง และสมมติว่า recursive call ถูกต้อง ยัง *ไม่พอ* ที่จะสรุปว่าอัลกอริทึมทั้งตัวถูกต้อง เรายังต้องตรวจอีกอย่างน้อยสองจุด:

1. ปัญหาย่อยที่เรียกต่อมีขนาดเล็กลงจริงหรือไม่
2. ขั้นที่นำคำตอบย่อยมาประกอบกลับเป็นคำตอบใหญ่ ยังรักษาความถูกต้องไว้หรือไม่

6.2 Divide-and-Conquer: Merge Sort

6.2.1 แนวคิด Divide--Conquer--Combine

Merge Sort เป็นตัวอย่างมาตรฐานของอัลกอริทึมแบบ divide-and-conquer โดยโครงสร้างมี 3 ชั้น:

1. **Divide:** แบ่งลิสต์เดิมออกเป็นลิสต์ย่อย 2 ส่วนที่เล็กกว่าเดิม
2. **Conquer:** sort แต่ละส่วนด้วยการเรียกตัวเอง
3. **Combine:** รวมลิสต์ย่อยที่ sort แล้วให้เป็นลิสต์เดียวที่ sort แล้ว

ในที่นี้ขั้น combine ไม่ใช่การต่อปลายเฉย ๆ แต่เป็นการรวมสองลิสต์ที่ sorted แล้วเข้าด้วยกันอย่างระมัดระวัง ผ่าน procedure ที่ชื่อว่า Merge

ดังนั้นถ้าจะพิสูจน์ความถูกต้องของ Merge Sort อย่างเป็นระบบ เราควรแยกเป็นสองชั้น:

1. พิสูจน์ว่า Merge ทำงานถูกต้อง
2. ใช้ผลนั้นไปพิสูจน์ว่า MergeSort ทำงานถูกต้อง

6.2.2 นิยาม sorted และสเปกของ MergeSort

ให้ลิสต์

$$A = (a_1, a_2, \dots, a_n)$$

เราเรียกว่า A เป็นลิสต์ที่ **sorted** แบบไม่ลดลง ถ้า

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

เพื่อให้การพิสูจน์ correctness ครบถ้วน เราไม่ควรระบุเพียงว่า output ``เรียงแล้ว" เพราะ sorting algorithm อาจให้ลิสต์ที่เรียงแล้วแต่สมาชิกไม่เหมือนเดิมก็ได้ ดังนั้น spec ที่ถูกต้องต้องมีทั้งเรื่อง *ความเรียง* และ *การคงสมาชิกเดิมไว้*

ให้ $\#_x(A)$ แทนจำนวนครั้งที่ค่า x ปรากฏในลิสต์ A

Spec ของ MergeSort

สำหรับทุกลิสต์ A , ผลลัพธ์ $MergeSort(A)$ ต้องมีคุณสมบัติ 2 ข้อพร้อมกัน:

1. $MergeSort(A)$ เป็นลิสต์ที่ sorted
2. สำหรับทุกค่า x , มี

$$\#_x(MergeSort(A)) = \#_x(A)$$

กล่าวคือ ผลลัพธ์เป็นเพียงการ ``เรียงใหม่" ของสมาชิกเดิม ไม่ทำสมาชิกหาย ไม่เพิ่มสมาชิกใหม่ และไม่เปลี่ยนจำนวนซ้ำของสมาชิก

จากข้อที่สอง โดยเฉพาะจะได้ว่า

$$|MergeSort(A)| = |A|.$$

6.2.3 Pseudocode ของ MergeSort และ Merge

Pseudocode: MergeSort และ Merge

MergeSort(A):

if $|A| \leq 1$ then return A

แบ่ง A เป็น L, R โดย $|L| = \lfloor |A|/2 \rfloor$ และ $|R| = \lceil |A|/2 \rceil$

return Merge(MergeSort(L), MergeSort(R))

Merge(L, R): (สมมติว่า L และ R sorted แล้ว)

if $L = []$ then return R

if $R = []$ then return L

if $\text{first}(L) \leq \text{first}(R)$ then

return $[\text{first}(L)] \parallel \text{Merge}(\text{tail}(L), R)$

else

return $[\text{first}(R)] \parallel \text{Merge}(L, \text{tail}(R))$

สัญลักษณ์ $\parallel$ หมายถึงการนำสมาชิกตัวหนึ่งไปต่อหน้าลิสต์ และ $\text{tail}(L)$ หมายถึงลิสต์ที่เหลือหลังตัดสมาชิกตัวแรกของ L ออก

6.3 พิสูจน์ความถูกต้องของ Merge

ขั้น combine ของ Merge Sort คือ Merge ดังนั้นเราต้องเริ่มจากตรงนี้ก่อน

Theorem 6.3.1 (Correctness of Merge). ถ้า L และ R เป็นลิสต์ที่ sorted แล้ว และให้

$$M = \text{Merge}(L, R),$$

จะได้ว่า

1. M เป็นลิสต์ที่ sorted
2. สำหรับทุกค่า x ,

$$\#_x(M) = \#_x(L) + \#_x(R)$$

ดังนั้นโดยเฉพาะ

$$|M| = |L| + |R|.$$

Proof idea. พิสูจน์ด้วย strong induction บนขนาดปัญหา

$$n = |L| + |R|.$$

ใจความสำคัญคือ สมาชิกตัวแรกที่ Merge ปล่อยออกมาจะต้องเป็นตัวที่เล็กที่สุดในบรรดาหัวของทั้งสองลิสต์ จากนั้นปัญหาที่เหลือจะมีขนาดลดลง 1 จึงใช้สมมติฐานขั้นอุปนัยกับส่วนที่เหลือได้

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ sorted L, R ที่ทำให้ $|L| + |R| = n$, ลิสต์ $\text{Merge}(L, R)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับผลรวมสมาชิกของ L และ R แบบนับซ้ำ

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$

ถ้า $|L| + |R| = 0$ จะต้องเป็น

$$L = [], \quad R = [].$$

ดังนั้น

$$\text{Merge}(L, R) = [].$$

ลิสต์ว่างย่อม sorted และมีสมาชิกครบถ้วนตามต้องการ จึงได้ว่า $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาลิสต์ sorted L, R ที่มี

$$|L| + |R| = k + 1.$$

ถ้า $L = []$ หรือ $R = []$ ผลลัพธ์ของ Merge คืออีกลิสต์หนึ่งทันที ซึ่งยังคง sorted และสมาชิกก็ครบถ้วนชัดเจน ดังนั้นกรณีนี้ไม่มีปัญหา

จึงเหลือกรณีที่ทั้ง L และ R ไม่ว่าง เขียน

$$L = (\ell_1, \ell_2, \dots, \ell_s), \quad R = (r_1, r_2, \dots, r_t).$$

เราพิจารณาเป็นสองกรณี

กรณีที่ 1: $\ell_1 \leq r_1$

จาก algorithm จะได้

$$\text{Merge}(L, R) = [\ell_1] \parallel \text{Merge}(L', R),$$

โดยที่ $L' = (\ell_2, \dots, \ell_s)$

สังเกตว่า

$$|L'| + |R| = (|L| - 1) + |R| = k.$$

จึงใช้สมมติฐานขั้นอุปนัยได้ ดังนั้น $\text{Merge}(L', R)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ L' กับ R รวมกันแบบนับซ้ำ

เราต้องตรวจสอบว่าเมื่อเอา ℓ_1 มาต่อหน้าลิสต์นี้ ผลลัพธ์ยัง sorted อยู่หรือไม่

เพราะ L sorted จึงสมาชิกทุกตัวใน L' ไม่เล็กกว่า ℓ_1 และเพราะ R sorted กับ $\ell_1 \leq r_1$ สมาชิกทุกตัวใน R ก็ไม่เล็กกว่า ℓ_1 เช่นกัน ดังนั้นสมาชิกทุกตัวใน $\text{Merge}(L', R)$ ไม่เล็กกว่า ℓ_1 จึงได้ว่า

$$[\ell_1] \parallel \text{Merge}(L', R)$$

ยัง sorted

ด้านสมาชิกแบบนับซ้ำ ลิสต์ผลลัพธ์มีสมาชิกคือ ℓ_1 บวกกับสมาชิกทั้งหมดของ L' และ R ซึ่งเท่ากับสมาชิกทั้งหมดของ L และ R รวมกันพอดี

กรณีที่ 2: $r_1 < \ell_1$

พิสูจน์แบบสมมาตร:

$$\text{Merge}(L, R) = [r_1] \parallel \text{Merge}(L, R'),$$

โดยที่ $R' = (r_2, \dots, r_t)$ และจากสมมติฐานขั้นอุปนัย ลิสต์ $\text{Merge}(L, R')$ sorted และมีสมาชิกครบ เมื่อรวมกับข้อเท็จ

จริงว่าสมาชิกทุกตัวใน L และ R' ไม่เล็กกว่า r_1 จึงได้ว่าผลลัพธ์สุดท้าย sorted และมีสมาชิกครบเช่นกัน

ดังนั้น $P(k+1)$ เป็นจริง สรุปด้วย strong induction ได้ว่า Merge ถูกต้อง □

6.4 พิสูจน์ความถูกต้องของ MergeSort

เมื่อเรารู้แล้วว่าขั้น combine ถูกต้อง จึงสามารถพิสูจน์ correctness ของ MergeSort ทั้งตัวได้

Theorem 6.4.1 (Correctness of MergeSort). สำหรับทุกลิสต์ A , ลิสต์ $\text{MergeSort}(A)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ A แบบนับซ้ำ

Proof idea. พิสูจน์ด้วย strong induction บนความยาวลิสต์

$$n = |A|.$$

ถ้าลิสต์มีความยาว 0 หรือ 1 อัลกอริทึมคืนลิสต์เดิมทันที ส่วนกรณีที่ลิสต์ยาวกว่านั้น อัลกอริทึมจะแบ่งลิสต์เดิมออกเป็นสองลิสต์ที่สั้นกว่าเดิม เรียกตัวเองกับแต่ละลิสต์ย่อย แล้วใช้ความถูกต้องของ Merge เพื่อรวมผลลัพธ์กลับมา

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ A ที่มีความยาว n , ลิสต์ $MergeSort(A)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ A แบบนับซ้ำ

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$ และ $n = 1$

ถ้า $|A| = 0$ หรือ $|A| = 1$, algorithm คืนค่า A ทันที ซึ่งย่อม sorted อยู่แล้ว และแน่นอนว่ามีสมาชิกเท่ากับ A เอง ดังนั้น $P(0)$ และ $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาลิสต์ A ที่มี

$$|A| = k + 1.$$

เพราะ $k + 1 > 1$, algorithm จะไม่หยุดที่กรณีฐาน แต่จะแบ่ง A เป็นลิสต์ย่อย L และ R โดยมี

$$|L| = \lfloor (k+1)/2 \rfloor, \quad |R| = \lceil (k+1)/2 \rceil.$$

ดังนั้นทั้ง L และ R มีขนาดไม่เกิน k

จากสมมติฐานขั้นอุปนัย

$$MergeSort(L)$$

เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ L แบบนับซ้ำ และ

$$MergeSort(R)$$

เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ R แบบนับซ้ำ

ตอนนี้ algorithm คืนค่า

$$Merge(MergeSort(L), MergeSort(R)).$$

แต่จากทฤษฎีบทความถูกต้องของ Merge เมื่อเอาลิสต์ sorted สองลิสต์มา merge กัน ผลลัพธ์จะยังเป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับสมาชิกของลิสต์ทั้งสองรวมกันแบบนับซ้ำ

ดังนั้นผลลัพธ์สุดท้ายของ $MergeSort(A)$ sorted และมีสมาชิกเท่ากับ L กับ R รวมกันแบบนับซ้ำ ซึ่งก็คือสมาชิกของ A เดิมทั้งหมด

จึงได้ว่า $P(k + 1)$ เป็นจริง สรุปด้วย strong induction ว่า MergeSort ถูกต้อง $\square$

6.5 Recursive Binary Search

หลังจากเห็นตัวอย่างของ recursive correctness proof ใน MergeSort แล้ว เราจะดูตัวอย่างอีกแบบหนึ่งที่โครงสร้างของ recursive call ไม่ได้เกิดจากการแบ่ง input ออกเป็นสองส่วนแล้วรวมคำตอบกลับ แต่เกิดจากการ *เลือกเพียงครั้งเดียว* ของปัญหาเดิมเพื่อเรียกต่อ นั่นคือ Binary Search

6.5.1 นิยามและสเปก

สมมติว่า

$$A[1], A[2], \dots, A[n]$$

เป็นลิสต์ที่ sorted แบบไม่ลดลง กล่าวคือ

$$A[1] \leq A[2] \leq \dots \leq A[n].$$

เราจะเขียน $A[\ell..r]$ เพื่อแทนช่วงดัชนีตั้งแต่ ℓ ถึง r และยอมให้กรณี $\ell > r$ เป็นช่วงว่าง นิยามฟังก์ชัน

$$RBSearch(A, x, \ell, r)$$

เพื่อค้นหา x ภายในช่วง $A[\ell..r]$

Spec ของ Binary Search

กำหนด input เป็นลิสต์ sorted A , ค่าที่ต้องการหา x , และดัชนี ℓ, r ที่ทำให้

$$1 \leq \ell \leq r + 1 \leq |A| + 1.$$

ให้

$$out = RBSearch(A, x, \ell, r).$$

ผลลัพธ์ที่ถูกต้องต้องเป็นอย่างใดอย่างหนึ่ง:

1. $out = 0$ และไม่มีดัชนี i ในช่วง $\ell \leq i \leq r$ ที่ทำให้ $A[i] = x$
2. $out \neq 0$ และ $A[out] = x$ โดยมี $\ell \leq out \leq r$

สังเกตว่า spec นี้ครอบคลุมทั้งกรณี ``เจอ" และ ``ไม่เจอ" ดังนั้นเวลาพิสูจน์ correctness induction hypothesis ก็ต้องถูกเขียนให้ครอบคลุมทั้งสองกรณีด้วยเช่นกัน

6.5.2 Pseudocode

Pseudocode: Recursive Binary Search

```

RBSearch( $A, x, \ell, r$ ):
  if  $\ell > r$  then return 0
   $m \leftarrow \lfloor (\ell + r)/2 \rfloor$ 
  if  $A[m] = x$  then return  $m$ 
  if  $x < A[m]$  then return RBSearch( $A, x, \ell, m - 1$ )
  else return RBSearch( $A, x, m + 1, r$ )

```

6.5.3 Termination

Claim 6.5.1 (Termination of **RBSearch**). สำหรับทุก input ที่อยู่ในโดเมนข้างต้น ฟังก์ชัน **RBSearch**(A, x, ℓ, r) จะหยุดทำงาน

Proof idea. วัดขนาดปัญหาด้วยความยาวช่วงค้นหา

$$n = r - \ell + 1.$$

ถ้าช่วงว่างแล้วก็หยุดทันที ถ้ายังไม่ว่าง recursive call ครั้งถัดไปจะไปยังช่วงที่สั้นกว่าช่วงเดิมจริง

Proof. พิจารณา

$$n = r - \ell + 1.$$

ถ้า $\ell > r$, algorithm เข้ากรณีฐานและคืนค่า 0 ทันที จึงหยุดทำงาน

ถ้า $\ell \leq r$, algorithm คำนวณ

$$m = \left\lfloor \frac{\ell + r}{2} \right\rfloor.$$

ถ้า $A[m] = x$ ก็หยุดทันทีเช่นกัน

ถ้าไม่เจอ จะเกิด recursive call เพียงกรณีใดกรณีหนึ่ง:

$$\text{RBSearch}(A, x, \ell, m - 1) \quad \text{หรือ} \quad \text{RBSearch}(A, x, m + 1, r).$$

ในกรณีแรก ความยาวช่วงใหม่คือ

$$(m - 1) - \ell + 1 = m - \ell < n.$$

ในกรณีที่สอง ความยาวช่วงใหม่คือ

$$r - (m + 1) + 1 = r - m < n.$$

ดังนั้นทุกครั้งที่เกิด recursive call ขนาดปัญหาจะลดลงจริง และเพราะขนาดปัญหาเป็นจำนวนเต็มไม่ลบ จึงไม่สามารถลดลงต่อไปเรื่อย ๆ ได้ไม่สิ้นสุด ท้ายที่สุด algorithm ต้องเข้าสู่กรณีฐานและหยุดทำงาน $\square$

6.5.4 Correctness

Theorem 6.5.2 (Correctness of RBSearch). ให้ A เป็นลิสต์ sorted และให้

$$out = RBSearch(A, x, \ell, r)$$

โดยที่

$$1 \leq \ell \leq r + 1 \leq |A| + 1.$$

แล้ว out ตรงตาม spec ของ Binary Search

Proof idea. พิสูจน์ด้วย strong induction บนความยาวช่วง

$$n = r - \ell + 1.$$

เหตุผลสำคัญคือ เราต้องการสมมติว่า recursive call ทุกตัวที่เกิดกับปัญหาที่สั้นกว่าเดิมทำงานถูกต้อง แล้วใช้ข้อเท็จจริงเรื่อง sortedness เพื่อแสดงว่าครั้งที่เราย้อนทิ้งไปนั้นไม่มีคำตอบจริง ๆ

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ sorted A , ทุกค่า x , และทุกดัชนี ℓ, r ที่ทำให้

$$1 \leq \ell \leq r + 1 \leq |A| + 1 \quad \text{และ} \quad r - \ell + 1 = n,$$

ถ้าเรียก

$$out = RBSearch(A, x, \ell, r),$$

แล้ว out ตรงตาม spec ของ Binary Search

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$

เมื่อ $r - \ell + 1 = 0$ จะได้ว่า $\ell = r + 1$ ดังนั้นช่วง $[\ell..r]$ เป็นช่วงว่าง algorithm คืนค่า 0 ทันที และแน่นอนว่าไม่มีดัชนี i โดยอยู่ในช่วงนี้ที่ทำให้ $A[i] = x$ ดังนั้น output ตรงตาม spec จึงได้ว่า $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

พิจารณา input ใด ๆ ที่มี

$$r - \ell + 1 = k + 1.$$

ดังนั้น $\ell \leq r$ และช่วงไม่ว่าง algorithm คำนวณ

$$m = \left\lfloor \frac{\ell + r}{2} \right\rfloor.$$

พิจารณาเป็นสามกรณี

กรณีที่ 1: $A[m] = x$

algorithm คืนค่า m ทันที และชัดเจนว่า

$$\ell \leq m \leq r, \quad A[m] = x.$$

ดังนั้น output ตรงตามข้อ (2) ของ spec

กรณีที่ 2: $x < A[m]$

algorithm เรียก

$$RBSearch(A, x, \ell, m - 1).$$

ช่วงใหม่มีความยาว

$$m - \ell < k + 1,$$

ดังนั้นใช้สมมติฐานขั้นอุปนัยได้

แต่เพียงแค่ว่า recursive call ถูกต้องในช่วง $[\ell..m - 1]$ ยังไม่พอ เราต้องอธิบายเพิ่มว่า ในช่วง $[m..r]$ ไม่มีทางมีสมาชิกเท่ากับ x

เพราะ A sorted และ $x < A[m]$, สำหรับทุก j ที่ $m \leq j \leq r$ จะมี

$$A[j] \geq A[m] > x.$$

ดังนั้นในช่วง $[m..r]$ ไม่มีสมาชิกเท่ากับ x

เพราะฉะนั้น:

- ถ้า recursive call คืนค่า 0, ก็แปลว่าในช่วง $[\ell..m - 1]$ ไม่มีสมาชิกเท่ากับ x เมื่อรวมกับการที่ช่วง $[m..r]$ ก็ไม่มี x , จะได้ว่าในช่วงเดิม $[\ell..r]$ ไม่มี x ดังนั้น output ตรงตามข้อ (1) ของ spec
- ถ้า recursive call คืนดัชนี $out \neq 0$, จากสมมติฐานขั้นอุปนัยจะมี

$$\ell \leq out \leq m - 1 \leq r$$

และ

$$A[out] = x.$$

ดังนั้น output ตรงตามข้อ (2) ของ spec

กรณีที่ 3: $x > A[m]$

algorithm เรียก

$$RBSearch(A, x, m + 1, r).$$

ช่วงใหม่มีความยาว

$$r - m < k + 1,$$

จึงใช้สมมติฐานขั้นอุปนัยได้อีกครั้ง

และเพราะ A sorted กับ $x > A[m]$, สำหรับทุก j ที่ $\ell \leq j \leq m$ จะมี

$$A[j] \leq A[m] < x.$$

ดังนั้นช่วง $[\ell..m]$ ไม่มีสมาชิกเท่ากับ x

จากนี้ให้เหตุผลแบบเดียวกับกรณีก่อนหน้า: ถ้า recursive call ไม่พบ x ในช่วง $[m + 1..r]$, ก็ไม่มี x ในช่วงเดิมทั้งหมด และถ้า recursive call ค้นดัชนีหนึ่งตำแหน่ง ดัชนีนั้นก็อยู่ในช่วงเดิมและทำให้ $A[out] = x$

ดังนั้นในทุกกรณี $P(k + 1)$ เป็นจริง สรุปด้วย strong induction ว่า $RBSearch$ ถูกต้อง □

6.6 ตัวอย่าง recursive algorithm เพิ่มเติม

ต่อไปเราจะดู recursive algorithm ที่เรียบง่ายกว่า เพื่อให้เห็นแพตเทิร์นของ correctness proof ชัดขึ้น แม้โครงของมันจะไม่ซับซ้อนเท่า MergeSort หรือ Binary Search

6.6.1 Power

นิยาม

นิยามฟังก์ชัน Power แบบ recursion โดย

$$Power(a, 0) = 1, \quad Power(a, n) = a \cdot Power(a, n - 1) \quad (n \geq 1).$$

Pseudocode: Power

Power(a, n):

 if $n = 0$ then return 1

 else return $a \cdot \text{Power}(a, n - 1)$

Correctness

Claim 6.6.1 (Correctness of Power). สำหรับทุก $a \in \mathbb{R}$ และทุก $n \in \mathbb{N}$,

$$\text{Power}(a, n) = a^n.$$

Proof. ให้

$$P(n) : \text{Power}(a, n) = a^n$$

สำหรับค่าคงที่ $a \in \mathbb{R}$

ขั้นฐาน: เมื่อ $n = 0$, algorithm คืนค่า 1, จึงได้

$$\text{Power}(a, 0) = 1 = a^0.$$

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$\text{Power}(a, k) = a^k.$$

เราต้องพิสูจน์ว่า

$$\text{Power}(a, k + 1) = a^{k+1}.$$

จาก algorithm,

$$\text{Power}(a, k + 1) = a \cdot \text{Power}(a, k).$$

แทนด้วยสมมติฐานขั้นอุปนัย จะได้

$$\text{Power}(a, k + 1) = a \cdot a^k = a^{k+1}.$$

จึงพิสูจน์ได้ครบ

□

6.6.2 SumList

นิยาม

ให้ลิสต์

$$A = (a_1, a_2, \dots, a_n)$$

และให้ $\text{tail}(A)$ หมายถึงลิสต์ที่เหลือหลังตัดสมาชิกตัวแรกออก

นิยามฟังก์ชัน SumList แบบ recursion โดย

$$\text{SumList}([]) = 0, \quad \text{SumList}([a_1, \dots, a_n]) = a_1 + \text{SumList}([a_2, \dots, a_n]).$$

Pseudocode: SumList

```

SumList( $A$ ):
    if  $|A| = 0$  then return 0
    else return  $A[1] + \text{SumList}(\text{tail}(A))$ 

```

Correctness

Claim 6.6.2 (Correctness of SumList). สำหรับทุกลิสต์จำนวนจริง

$$A = (a_1, \dots, a_n),$$

มี

$$\text{SumList}(A) = a_1 + a_2 + \dots + a_n,$$

และถ้า A เป็นลิสต์ว่าง ให้ผลรวมเป็น 0

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ A ที่มีความยาว n , ค่าของ $\text{SumList}(A)$ เท่ากับผลรวมของสมาชิกทุกตัวใน A

ขั้นฐาน: $n = 0$

ลิสต์ต้องเป็นลิสต์ว่าง algorithm คืนค่า 0 ทันที ซึ่งตรงกับนิยามของผลรวมของลิสต์ว่าง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า $P(k)$ เป็นจริง พิจารณาลิสต์ A ที่มีความยาว $k + 1$ เขียนได้เป็น

$$A = [a_1, a_2, \dots, a_{k+1}].$$

ดังนั้น

$$\text{tail}(A) = [a_2, \dots, a_{k+1}]$$

มีความยาว k

จาก algorithm,

$$\text{SumList}(A) = a_1 + \text{SumList}(\text{tail}(A)).$$

จากสมมติฐานขั้นอุปนัย,

$$\text{SumList}(\text{tail}(A)) = a_2 + \dots + a_{k+1}.$$

จึงได้

$$\text{SumList}(A) = a_1 + (a_2 + \dots + a_{k+1}) = a_1 + a_2 + \dots + a_{k+1}.$$

ดังนั้น $P(k + 1)$ เป็นจริง

□

6.7 Pattern Summary

เมื่อมองทั้ง MergeSort, Binary Search, Power, และ SumList ร่วมกัน จะเห็นแพตเทิร์นกลางของ recursive correctness proof ซัดขึ้นมาก

Pattern ของ Recursive Correctness Proof

เมื่อต้องการพิสูจน์ความถูกต้องของ recursive algorithm ให้ถามตัวเองเป็นลำดับดังนี้

1. **Input** ที่อนุญาตคืออะไร
ต้องกำหนดโดเมนให้ชัดก่อน
2. **Output** ที่ถูกต้องต้องเป็นแบบไหน
นี่คือ spec
3. จะวัดขนาดปัญหาด้วยอะไร
เช่น ความยาวลิสต์, ความยาวช่วง, หรือจำนวนสมาชิก
4. **recursive call** ไปสู่ปัญหาที่เล็กลงจริงหรือไม่
นี่คือหัวใจของ termination
5. ถ้าสมมติว่า recursive call ถูกต้องแล้ว
อัลกอริทึมใช้ผลย่อยเหล่านั้นอย่างไรเพื่อสร้างคำตอบของปัญหาเดิม
6. ถ้ามีขั้น **combine**
ต้องพิสูจน์ให้ชัดว่าขั้น combine นั้นไม่ทำลายความถูกต้อง

กล่าวอีกแบบหนึ่ง ในบทอุปนัย เราพิสูจน์ว่า ``ข้อความ" เป็นจริงสำหรับทุกขนาด แต่ในบทนี้ เราใช้แนวคิดเดียวกันเพื่อพิสูจน์ว่า ``อัลกอริทึม" ทำงานถูกต้องสำหรับทุกขนาดของ input

แบบฝึกหัดเพิ่มเติม

Exercise 6.7.1. จงเขียนพิสูจน์ termination ของ *RBSearch* โดยไม่ใช้อุปนัย แต่ใช้เหตุผลว่าความยาวช่วงค้นหาลดลงทุกครั้งที่จำเป็นต้องเข้าสู่กรณีฐาน

Exercise 6.7.2. ในพิสูจน์ correctness ของ *RBSearch*, อธิบายให้ละเอียดว่า sortedness ถูกใช้ตรงไหน และถ้าเอาเงื่อนไข sorted ออกไป ส่วนใดของบทพิสูจน์จะพังทันที

Exercise 6.7.3. จงพิสูจน์ correctness ของฟังก์ชัน

$$\text{Len}([]) = 0, \quad \text{Len}([a_1, \dots, a_n]) = 1 + \text{Len}([a_2, \dots, a_n]).$$

กล่าวคือพิสูจน์ว่า $\text{Len}(A) = |A|$ สำหรับทุกลิสต์ A

Exercise 6.7.4. พิจารณาข้อความต่อไปนี้:

“ถ้า recursive algorithm มี base case ถูกต้อง และ recursive call ถูกต้อง ก็เพียงพอแล้วที่จะสรุปว่าอัลกอริทึมทั้งหมดถูกต้อง”

จงอธิบายว่าข้อความนี้จริงหรือเท็จ พร้อมยกเหตุผลจากตัวอย่างในบทนี้ว่าเป็นระบบ

บทที่ 7

Recurrent Relation

7.1 Introduction to Recurrence Relations

ในหลายปัญหาของคณิตศาสตร์ไม่ต่อเนื่องและอัลกอริทึม เรามักไม่ได้คำตอบออกมาเป็นสูตรปิดตั้งแต่ต้น แต่ได้ความสัมพันธ์ที่บอกว่า “คำตอบของปัญหขนาดปัจจุบัน” เขียนในรูปของ “คำตอบของปัญหขนาดเล็กลง” ได้อย่างไร ความสัมพันธ์ลักษณะนี้เรียกว่า *recurrence relation*

Definition 7.1.1 (recurrence relation). สมการที่นิยามพจน์ $T(n)$ โดยอ้างอิงพจน์ก่อนหน้า เช่น

$$T(n) = T(n-1) + 2n - 1 \quad \text{หรือ} \quad T(n) = 2T(n/2) + n - 1.$$

แนวคิดสำคัญคือ recurrence relation ยังไม่ใช่คำตอบสุดท้าย แต่เป็นการบอกโครงสร้างการเกิดของคำตอบ ดังนั้นเมื่อเห็น recurrence สิ่งที่เราสนใจต่อมาก็คือคำถาม 4 ข้อ: ข้อความนี้แกะออกอย่างไร, มีรูปแบบซ่อนอยู่หรือไม่, เรา *closed form* ได้ไหม, และเมื่อเราได้แล้วจะพิสูจน์อย่างไรให้ถูกต้อง

เป้าหมายของส่วนนี้

- แกะ recurrence ด้วยการแทนค่าถอยกลับ
- มองหา pattern
- เรา closed form
- เตรียมไปพิสูจน์ closed form ด้วย induction

7.1.1 ตัวอย่างที่ 1: recurrence แบบง่าย

เราจะเริ่มจากตัวอย่างที่ง่ายที่สุดก่อน เพื่อให้เห็นกลไกของการ “คลี่ recurrence” โดยไม่ต้องมีรายละเอียดของอัลกอริทึมเข้ามาปะปน

Example 7.1.2. กำหนดให้

$$T(0) = 0, \quad T(n) = T(n-1) + (2n-1) \quad (n \geq 1).$$

จงหา closed form ของ $T(n)$

Proof idea. แทนค่าถอยกลับไปเรื่อย ๆ จนสุดที่ฐาน $T(0)$ แล้วสังเกตว่าพจน์ที่ถูกบวกเข้ามาคือ

$$1, 3, 5, \dots, 2n-1$$

ซึ่งเป็นผลบวกของจำนวนคี่ n พจน์แรก

Proof. เริ่มจาก recurrence:

$$T(n) = T(n-1) + (2n-1).$$

แทน $T(n-1)$ ด้วย recurrence เดิมอีกครั้ง จะได้

$$T(n) = (T(n-2) + (2(n-1)-1)) + (2n-1).$$

ดังนั้น

$$T(n) = T(n-2) + (2n-3) + (2n-1).$$

แทนต่ออีกหนึ่งครั้ง:

$$T(n) = T(n-3) + (2n-5) + (2n-3) + (2n-1).$$

ทำเช่นนี้ต่อไปเรื่อย ๆ จะเห็นรูปแบบว่า

$$T(n) = T(0) + 1 + 3 + 5 + \dots + (2n-1).$$

เพราะ $T(0) = 0$ จึงได้

$$T(n) = 1 + 3 + 5 + \dots + (2n-1).$$

แต่ผลบวกของจำนวนคี่ n พจน์แรกเท่ากับ n^2 ดังนั้น

$$T(n) = n^2.$$

□

กล่าวอีกแบบหนึ่ง recurrence ข้อนี้อธิบายว่าค่า $T(n)$ เกิดจากการนำ $T(n-1)$ ไปบวก "ขั้นใหม่" ที่มีขนาด $2n-1$ และเมื่อรวมขั้นทั้งหมดตั้งแต่ 1 ถึง n จึงได้กำลังสองพอดี

Exercise 7.1.3. เมื่อเดา closed form ได้แล้ว จงพิสูจน์ด้วย induction ว่าสูตร

$$T(n) = n^2$$

ถูกต้องจริงสำหรับทุก $n \in \mathbb{N}$

7.1.2 ตัวอย่างที่ 2: Merge Sort recurrence (กรณี n เป็นกำลังของ 2)

ตัวอย่างถัดไปมาจากอัลกอริทึม และสำคัญมากเพราะเป็น recurrence ที่มีรูป ``แก้ปัญหาห่วยสองอัน แล้วบวกงานที่ต้องทำตอนรวมคำตอบ``

ให้ $T(n)$ แทนจำนวน comparisons มากที่สุดของ Merge Sort โดยสมมติว่า n เป็นกำลังของ 2 จะได้ recurrence

$$T(1) = 0, \quad T(n) = 2T(n/2) + (n - 1) \quad (n \geq 2).$$

ภาพรวมของ Merge Sort

ทบทวนแนวคิดของ Merge Sort:

1. ถ้าลิสต์มีขนาด 1 ก็ sorted อยู่แล้ว
2. ถ้าลิสต์มีขนาดมากกว่า 1 ให้แบ่งออกเป็นสองครึ่ง
3. เรียงแต่ละครึ่งแบบ recursion
4. นำสองลิสต์ที่ sorted แล้วมา **merge** กลับเข้าด้วยกัน

สิ่งสำคัญคือ recurrence นี้ไม่ได้เกิดจากการเดาสุ่ม แต่เกิดจากการอ่านโครงสร้างของอัลกอริทึมตรง ๆ: มี recursive call สองครั้งบนข้อมูลขนาด $n/2$ และมีค่าใช้จ่ายเพิ่มตอน merge อีกหนึ่งส่วน

Example 7.1.4 (Merge step). อธิบายว่าทำไมการ merge ลิสต์ sorted สองลิสต์ที่มีสมาชิกทั้งหมดรวมกัน n ตัว ใช้ comparisons มากที่สุดไม่เกิน $n - 1$ ครั้ง และทำไม recurrence relation จึงเป็น

$$T(n) = 2T(n/2) + (n - 1), \quad T(1) = 0.$$

Proof idea. ในขั้น merge แต่ละครั้ง เราดูหัวของสองลิสต์แล้วเลือกตัวที่น้อยกว่าออกมาใส่ผลลัพธ์ ทุก comparison ทำให้มีสมาชิกถูกปล่อยออกจากหัวลิสต์อย่างน้อยหนึ่งตัว และทันทีที่ลิสต์ใดลิสต์หนึ่งหมด สมาชิกที่เหลือของอีกลิสต์ไม่ต้องเปรียบเทียบอีกแล้ว จึงใช้ comparisons มากที่สุดไม่เกิน $n - 1$ ครั้ง

Proof. ให้ลิสต์ sorted สองลิสต์มีความยาวรวมกันเป็น n ทุกครั้งที่เปรียบเทียบหัวของสองลิสต์ เราจะย้ายสมาชิกออกไปสู่ลิสต์ผลลัพธ์หนึ่งตัว ดังนั้นจำนวนสมาชิกที่ยัง ``ค้าง`` อยู่จะลดลงทีละหนึ่ง

เมื่อใดก็ตามที่ลิสต์หนึ่งหมด สมาชิกทั้งหมดของอีกลิสต์สามารถต่อท้ายเข้าผลลัพธ์ได้ทันที โดยไม่ต้องเปรียบเทียบอีก ดังนั้นในกรณีที่ดีที่สุด เราจะต้องเปรียบเทียบจนเหลือสมาชิกค้างอยู่เพียงหนึ่งตัวสุดท้าย ทำให้จำนวน comparisons มากที่สุดไม่เกิน $n - 1$

ทีนี้สำหรับ Merge Sort บนลิสต์ขนาด n :

- recursive call ครั้งที่หนึ่ง ใช้มากที่สุด $T(n/2)$
- recursive call ครั้งที่สอง ใช้มากที่สุด $T(n/2)$

- ชั้น merge ใช้มากที่สุด $n - 1$

จึงได้

$$T(n) = 2T(n/2) + (n - 1)$$

และฐานคือ

$$T(1) = 0$$

เพราะลิสต์ขนาด 1 ไม่ต้องเปรียบเทียบเลย

□

7.1.3 Plug-and-chug สำหรับ Merge Sort

ตอนนี้เป้าหมายของเราไม่ใช่เพียงอ่าน recurrence ให้เป็น แต่ต้องคลี่มันออกเพื่อหารูปแบบของคำตอบ

Example 7.1.5. จงแทนค่าถอยกลับเพื่อหา pattern ของ $T(n)$:

$$T(n) = 2T(n/2) + (n - 1), \quad T(1) = 0$$

คำแนะนำ: ใช้ $n = 2^k$

Proof idea. ถ้าเขียน $n = 2^k$ จำนวนครั้งที่เราหารสองได้ก่อนถึง 1 คือ k ครั้งพอดี จึงทำให้รูปแบบของการแทนค่าชัดขึ้นมาก

Proof. ให้ $n = 2^k$ เริ่มจาก

$$T(n) = 2T(n/2) + (n - 1).$$

แทนค่าถอยกลับอีกครั้ง:

$$T(n) = 2 \left(2T(n/4) + \left(\frac{n}{2} - 1 \right) \right) + (n - 1) = 2^2 T(n/4) + n - 2 + n - 1.$$

จึงได้

$$T(n) = 2^2 T(n/4) + 2n - 3.$$

แทนต่อไปอีกหนึ่งขั้น:

$$T(n) = 2^3 T(n/8) + 3n - 7.$$

จะเห็น pattern ว่าหลังแทนไป i ชั้น ได้

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in - (2^i - 1).$$

เมื่อ $n = 2^k$ และเลือก $i = k$ จะได้ว่า

$$\frac{n}{2^k} = 1, \quad 2^k = n.$$

ดังนั้น

$$T(n) = 2^k T(1) + kn - (2^k - 1).$$

ใช้ฐาน $T(1) = 0$ และ $2^k = n$ จึงได้

$$T(n) = kn - n + 1.$$

เมื่อ $k = \log_2 n$ จะได้ closed form เป็น

$$T(n) = n \log_2 n - n + 1.$$

ดังนั้น Merge Sort มีการเติบโตระดับ

$$T(n) = \Theta(n \log n).$$

□

Example 7.1.6. จงพิสูจน์ closed form ที่ได้ด้วย induction

Proof idea. เพราะเราเดาสู่ตรจากกรณี $n = 2^k$ การพิสูจน์ที่เป็นธรรมชาติที่สุดคือพิสูจน์บนตัวแปร k โดยกำหนด

$$S(k) : T(2^k) = k2^k - 2^k + 1.$$

Proof. ให้

$$S(k) : T(2^k) = k2^k - 2^k + 1.$$

ขั้นฐาน: เมื่อ $k = 0$,

$$T(2^0) = T(1) = 0$$

และด้านขวาเป็น

$$0 \cdot 2^0 - 2^0 + 1 = 0 - 1 + 1 = 0.$$

ดังนั้น $S(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$T(2^k) = k2^k - 2^k + 1.$$

ต้องการพิสูจน์ว่า

$$T(2^{k+1}) = (k+1)2^{k+1} - 2^{k+1} + 1.$$

จาก recurrence,

$$T(2^{k+1}) = 2T(2^k) + (2^{k+1} - 1).$$

แทนสมมติฐานขั้นอุปนัยลงไป:

$$T(2^{k+1}) = 2(k2^k - 2^k + 1) + 2^{k+1} - 1.$$

จัดรูปได้เป็น

$$T(2^{k+1}) = k2^{k+1} - 2^{k+1} + 2 + 2^{k+1} - 1 = k2^{k+1} + 1.$$

และ

$$(k+1)2^{k+1} - 2^{k+1} + 1 = k2^{k+1} + 2^{k+1} - 2^{k+1} + 1 = k2^{k+1} + 1.$$

ดังนั้น

$$T(2^{k+1}) = (k+1)2^{k+1} - 2^{k+1} + 1.$$

จึงได้ว่า $S(k+1)$ เป็นจริง

สรุปด้วย induction ว่า

$$T(2^k) = k2^k - 2^k + 1$$

สำหรับทุก $k \geq 0$ หรือเท่ากับ

$$T(n) = n \log_2 n - n + 1$$

เมื่อ n เป็นกำลังของ 2

□

7.2 Binary Search recurrence

ตัวอย่างก่อนหน้าของ Merge Sort มีลักษณะว่า แยกปัญหาออกเป็นสองส่วนย่อย แก้ทั้งสองส่วน แล้วค่อยรวมคำตอบกลับ แต่ Binary Search ต่างออกไป: มันยังแบ่งปัญหาเหมือนกัน แต่หลังจากเปรียบเทียบกับค่ากลางแล้ว เราทิ้งไปได้ครึ่งหนึ่งที่ ดังนั้น recursive call จะเหลือเพียงด้านเดียว

Pseudocode: Recursive Binary Search

```

RBSearch( $A, x, \ell, r$ ):
    if  $\ell > r$  then return 0
     $m \leftarrow \lfloor (\ell + r)/2 \rfloor$ 
    if  $A[m] = x$  then return  $m$ 
    if  $x < A[m]$  then return RBSearch( $A, x, \ell, m - 1$ )
    else return RBSearch( $A, x, m + 1, r$ )

```

Example 7.2.1. หา 5 จาก sorted array $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]$

จากตัวอย่างนี้จะเห็นว่า ทุกครั้งที่เปรียบเทียบกับค่ากลาง เราตัดทิ้งสมาชิกไปประมาณครึ่งหนึ่ง จึงเป็นธรรมชาติที่จะได้ recurrence ที่เกี่ยวกับ $n/2$

7.2.1 การวิเคราะห์

“ในกรณีแย่ที่สุด Binary Search ใช้การเปรียบเทียบกี่ครั้ง?”

ตั้งฟังก์ชันต้นทุน

ให้ $B(n)$ แทนจำนวน comparisons มากที่สุดที่ Binary Search ใช้กับลิสต์ sorted ขนาด n
 เพื่อให้ง่าย เราจะเริ่มจากกรณีที่ n เป็นกำลังของ 2 และใช้วิธีตัดแบ่งครึ่งเลย

Example 7.2.2. เพราะเหตุใด recurrence ของ Binary Search จึงควรเป็น

$$B(1) = 1, \quad B(n) = B(n/2) + 1 \quad (n \geq 2)?$$

Proof idea. ในกรณีแย่ที่สุด เราเปรียบเทียบกับสมาชิกตรงกลางก่อนหนึ่งครั้ง แล้วถ้ายังไม่พบ จะเหลือปัญหาย่อยเพียงครึ่งเดียวให้ทำต่อ

Proof. เมื่อลิสต์มีขนาด 1 ในกรณีแย่ที่สุดเราต้องเปรียบเทียบกับสมาชิกตัวนั้นหนึ่งครั้ง จึงได้

$$B(1) = 1.$$

สำหรับ $n \geq 2$, Binary Search จะเลือกค่ากลางขึ้นมาเปรียบเทียบกับก่อนหนึ่งครั้ง ถ้ายังไม่พบคำตอบ เพราะลิสต์ถูกเรียงไว้แล้ว เราจะรู้ทันทีว่าควรไปค้นต่อเพียงฝั่งซ้ายหรือฝั่งขวา ซึ่งมีขนาดประมาณ $n/2$
 ดังนั้นต้นทุนในกรณีแย่ที่สุดจึงเท่ากับ

$$\text{ต้นทุนของปัญหาย่อยขนาด } n/2 \quad + \quad \text{comparison ปัจจุบันอีก 1 ครั้ง}$$

หรือเขียนเป็น

$$B(n) = B(n/2) + 1.$$

□

7.2.2 Plug-and-chug

Example 7.2.3. จงแทนค่าออกกลับ:

$$B(n) = B(n/2) + 1.$$

Proof idea. เนื่องจากแต่ละขั้นหารขนาดปัญหาด้วย 2 จำนวนครั้งที่ทำได้ก่อนถึง 1 จึงควรสัมพันธ์กับ $\log_2 n$

Proof. เริ่มจาก

$$B(n) = B(n/2) + 1.$$

แทนค่าต่อ:

$$B(n) = B(n/4) + 2.$$

อีกหนึ่งขั้น:

$$B(n) = B(n/8) + 3.$$

จึงเห็น pattern ว่าหลังแทนไป i ชั้น

$$B(n) = B\left(\frac{n}{2^i}\right) + i.$$

เมื่อ $n = 2^k$ และเลือก $i = k$ จะได้

$$\frac{n}{2^k} = 1.$$

ดังนั้น

$$B(n) = B(1) + k = 1 + k.$$

และเพราะ $k = \log_2 n$ จึงได้ว่า

$$B(n) = \log_2 n + 1.$$

ดังนั้น Binary Search โตรระดับ

$$B(n) = \Theta(\log n).$$

□

สิ่งที่ควรได้

ถ้า $n = 2^k$ จะได้

$$B(n) = \log_2 n + 1.$$

ดังนั้น Binary Search โตรระดับ

$$B(n) = \Theta(\log n).$$

Exercise 7.2.4. เปรียบเทียบ Merge Sort กับ Binary Search:

1. เหตุใดทั้งสองตัวอย่างจึงเป็น recursion เหมือนกัน แต่ได้อัตราการเติบโตไม่เหมือนกัน
2. ใน recurrence ของ Merge Sort มีส่วนใดที่ ``แพงเพิ่ม" เมื่อเทียบกับ Binary Search

7.3 From Recurrence to Closed Form and Back Again

ถึงจุดนี้ควรเห็นลำดับการทำงานของหัวข้อนี้ชัดเจนแล้ว:

1. เริ่มจากปัญหาหรืออัลกอริทึม
2. อ่านโครงสร้างของมันให้ออก แล้วเขียนเป็น recurrence
3. แทนค่าถอยกลับเพื่อหา pattern
4. หา closed form
5. ใช้ induction พิสูจน์ว่าสูตรที่หาได้นั้นถูกต้องจริง

กล่าวอีกแบบหนึ่ง recurrence relation เป็นเหมือน ``ภาษากลาง" ระหว่างตัวปัญหากับสูตรคำตอบ มันช่วยให้เราเห็นว่าคำตอบเกิดขึ้นอย่างไรที่ละขั้น ก่อนจะสรุปออกมาเป็นสูตรที่อ่านง่ายกว่าในตอนท้าย

ข้อสรุปของบทนี้

recurrence relation ไม่ได้มีเป้าหมายเพียงให้เราแทนค่าถอยกลับได้ แต่มีเป้าหมายให้เราเห็นว่า

- สูตรเกิดจากโครงสร้างของปัญหาอย่างไร
- closed form ที่เราได้ควรพิสูจน์กลับด้วย induction เสมอ
- recurrence คนละแบบ อาจให้การเติบโตต่างกันมาก

แม้จะดูคล้ายกันว่า ``เรียกตัวเองกับปัญหาที่เล็กลง"

ภาค III

ทฤษฎีกราฟเบื้องต้น

ภาค IV

การนับเบื้องต้น

